

OhLaLa: Combined Security against Region Fault Attacks in Constant Protection Order

Abstract. Ensuring rigorous protection against physical attacks that simultaneously combine passive side-channel observations with active fault injections is a recent and rapidly evolving research area. Polynomial masking is a promising countermeasure, as it provides both privacy against leakage and redundancy against faults. At CRYPTO 2023, Berndt et al. introduced the “LaOla” scheme, which protects arbitrary circuits against such attacks. However, their model tolerates only a global threshold of e faults across the entire circuit, which fundamentally limits its scalability. In this work, we overcome this restriction by extending fault robustness to the *region fault model*, where an adversary can inject up to e faults in each gadget. We present the first gadgets secure against combined leakage and fault attacks in this stronger model. This improvement allows us to tolerate up to $x \cdot e$ faults across a circuit with x gadgets instead of just a total of e as in previous works. Consequently, the protection order e can remain constant. To achieve this, we introduce sufficient and practically attainable security definitions that are strictly stronger than those of Berndt et al. We further apply this principle to the LaOla construction, proving our stronger security notions for the adapted multiplication gadget.

1 Introduction

Many, if not most, vulnerabilities in cryptographic systems turn out to be related to the concrete implementation instead of the underlying theoretical construction. Two of the most prevalent *physical* attacks on implementations are *passive* side-channel attacks and *active* fault attacks. In the former, the attacker is able to passively obtain some information about the internal state of the implementation, e.g., by measuring the timing behavior [29], the power consumption [30], or the electromagnetic radiation [26]. In the latter, the attacker actively tampers with the internal state of the implementation by changing variables or disrupting computations, e.g., by using electromagnetic pulses [15,19], clock glitches [17], or voltage glitches [37]. While both of these attacks have been studied in detail on an individual basis, the study of attacks combining both has started more recently. Recent research has identified two main countermeasures to protect against combined attacks: In the first approach [16,23,22,24], an implementation is first hardened against passive side-channel attacks, e.g., by applying additive masking. Then, replication is introduced to prevent active fault attacks, e.g., by duplicating the hardened implementation and adding a consistency check to correct faults. In the other approach [33,12,4,21], the individual countermeasures against these two kinds of attacks are intertwined more carefully by using a

masking approach that simultaneously offers privacy against side-channel attacks and redundancy against fault attacks. Polynomial masking in the form of Reed-Solomon codes belongs to the latter category. While earlier approaches [33] took inspiration from classical MPC techniques, the resulting computational overhead and the number of shares used were typically too high to be used in practice. Berndt et al. [12] showed an alternative approach to reduce both the computational overhead and the number of shares. Besides providing a formal definition of how to capture security against combined attacks by lifting the security against individual attacks under certain circumstances, the authors provided a novel multiplication approach called *LaOla*. In turn, these results were improved by Arnold et al. [4] by embedding two secrets per sharing and by Faust et al. [21] by showing that the classical approach of packed secret sharing [25], i.e., embedding even more secrets per sharing, can also be used in *LaOla*. Furthermore, recent work by Arnold *et al.* [3] presents an efficient and executable implementation that achieves further improvements in both computational complexity and randomness consumption for *LaOla*. Despite these advances in concrete efficiency, the security analyses of these schemes are carried out within a common computational framework. In particular, both duplicated masking and polynomial masking are studied in a circuit-based security model that formalizes computation and adversarial interaction at the level of wires and gates.

Security Model. Research on duplicated and polynomial masking models computation as a circuit, in which every intermediate computational value is represented by a wire, and each operation (e.g., addition and multiplication) is described as a gate that takes input wires and produces output wires. While the line of research on duplicated masking uses Boolean circuits, i.e., logical AND and XOR gates, to more realistically model hardware computation, this work follows the line of research on polynomial masking and considers arithmetic circuits over a finite field $\mathbb{F}$, i.e., field addition and multiplication gates. They establish combined probing and fault resilience by formally proving that an adversary cannot influence the computation in an undetected manner or obtain any information about the underlying secrets, such as a secret key, even when independently faulting up to e wires and probing up to d wires simultaneously. In other words, they show resilience against an active adversary that can manipulate up to e and learn up to d intermediate values during the computation. This (d, e) -combined attack motivates the design of generic countermeasures that provide such combined probing and fault resilience.

Countermeasures. To mitigate such attacks, the original (unprotected) circuit is transformed into a circuit that is secure against these adversaries. For instance, in duplicated masking, the original circuit is first transformed into a leakage-resilient circuit using an additive masking scheme, where each sensitive value is split into $d+1$ uniform random shares whose sum equals the original value. Then, the addition and multiplication gates are replaced by circuits, called *gadgets* that compute on encodings of secret values, denoted *sharings*. This transformation ensures security against probing attacks on up to d wires. The resulting masked

computation is then duplicated $e+1$ times and combined with consistency checks, yielding a construction with $n = de + e + d + 1$ shares per input that additionally provides resilience against up to e faults. In polynomial masking, each sensitive value is represented as the evaluation of a random polynomial of degree d at $n = d + e + 1$ public points, while the secret itself is defined as the polynomial evaluated at a different point (e.g., zero). This approach naturally combines two fundamental coding-theoretic primitives: Reed-Solomon codes, which provide error detection against up to e faulty shares, and Shamir’s secret sharing, which ensures privacy against an adversary observing up to d shares. This work focuses on polynomial masking with Shamir’s Secret Sharing and even d, n [34].

Polynomial Masking. Given a secret value $s \in \mathbb{F}$, a polynomial $f(X)$ of degree at most d is constructed by setting the constant coefficient f_0 to s and choosing the remaining coefficients $(f_i)_{i \in [d]}$ uniformly and independently at random from $\mathbb{F}$. The corresponding shares $(F_i)_{i \in [n]}$ are obtained by evaluating $f(X)$ at n distinct, predetermined non-zero support points $(x_i)_{i \in [n]}$ s.t. $F_i = f(x_i)$. *Privacy*, i.e., security against d probes, follows from the fact that for any subset of d shares encoding a secret value $\tilde{s}$, there exists another polynomial of degree d that produces the same shares, but these encode a distinct secret value $\tilde{s}' \in \mathbb{F}$. Gadgets computing on shares are secure when preserving this indistinguishability by ensuring that any d wires do not yield any information on the encoded secret value. The strong non-interference (SNI) security notions [5,6] formally express this privacy property and guarantee its preservation under composition. *Fault robustness*, i.e., security against e faults, comes from the fact that a valid sharing corresponds to a codeword of a Reed-Solomon code, which is known to allow for the detection of up to $e = n - d - 1$ errors. In a valid sharing, the higher-order coefficients $(f_i)_{i \in [n], i > d}$ are zero. Intuitively, faulting at most e out of $n > d + e$ shares of a degree- d encoding leads to a higher degree. This means that at least one of the $e = n - d - 1$ redundant coefficients, $f_{d+1}, \dots, f_{n-1}$, is non-zero, which allows for detection or even correction of faults.

Protecting against combined attacks is even more involved, as faults might yield sensitive information, e.g., faulted sharings leaking information through probes. Hence, gadgets claiming combined security need to be carefully designed to prevent attacks exhaustively. Addition gates can be replaced by share-wise additions trivially. However, a share-wise multiplication of sharings $(F_i)_{i \in [n]}$ and $(G_i)_{i \in [n]}$ corresponding to degree d polynomials $f(X)$ and $g(X)$ produces a sharing of the product polynomial, i.e., $F_i \cdot G_i = f(x_i) \cdot g(x_i) = (f \cdot g)(x_i)$ of degree $2d$. Consequently, a degree reduction must be applied to re-obtain a degree d sharing. Moreover, $n > 2d$ is required for correctness.

The degree reduction can be applied either before or after the share-wise multiplication. In the former case, the input sharings are first transformed into multiple privacy-preserving polynomials of degree $\frac{d}{2}$, following the recently introduced LaOla approach [12]. Alternatively, degree reduction can be performed after the multiplication using the classical BGW technique [11]. Notably, the

LaOla approach requires only $n \geq d + 1$ shares, whereas the classical approach requires $n \geq 2d + 1$ shares throughout the entire protected circuit.

Comparison of Masking Paradigms and Open Problems. Polynomial masking achieves a tight bound on the number of required shares, namely $O(e + d)$, whereas the duplication approach requires $O(e \cdot d)$ shares. Furthermore, the duplication approach requires numerous intermediate error-correction algorithms [16,23,22], making the gadgets relatively inefficient compared to polynomial masking, as shown by [21]. However, one of the main limitations of both polynomial and duplicated masking so far, is that the analysis of these countermeasures has only been carried out in the fault-threshold setting, where the adversary is restricted to faulting at most e values in total, regardless of the overall circuit size. While Berndt et al. [12] have shown that their construction can also tolerate up to d probes in each gadget, thus allowing a significantly larger number of probes that scales with the circuit size, they still require a global fault threshold over the entire circuit. As a result, the overall fault tolerance does not scale with the size of the circuit. It remains unclear how these constructions behave when the amount of faults increases with the circuit size. Additionally, Berndt *et al.* [12] propose an efficient technique for fault detection based on analyzing how faults propagate through the circuit. This technique allows one to significantly reduce the cost of error-detection mechanisms. However, their formal proofs rely on an independence assumption on the propagation of different faults, which we show to not always hold and therefore requires more careful investigation.

1.1 Contributions and Outline

In this work, we address the shortcomings outlined above. While, to the best of our knowledge, there is only one existing scheme that is secure in the combined threshold fault and region probing model, we additionally provide security in the region fault model. In particular, our main contribution is the design of gadgets that are secure under composition and tolerate up to e arbitrary wire-independent faults per gadget, including stuck-at-constant, bit-flip, and random faults. To this end, we revisit and strengthen the security definitions of [12] to capture region-fault security. Along this line, we further formalize their framework to incorporate independence properties that were not considered in the original analysis of fault propagation.

(1) *Region Fault Robust Gadgets.* To the best of our knowledge, we present the first gadgets that are secure against combined leakage and fault attacks, tolerating not only up to e faults in the circuit but also up to e faults in each gadget. In particular, for a total of x gadgets, our construction supports $x \cdot e$ faults overall, which constitutes a strict improvement over the state of the art, where only e faults in total are tolerated. Our results extend prior constructions that were secure in the combined threshold fault model and region probing model by additionally providing provable security in the region fault model. For this purpose, we extend the security definitions for combined security, namely *Fault*

Robustness and *Fault Invariance*, and formally prove that all proposed gadgets satisfy the corresponding security notions introduced in this work. We discuss these notions in more detail below.

(2) *Region Fault Robustness*. We extend the notion of fault robustness introduced by Berndt et al. [12] to enable secure composition in the region fault model, where the adversary may introduce up to e faults *per gadget* rather than only e faults in the entire circuit as considered in [12]. In this line of work, we first refine the original definition of fault robustness to provide stronger security guarantees and to correct a flaw that we identified in the general security proofs regarding fault robustness in [12]. Specifically, their analysis assumes that all faults in the gadgets are either ineffective, affect only a single output share, or have an independent random effect on the outputs. However, we show in Supp. J that this latter assumption does not always hold. While the evaluation in [3] shows that this mismatch is not exploitable yet for random faults, we resolve it in order to obtain a sound and provable notion of fault robustness for arbitrary faults. Building on this refinement, we then further extend the framework and define *region fault robustness* to formally capture region fault security. Additionally, we provide formal security proofs showing that our definitions support secure composition, namely that a composition of region fault robust gadgets remains region fault robust.

(3) *Circuit Fault Invariance*. We revisit and refine the notion of fault-invariance originally introduced by Berndt et al. [12]. Intuitively, a circuit is fault invariant when the effect of an intermediate fault can be described as either an input or an output fault. They showed that any circuit that is (S)NI and fault-invariant is perfectly simulatable in the presence of faults. However, fault-invariance can be considered for the circuit itself or for the leakage simulator. In [12] these notions were implicitly assumed to be equivalent, which is not necessarily true, as we show in Supp. I. To address this gap, we introduce the notion of *circuit fault-invariance*. We nevertheless observe that their gadgets also fulfill this stronger property, and provide more details in Supp. I.1. Moreover, we formally prove that a gadget that is (S)NI and circuit fault invariant with respect to additive faults is perfectly simulatable in the presence of additive faults. This result extends to wire-independent faults with only minor parameter adjustments, in line with [12]. Our proof strategy differs from prior work and relies on graph-theoretic methods, in particular on a construction inspired by maximal common subgraphs.

Outline. Notation, adversarial model and security notions for the original constructions and proofs of [12] are given in Section 2. In Section 3, we introduce the definitions of random and region fault robustness. We prove the composability of gadgets and bounds on attack success probabilities. In Section 4, we introduce our adapted notion of fault invariance. Moreover, we show in this section that any SNI circuit that fulfills to our adapted fault invariance has the properties originally, but incorrectly, derived in [12].

In Supp. J, we provide an example to show that the original LaOla construction does not satisfy threshold fault robustness due to dependencies among high-order coefficients. We provide a method to eliminate these dependencies in Sections 5 and 6. Section 5 introduces our new refresh gadget, denoted **P-Refresh**, which ensures that any error affects *all* high-order coefficients. We prove that **P-Refresh** satisfies SNI, circuit fault invariance, and fault robustness. Section 6 presents the secure multiplication gadget **Mult — LaOla** which also satisfies SNI, circuit fault invariance, and threshold fault robustness.

In summary, we introduce new definitions and techniques to provably ensure that faults affect all higher-order coefficients in a randomized and secret-independent manner. We apply these techniques to multiplication and refresh gadgets, for which we obtain provably secure constructions against combined leakage and fault attacks in the region setting.

1.2 Related Work

Although probing security and fault resilience have been studied extensively, efficient defenses against their combination are not known in the region fault setting, where an adversary can place in each region a given number of faults, and have only recently been achieved in the threshold combined setting. Accordingly, the works discussed below are restricted to the weaker threshold model. From a technical standpoint, combined security is achieved either through duplicated Boolean masking [16,23,32,22,24] or polynomial masking [12,4,21,3].

For a more detailed overview of duplicated masking, we refer to Feldtkeller *et al.* [24]. In a nutshell, Dhooghe and Nikova [16] proposed duplicated masking to realize composable computation-with-abort. However, these abort constructions rely on an error-detection mechanism that is generally difficult to realize in practice, and Richter-Brockmann *et al.* [32] identified a security flaw in one of the proposed gadgets. In subsequent work, Feldtkeller *et al.* [23] proposed composable gadgets based on an approach that isolates the impact of faults to a limited number of output wires. However, their analysis did not consider the increased leakage caused by ineffective fault propagation, which was later addressed by Feldtkeller *et al.* [22], but at the cost of less efficient constructions. Alternatively, Tollec *et al.* [35] proposed the Fault-Resistant Partitioning approach, in which faults cannot propagate to other partitions without being detected. More recently, Toprakhisar *et al.* [36] proposed combined stability as an alternative composition property that only requires error detection at the end of the computation.

Duplicated masking is particularly well suited for hardware implementations, as it works with binary masking and allows handling hardware-specific behaviors such as glitches, transitions, and couplings. However, it also suffers from the drawback of requiring a large number of shares, i.e. $\mathcal{O}(de)$ shares for each input. While this approach is inefficient for linear and affine gadgets due to the large number of shares [12], it performs well for sequential computations over binary fields [21]. In contrast, polynomial masking schemes rely on Shamir’s secret sharing [34]. The key intuition is that this encoding inherently provides both leakage

and fault-resilience properties. These schemes achieve combined security using $\mathcal{O}(d + e)$ shares [12], and further allow the embedding of multiple secrets [4,21].

One of the first steps toward a fault model that scales with circuit size was taken by Belaïd *et al.* [9], who introduced a general random fault model as well as a general random combined model. However, their focus was not on the design of secure gadgets, but rather on developing verification tools for this model. While they consider an adversary that faults every value with some noise, we consider an adversary that can fault specific values with concrete values of its choice. In particular, our model captures faults such as bit flips or setting values to zero.

An interesting direction for future work is to investigate how these models relate to each other, in analogy to the relationships established for standalone leakage models [18,20,31,13]. A first step in this direction was taken by Cassiers *et al.* [14], which shows the relationship between the uniform random fault model, where all faults are placed using the same random fault function, and the p -random fault model, where faults are injected only with some success probability. Finally, we note that in the context of standalone leakage, a common modeling approach already follows the region paradigm, where the adversary is allowed to probe a bounded number of wires in each region of the circuit [28,2,8].

2 Preliminaries

The zero vector with $n \in \mathbb{N}$ components is denoted by $\mathbf{0}_n = (0, \dots, 0)^T \in \mathbb{F}^n$, where $\mathbb{F}$ denotes a finite field. For $n, a, b \in \mathbb{N}$, let $[n] := \{0, \dots, n-1\}$ and $[a, b] := \{a, \dots, b-1\}$. The j -th row of a matrix M is denoted by $M_{j,:}$ or $M_{[j,:]}$.

The Vandermonde matrix corresponding to the pairwise distinct, non-zero support points $(x_i)_{i \in [n]}$ is denoted by $\text{Van}_{[n] \times [n]} = (x_0, \dots, x_{n-1})$, or V . A Vandermonde matrix with pairwise distinct non-zero support points is always invertible [1]. Hence, the inverse of V is denoted as $V^{-1} := (\lambda_{i,j})_{i \in [n], j \in [n]}$.

The coefficients of polynomials $f(X)$ are denoted by $\mathbf{f} = (f_0, \dots, f_{n-1})^T$ and we assume that all coefficients are in the finite field $\mathbb{F}$. Shares $\mathbf{F}' = (F'_0, \dots, F'_{n-1})^T$ are called *valid* shares (or valid encoding) if they correspond to the polynomial $f'(X) = f'_0 + \dots + f'_d X^d$ of degree d , i.e., $F_i = f(x_i)$ for pairwise distinct, non-zero support points $x_0, \dots, x_{n-1}$. Hence, we obtain $f_j = \sum_{i=0}^{n-1} F_i \cdot \lambda_{j,i}$ if $F_j = \sum_{i=0}^{n-1} f_i \cdot x_i^j$, and vice versa. Shares are called *non-valid* shares if the corresponding polynomial has a degree higher than d . Notably, n denotes the number of shares, d the upper bound for the degree of valid encodings, and e the number of faults we aim to tolerate. Unless stated otherwise, we assume $n \geq d + e + 1$.

The degree of a sharing $(F_i)_{i \in [n]}$ can be reduced from d_1 to d_2 by multiplying with matrix $(\lambda_{i,j}^{(d_1 \rightarrow d_2)})_{i,j \in [n]^2}$, defined in Supp. K. Intermediate (or internal) faults are faults within a gadget, excluding those on its inputs or outputs.

We define *d-wise uniformly independent* random variables as a tuple of random variables where any d -tuple is independently and uniformly distributed in $\mathbb{F}^d$. The uniform distribution is denoted as U . Eager sampling refers to the process of drawing all random values at the beginning of an algorithm, in contrast

to the traditional lazy sampling, where values are generated when accessed. The distance of distributions $\mathcal{D}_1, \mathcal{D}_2$ defined on sample space Ω is defined as $\Delta(\mathcal{D}_1, \mathcal{D}_2) = \max_{X \subset \Omega} |\Pr_{\mathcal{D}_1}(X) - \Pr_{\mathcal{D}_2}(X)|$ and $\mathcal{D}_1 \equiv \mathcal{D}_2 \Leftrightarrow \Delta(\mathcal{D}_1, \mathcal{D}_2) = 0$. A distribution D is ϵ -indistinguishable iff $\Delta(D, U) \leq \epsilon$.

Let A, A', B be sets such that $A' \subseteq A$. If $f: A \rightarrow B$ is a function from A to B , then the *restriction* of f to A' is the function $f|_{A'}: A' \rightarrow B$ given by $f|_{A'}(x) = f(x)$ for every $x \in A'$. $\text{weight}(\mathbf{Y})$ refers to the hamming weight of vector $\mathbf{Y}$, i.e., the number of non-zero elements.

Lemma 1 (Fundamental Lemma). *Let $f \in \mathbb{F}[X]$ be a polynomial of non-zero degree. If f has ℓ distinct roots, $\deg(f) \geq \ell$.*

2.1 Attacker Model

Our adversary model is as in [12] and considers a combined adversary model involving both passive and active attacks on a circuit. A passive adversary observes the input-output behavior, as well as values of some internal wires. An active adversary injects faults into internal values, as well as the in- and outputs of a gadget. Moreover, adversaries are able to simultaneously probe and fault the values of the implementation. This model is strictly stronger than treating probing and faulting separately, as the adversary can adapt fault injections based on previously observed values.

Computations are represented as circuits C over a finite field $\mathbb{F}$, i.e., a directed acyclic graph G (cf. Definition 17). We recall definitions for directed acyclic graphs (DAGs) and their topological sorting in Supp. C. Each node (vertex) in G is an input gate, output gate, random gate, addition gate, multiplication gate, or constant transformation gate. Random gates have no parent node and output uniform and independent random elements from $\mathbb{F}$. Constant can appear in constant transformation gates only. These have two constants a and b and output $ax + b$ on input x . A gadget G is a circuit that returns k valid output sharings for any l valid input sharings for fixed $k > 0, l \geq 0$. A *region* R is a subset of wires of a circuit. The semantic computation of a circuit C proceeds by iteratively assigning values to output wires of gates where all input wires have values. We denote by $C^R((\mathbf{Y}_i)_{i \in [l]})$ the joint distribution of the outputs of a circuit (or gadget) for inputs $(\mathbf{Y}_i)_{i \in [l]}$ and independently and uniformly distributed randomness R , of which may be fixed within the context. Similarly, $G^R((\mathbf{F}_i)_{i \in [l]})$ or $G(R, (\mathbf{F}_i)_{i \in [l]})$, refers to the evaluation of the gadget (or graph) G using fixed randomness R and input sharings $(\mathbf{F}_i)_{i \in [l]}$, that is, each wire is assigned a concrete value determined by the inputs $(\mathbf{F}_i)_{i \in [l]}$ and randomness R .

A compiler transforms a circuit C into a circuit C' that is secure in our attacker model.

A *passive t -probing adversary* $\mathcal{A}$ can observe the values of up to $|I| \leq t$ intermediate wires $(W_i)_{i \in [n]}$ during the execution of a circuit. Let $b \leftarrow^* \{0, 1\}$ and let the adversary choose two inputs Y_0 and Y_1 . A circuit C on input Y_b is *perfectly secure* against a t -probing adversary $\mathcal{A}$, if the probability that the adversary $\mathcal{A}$ cannot distinguish whether $b = 0$ or $b = 1$ with probability greater than $\frac{1}{2}$.

Definition 1 (NI, SNI [5,6]). A gadget G is t -non-interfering (t -NI) if for any set of t_{int} intermediate variables and any subset $\mathcal{O}$ of output indices with $t_{\text{int}} + |\mathcal{O}| \leq t$, there exists a subset of input indices I with $|I| \leq t_{\text{int}} + |\mathcal{O}|$ such that the distribution of the t_{int} intermediate and the output variables in $\mathcal{O}$ is perfectly simulatable from inputs with indices in I . G is t -strong non-interfering (t -SNI) if $|I| \leq t_{\text{int}}$.

When considering *active attacks*, we focus on the general class of wire independent faults $\mathcal{F}^{\text{ind}} := \{\text{all functions } \xi : \mathbb{F} \rightarrow \mathbb{F}\}$, where a wire with value x can be arbitrarily faulted but independently of the values of other wires in the circuit. We further consider a subset of wire-independent faults called additive faults, which is defined as $\mathcal{F}^+ := \{\xi : \xi(x) = x + a \text{ for all } a \in \mathbb{F}\} \subset \mathcal{F}^{\text{ind}}$.

A fault attack $T = (\xi_0, \dots, \xi_\ell)$ is a tuple of functions in $\mathcal{F}$ that specifies how each wire is faulted. Notably, we consider *wire* faults. This means that for a gate with multiple output vertices, we either assume a fault on the input, causing all outputs to be affected, or a fault occurring on a single output wire at a time, corresponding to a fault injected during the *transport of information* along an edge.

We denote the set of all possible fault attacks T using functions in $\mathcal{F}$ by $\mathcal{A}(\mathcal{F})$. Moreover, for a fault attack $T = (\xi_0, \dots, \xi_{\ell-1})$ we define $|T|$ to be the number of non-identity components, namely $|T| = |\{i \in [\ell] \mid \xi_i \neq \text{id}\}|$.

Combined Attacks. We restate the notions of privacy in settings with faults.

Definition 2 (Fault-resilient (S)NI [12, Def. 8]). A gadget G is d -fault-resilient (strong-) non-interfering (d -fr(S)NI) with respect to a fault set $\mathcal{F}$ if $T[G]$ is d -(S)NI for any fault attack $T \in \mathcal{A}(\mathcal{F})$.

Definition 3 (Weak fault-resilient NI [12, Def. 6]). A gadget G is d -weak-fault-resilient non-interfering (d -wfrNI) with respect to $\mathcal{F}$ if every set of d' probes in $T[G]$ can be (perfectly) simulated with $d' + |T|$ shares of each input sharing for any fault attack $T \in \mathcal{A}(\mathcal{F})$ with $|T| + d' \leq d$.

Definition 4 (Weak fault-resilient SNI [12, Def. 7]). A gadget G is d -weak-fault-resilient strong-non-interfering (d -wfrSNI) with respect to $\mathcal{F}$ if every set of d_1 internal probes and d_2 output probes in $T[G]$ can be (perfectly) simulated with $d_1 + \epsilon_1$ shares of each input sharing for any fault attack $T \in \mathcal{A}(\mathcal{F})$ with ϵ_1 internal faults and ϵ_2 output faults with $d_1 + d_2 + \epsilon_1 + \epsilon_2 \leq d$.

3 Fault Robustness

We consider the notion of *fault robustness*, which describes how faults injected into the inputs and during the gadget's computation propagate to its outputs. Informally, the resulting faults in the output are decomposed into a linear part that is controlled by the attacker but has bounded weight, and a non-linear part that is randomized and outside the attacker's control.

This section is organized as follows. Section 3.1 consists of an informal discussion on fault robustness and includes an example of how faults propagate through a fault robust gadget. Next, Section 3.2 revisits and adapts the definitions of fault robustness in the threshold model from [12]. Finally, Section 3.3 presents our new notion of *region* fault robustness.

3.1 An Intuitive Overview on Fault Robustness

Berndt et al. [12] defined the notion of fault robustness, which guarantees that any non-trivial fault can be detected. We keep their idea, but strengthen it to ensure important independence assumptions that enable security against *arbitrary* faults and smooth composability.

We begin by considering two kinds of effects that t injected faults on the inputs might induce: First, we have effects that can be *isolated* to at most t output shares and thus propagate through a gadget in a linear manner, i.e., t faulted output shares correspond directly to t faulted input shares. We call such an effect to be *linear*. However, for non-linear gadgets, a single input share might contribute to different output shares and a single fault can thus affect more than one or even all output shares simultaneously. Hence, having e redundant shares is not sufficient to guarantee error detection. Our core idea is to design gadgets such that all faults that affect multiple shares lead to a *rerandomization* effect, where every high order coefficient (with indices in $[d+1, n]$) of the underlying polynomial is randomized independently from the introduced faults. We aim to stop a deterministic fault propagation of the inputs in the non-linear scenario, as it can cause problems with statistical ineffective faults (an example for such a problematic deterministic fault propagation is presented in Supp. J). In the context of region security, we extend this requirement to linear computations as well, in order to prevent faults from accumulating across regions. We can stop a deterministic fault propagation from the inputs by randomizing all high order coefficients (i.e. coefficients with indices $d+1, \dots, n-1$) of the underlying polynomial. Thus, it becomes highly unlikely that the output corresponds to a valid encoding, and consequently, such faults propagate with sufficiently high probability, which in turn allows for error detection. We construct our gadgets to ensure that the effect of *every* (input or internal) fault can be partitioned into a *deterministic* effect of sufficiently low weight and a *rerandomization* effect that randomizes the output.

As an intuitive example, we will present a (simplified) version of our refresh gadget and refer to Section 5 for further details. For simplicity, we restrict ourselves to wire-independent additive faults in the following example. Our starting point is the standard design of a refresh gadget, which takes an input sharing $\mathbf{F}$ and outputs another sharing $\mathbf{S}$ of the same secret by adding a so called zero-encoding. Namely, we create $r(X) = 0 + r_1X + \dots + r_dX^d$ with $r_i \leftarrow_{\$} \mathbb{F}, i \in [1, d+1]$, evaluate it to obtain $\mathbf{R}$ and set $\mathbf{S} = \mathbf{F} + \mathbf{R}$. Thus, the polynomial underlying $\mathbf{S}$ is $s(X) = f(X) + r(X)$ and by construction it holds that $s(0) = f(0) + r(0) = f(0)$. We now discuss each position where an attacker could inject faults, and the resulting propagation. An attacker could either inject up to

t faults directly to $\mathbf{S}$, but this only affects up to t shares of the output $\mathbf{S}$, making the fault effect *deterministic*. Another position to inject a fault is to modify the coefficients of r . Here, a single modification, e.g., of r_1 can influence *all* output shares, due to the evaluation of the now changed polynomial $r(X)$. However, since all coefficients r_i are sampled independently uniformly at random, their distribution remains unchanged under additive wire-independent faults. Finally, the attacker might fault the input $\mathbf{F}$. Since the output $\mathbf{S}$ is obtained by adding the input $\mathbf{F}$ to the zero encoding $\mathbf{R}$, the input faults propagate deterministically through the gadget. However, region fault robustness requires re-randomization of input faults. To guarantee this, we not only add $r(X)$ to $f(X)$, but also randomize *all* high order coefficients of $s(X)$, whenever at least one high order coefficient of $f(X)$ is non-zero. Concretely, for $i \leq d$ we set the i -th coefficient of $s(X)$ as $s_i = f_i + r_i$, and for $i \in [d+1, n]$ we set $s_i = f_i + r_i + \sum_{j \geq 1} f_{d+j} r_{d+j}^{(i)}$ where $r_{d+j}^{(i)} \leftarrow \mathbb{F}$ are fresh, uniform elements. If $\mathbf{F}$ was not faulted, $f_{d+j} = 0$ for all j , so the result is not changed, i.e., $s(X) = f(X) + r(X)$. Otherwise, there exists a coefficient $f_{d+j^*} \neq 0$ for at least one $j^* \geq 1$ (by the Fundamental Lemma 1) and thus the resulting coefficients s_{d+j} of s are uniformly distributed. Hence, the refresh gadget *rerandomizes* the higher-order coefficients of the faulted inputs, leading to an invalid sharing (with very high probability).

In summary, every fault introduced into our refresh gadget either has a *deterministic* effect (which can be represented as the addition of a low-weight vector to a sharing) or a *rerandomizing* effect on the higher-order coefficients (which can be represented as the addition of a random vector to the polynomial coefficients). Finally, note that the interpolation of $\mathbf{F}$, addition of polynomial coefficients r , and the evaluation of s can also be transformed into a single operation to prevent certain attacks.

3.2 Threshold Fault Robustness

We begin by introducing the concept of fault vector, describing how faults alter the structure of (input or output) sharings.

Definition 5 (e -deterministic fault vector). An e -deterministic fault vector $\mathbf{v}$ has $\text{weight}(\mathbf{v}) \leq e$.

Definition 6 ((e, ϵ) -fault vector). An (e, ϵ) -fault vector $\mathbf{v}$ is defined as $\mathbf{v} = \mathbf{v}' + \mathbf{v}''$ where $\mathbf{v}'$ is an e -deterministic fault vector and $\mathbf{v}''$ is either the zero vector or an encoding $\bar{\mathbf{v}}$ such that the high-order coefficients $p_{d+1}^{(\bar{\mathbf{v}})}, \dots, p_{n-1}^{(\bar{\mathbf{v}})}$ of the underlying polynomial $p^{(\bar{\mathbf{v}})} \in \mathbb{F}_q[X]$ are $\mathcal{D}_{\mathbf{v}}$ -distributed, where $\mathcal{D}_{\mathbf{v}}$ is ϵ -indistinguishable and independent of $\mathbf{v}'$.

We discuss the role of the parameters e and ϵ in the detection probability of faults. Consider the deterministic case, i.e., $\mathbf{v}' \neq 0, \mathbf{v}'' = 0$ for $e < n - d$. Then, Lemma 1 guarantees that the overall fault $\mathbf{v} = \mathbf{v}' + \mathbf{v}''$ corresponds to a polynomial with degree at least $n - \text{weight}(\mathbf{v}) = n - e$, which is strictly larger than d by construction. Therefore, the fundamental lemma (cf. Lemma

1) ensures that detection certainly succeeds. Now, assume $\epsilon = 0$ for clarity and consider the case where $\mathbf{v}'' = \bar{\mathbf{v}}$ is nonzero. Then, the fault vector $\mathbf{v}$ is a valid encoding iff all the high-order coefficients $p_{d+1}^{(\bar{\mathbf{v}})}, \dots, p_{n-1}^{(\bar{\mathbf{v}})}$ of $\bar{\mathbf{v}}$ and those of $\mathbf{v}'$ sum up to zero component-wise, which happens with probability $|\mathbb{F}|^{-(n-d-1)}$. Consequently, with probability $1 - |\mathbb{F}|^{-(n-d-1)}$ the overall fault vector $\mathbf{v}$ is an invalid encoding and can be detected. This probability decreases with increasing ϵ in the ϵ -indistinguishability of $\mathbf{v}''$.

We now have the preliminaries needed to define our two notions of fault robustness as follows.

Linear Fault Robustness. Linear fault robustness relies on the share-wise nature of linear gadget computations, where each fault propagates in isolation.

Definition 7 (e -linear fault robustness). *Gadget G with l inputs and 1 output is e -linear fault-robust with respect to fault set $\mathcal{F}$ if for any tuple of fault vectors ξ_i for $i \in [l]$, for any fault attack on internal and output wires $T \in \mathcal{A}(\mathcal{F})$ with $|T| \leq e$ and $(\mathbf{Y} + \omega + \omega') \equiv T[G^R]((\mathbf{F}_i + \xi_i)_{i \in [l]})$ where $\mathbf{Y} := G^R((\mathbf{F}_i)_{i \in [l]})$*

- i) $\text{weight}(\omega) \leq |T|$ (internal linear part),
- ii) ω' is equal to a linear combination of $\xi_0, \xi_1, \dots$ (input linear part).

Clearly, e -linear fault robustness is preserved under gadget composition.

Lemma 2. *Let $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ be a set of wire-independent faults. The composition of two e -linear fault robust gadgets w.r.t. $\mathcal{F}$ is also e -linear fault robust w.r.t. $\mathcal{F}$.*

The proof appears in Supp. E.1 due to space constraints.

Threshold Fault Robustness. We generalize the original definition of e -fault robustness [12] to the notion of (e, δ, ϵ) -threshold fault robustness, where the entries of the non-linear randomized part are sampled from an ϵ -indistinguishable distribution (Section 2).

While the parameters e and ϵ were already discussed in the beginning of this section, we now introduce an additional parameter $\delta \in \mathbb{R}_{\leq 1}$, describing the fault robustness failure probability. Undetected propagation of faults is a security issue which cannot be mitigated in polynomial masking. This parameter therefore upper bounds the probability for such events, i.e., fault propagating into the outputs as valid fault vectors.

Intuitively, our notion of threshold fault robustness ensures that whenever the internals and inputs of a gadget are faulted within some bounds, then the output is faulted with a fault vector, except with probability δ . Our notion addresses gadgets with multiple inputs and one output. Looking ahead, we introduce joint fault vectors in order to obtain a composition result for our (random) fault-robustness. Specifically, the goal is to preserve the ϵ -indistinguishability of the high order coefficients of our fault-vectors without increasing ϵ . In order to obtain this, we require that the high-order coefficients of the randomized components of different inputs are mutually independent.

We state the relevant definitions before the actual definition of threshold fault robustness (cf. Definition 10).

Definition 8 (Joint $(e_i)_{i \in [k]}$ -deterministic fault vectors). A tuple $(\mathbf{v}'_i)_{i \in [k]}$ is called a joint $(e_i)_{i \in [k]}$ -deterministic fault vector if $\mathbf{v}'_i$ is an e_i -deterministic fault vector for every $i \in [k]$.

Definition 9 (Joint $((e_i)_{i \in [k]}, \epsilon)$ -fault vectors). Let $(\mathbf{v}_i)_{i \in [k]}$ be a tuple such that every $\mathbf{v}_i$ is an (e_i, ϵ) -fault vector with decomposition $\mathbf{v}_i = \mathbf{v}'_i + \mathbf{v}''_i$, for $i \in [k]$. They are joint $((e_i)_{i \in [k]}, \epsilon)$ -fault vectors if $(\mathbf{v}'_i)_{i \in [k]}$ are joint $(e_i)_{i \in [k]}$ -deterministic fault vectors and each $\mathbf{v}''_i$ is either zero or an encoding $\bar{\mathbf{v}}_i$ such that the high-order coefficients $p_{d+1}^{(\bar{\mathbf{v}}_i)}, \dots, p_{n-1}^{(\bar{\mathbf{v}}_i)}$ of the underlying polynomial $p^{(\bar{\mathbf{v}}_i)}$ are $\mathcal{D}_{\mathbf{v}_i}$ -distributed, where $\mathcal{D}_{\mathbf{v}_i}$ is a distribution that is ϵ -indistinguishable from uniform and independent of the other vectors.

When $e_i \leq e$ for every $i \in [k]$ we abbreviate and write (e, ϵ) -fault vectors, respectively e -deterministic fault vectors.

Definition 10 $((e, \delta, \epsilon)$ -threshold fault robustness). Let G be a gadget with l inputs and 1 output. Gadget G is (e, δ, ϵ) -threshold fault robust w.r.t. $\mathcal{F}$ if for any fault attack $T \in \mathcal{A}(\mathcal{F})$ on internal and output wires, and for any joint $((e_i)_{i \in [l]}, \epsilon)$ -fault vectors $(\xi_i)_{i \in [l]}$ with respect to distributions $\mathcal{D}_{\xi_i}$, s.t. $|T| + \sum_{i \in [l]} e_i \leq e$, there exists an (s, ϵ) -fault vector $\mathbf{v}$ with $s \leq |T| + \sum_{i \in [l]} e_i$, s.t. for any valid input $\mathbf{F}_i$ it holds with probability $1 - \delta$ that

$$T[G^R]((\mathbf{F}_i + \xi_i)_{i \in [l]}) \equiv G^R((\mathbf{F}_i)_{i \in [l]}) + \mathbf{v},$$

where the output distributions are induced by the randomness R . Moreover, we require that the distribution $\mathcal{D}_{\mathbf{v}}$ for the fault vector $\mathbf{v}$ only depends on $\mathcal{D}_{\xi_i}$, $i \in [l]$ and the internal randomness of the gadget.

We discuss the need for the failure probability δ in more detail. Consider a single-output threshold fault robust gadget whose input $\mathbf{F}$ is faulted with a non-zero $(e, 0)$ -fault vector ξ , which can be valid with probability $|\mathbb{F}|^{-(n-d-1)}$. If ξ is valid, the overall input is valid and leads by correctness to a valid output, which equals the unfaulted output plus a non-zero and valid output fault vector $\mathbf{v}_0$. This is a *deterministic* output fault vector and its validity implies that the underlying polynomial has degree at most d . However, it is not an $(e, 0)$ -fault vector, since by Lemma 1 it has at most d roots and hence $\text{weight}(\mathbf{v}_0) \geq n - d$. Consequently, the event where all high-order coefficients of the random fault vector ξ are zero must always be included in the failure probability δ , yielding $\delta \geq |\mathbb{F}|^{-(n-d-1)}$ for $\epsilon = 0$.

However, δ is not simply a function of e, ϵ, n, d , and $|\mathbb{F}|$. To see this, consider two $(e, \delta, 0)$ -threshold fault robust gadgets G_1 and G_2 and their composition $G = G_2 \circ G_1$. Assume that the input is faulted by an (e, ϵ) -fault vector, and the internal faults are all within G_1 . We now analyze the failure probability of the overall gadget G , i.e., the cases where the output is not faulted with a (e, ϵ) -fault vector. One case where this happens is when the output of G_1 is not an (e, ϵ) -fault vector, occurring when the fault robustness of G_1 fails. In this case, we cannot argue about G_2 's output because we don't know anything about its

input. The other case is the one where the output of G_1 is an (e, ϵ) -fault vector, but the fault robustness of G_2 fails. Therefore, the error probability of G is greater than that of G_1 and G_2 individually. This shows that δ also depends on the gadget's underlying components and the corresponding error probabilities.

In the following, we show that a linear fault-robust gadget is a threshold fault-robust gadget, which follows directly from the definitions.

Lemma 3. *Let $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ be a wire-independent fault set. Consider a gadget G with l input and 1 output sharings which is e -linear fault-robust with respect to $\mathcal{F}$. Then, G is (e, δ, ϵ) -threshold fault-robust with respect to $\mathcal{F}$, where δ denotes the probability that at least one of the joint $(e_i)_{i \in [l]}$ -fault vectors that are part of the input is a valid nonzero encoding.*

The proof of this lemma is deferred to Supp. E.2. We conclude this section by stating that a composition of (e, δ, ϵ) -threshold fault robust gadgets remains threshold fault robust with respect to the same parameters e and ϵ , and slightly adapted δ . The proof of the following lemma can be found in Supp. E.3.

Lemma 4. *Let $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ be a wire-independent fault set. Further, let G' be an (e, δ, ϵ) -threshold fault robust gadget with respect to $\mathcal{F}$ with k input sharings and one output sharing. For each $i \in [k]$, let G_i be an (e, δ, ϵ) -threshold fault robust gadget with respect to $\mathcal{F}$ and l_i input sharings. We assume that each gadget $G_0, \dots, G_{k-1}, G'$ takes independent randomness. Then the composition $G = G'((G_i(\cdot))_{i \in [k]})$ is an (e, δ', ϵ) -threshold fault robust gadget with respect to $\mathcal{F}$ and $\delta' = 1 - (1 - \delta)^{k+1}$.*

3.3 Region Fault Robustness

We now turn to a setting that more realistically captures practical attacks by partitioning circuits into regions and allowing adversaries to inject e faults in each region. Region fault robustness generalizes threshold fault robustness by ensuring protection against e faults per region instead of a total of e faults within the entire circuit.

Definition 11 ((e, δ, ϵ) -region fault robustness). *Let G be a gadget with l inputs and 1 output and let $(R_i)_{i \in [N]}$ be $N \geq 1$ disjoint regions within G . G is (e, δ, ϵ) -region fault robust with respect to $\mathcal{F}$ if for any fault attack $T \in \mathcal{A}(\mathcal{F})$ on internal and output wires with $|T \upharpoonright_{R_j}| \leq e$ for every $j \in [N]$ and for any joint (e, ϵ) -fault vectors $(\xi_i)_{i \in [l]}$ there exist a (s, ϵ) -fault vector $\mathbf{v}$, whose distribution $\mathcal{D}_{\mathbf{v}}$ only depends on the internal randomness of the gadget, such that for any valid input $(\mathbf{F}_i)_{i \in [l]}$ it holds with probability $1 - \delta$ that $G^R((\mathbf{F}_i)_{i \in [l]}) + \mathbf{v} \equiv T[G^R]((\mathbf{F}_i + \xi_i)_{i \in [l]})$, where $s \leq \max_{j \in N} |T \upharpoonright_{R_j}|$ and the output distributions are induced by the randomness R .*

We can also show that every threshold fault robust gadget can be compiled into a region fault-robust gadget without the loss of security depending on the number of inputs, and with only a slight computational overhead. Looking ahead,

this compiler essentially describes how we define regions throughout this work: inputs are first refreshed to randomize any deterministic fault, and only then are they used in computation.

Lemma 5. *Let $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ be a wire-independent fault set. Further, let G' be a (e, δ, ϵ) -region fault-robust gadget with respect to $\mathcal{F}$ with k input sharings and one output sharing. For each $i \in [k]$, let G_i be an (e, δ, ϵ) -threshold fault robust gadget with respect to $\mathcal{F}$ with l_i input and one output sharing. Moreover, let each gadget use independently sampled randomness. Then the composition $G = G'((G_i(\cdot))_{i \in [k]})$ is an $(\frac{e}{2}, \delta', \epsilon)$ -region fault robust gadget with respect to $\mathcal{F}$, $N = 1$ and $\delta' = 1 - (1 - \delta)^{k+1}$.*

Proof. In the $N = 1$ setting, the single region is the whole composed circuit. Fix any valid inputs $\mathbf{F}_i^j, i \in [k], j \in [l_i]$ and any attack vector T with $|T| \leq \frac{e}{2}$ that can fault internal and output wires anywhere in the composition. Since $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$, we can decompose T into disjoint sub-attacks $T = T' \cup \bigcup_{i=0}^{k-1} T_i$, where T_i acts inside the preprocessor gadget G_i and T' acts inside the core gadget G' ; hence $|T| = |T'| + \sum_i |T_i|$. According to the definition of $(\frac{e}{2}, \delta', \epsilon)$ -region fault-robustness we need to show for $\mathbf{Y} + \omega \leftarrow T[G^R] \left((\mathbf{F}_i^0 + \xi_i^0, \dots, \mathbf{F}_i^{l_i-1} + \xi_i^{l_i-1})_{i \in [k]} \right)$ and joint $(\frac{e}{2}, \epsilon)$ -fault vectors $(\xi_i^j)_{i \in [k], j \in [l_i]}$, that with probability $\delta' = 1 - (1 - \delta)^{k+1}$ ω is an $(\frac{e}{2}, \epsilon)$ -fault vector whose distribution $\mathcal{D}_\omega$ is only dependent on the internal randomness of the gadget G .

Preprocessing Stage. Since for each $i \in [k]$ it holds that the $(\xi_i^j)_{j \in [l_i]}$ are joint $(\frac{e}{2}, \epsilon)$ -fault vectors and it holds that $|T_i| + \frac{e}{2} \leq e$, we can apply the (e, δ, ϵ) -threshold fault robustness of G_i , and thus $\mathbf{Y}_i' + \xi_i' \leftarrow T_i[G_i^{R_i}] \left((\mathbf{F}_i^j + \xi_i^j)_{j \in [l_i]} \right)$ for each $i \in [k]$. With attack vector T_i , the output of G_i can be written with probability $1 - \delta$ as the valid (polynomial) sharing $\mathbf{Y}_i'$ plus an (s_i, ϵ) -fault vector ξ_i' , with $s_i \leq |T_i| + \frac{e}{2}$ and whose distribution $\mathcal{D}_{\xi_i'}$ only depends on $\mathcal{D}_{\xi_i^j}$ for $j \in [l_i]$ and the internal randomness of the gadget G_i . Notably, $(\xi_i')_{i \in [k]}$ are joint $((s_i)_{i \in [k]}, \epsilon)$ -fault vectors because $(\xi_i^j)_{i \in [k], j \in [l_i]}$ are joint fault vectors and each gadget G_i uses distinct inputs and independent randomness.

Core Stage. By assumption, the k preprocessed outputs $\mathbf{Y}_i' + \xi_i'$ are the inputs of G' . Since $s_i \leq e$ for all $i \in [k]$ and $|T'| \leq e$, the (e, δ, ϵ) -region fault robustness of G' under attack T' results in $\mathbf{Y} + \omega \leftarrow T'[G'^R] (\mathbf{Y}_0' + \xi_0', \mathbf{Y}_1' + \xi_1', \dots)$ with $s \leq |T'| \leq \frac{e}{2}$ such that it holds with probability $1 - \delta$ that ω is a (s, ϵ) -fault vector, i.e., an $(\frac{e}{2}, \epsilon)$ -fault vector, whose related distribution $\mathcal{D}_\omega$ only depends on the internal randomness of G' .

Conclusion. If we combine the intermediate independent failure probabilities, i.e., compute the probability that no failure happens as $\delta' = 1 - (1 - \delta)^{k+1}$, we get precisely the requirement of $(\frac{e}{2}, \epsilon)$ -region fault robustness for $N = 1$: for any attack of size at most $\frac{e}{2}$ on the (single) region, the output fault vector is an $(\frac{e}{2}, \epsilon)$ -fault vector.

Similar to Lemma 5, we can also compose random and region fault robust gadgets in the opposite order. The following lemma shows that when the output of an (e, δ, ϵ) -region fault robust refresh gadget is used as input to an (e, δ, ϵ) -region fault robustness, the overall gadget remains $(\frac{e}{2}, \delta', \epsilon)$ -region fault robust.

Lemma 6. *Let $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ be a wire-independent fault set. Further, let G' be a (e, δ, ϵ) -threshold fault robust gadget with respect to $\mathcal{F}$ with k input sharings and one output sharing. For each $i \in [k]$, let G_i be an (e, δ, ϵ) -region fault robust gadget with respect to $\mathcal{F}$ and l_i input and one output sharings. Moreover, let each gadget use independent sampled randomness. Then the composition $G = G'((G_i(\cdot))_{i \in [k]})$ is an $(\frac{e}{2}, \delta', \epsilon)$ -region fault robust gadget with respect to $\mathcal{F}$, $N = 1$ and $\delta' = 1 - (1 - \delta)^{k+1}$.*

The proof of Lemma 6 is deferred to Supp. E.4, as the proof of this lemma is very similar to Lemma 5.

Having shown how to build region fault robust gadgets, we now present our main composability result.

Lemma 7. *Let $G^{(0)}, \dots, G^{(t-1)}$ denote $t \geq 2$ gadgets such that*

1. $G^{(j)}$ has $N^{(j)}$ disjoint regions $R_0^{(j)}, \dots, R_{N^{(j)}-1}^{(j)}$ for every $j \in [t]$,
2. $G^{(j)}$ has $l^{(j)}$ inputs and one output for every $j \in [t-1]$, while $G^{(t-1)}$ has $t-1$ inputs and one output,
3. each gadget is (e, δ, ϵ) -region fault-robust with respect to $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$.

Then the composition $G^{(\text{comp})} = G^{(t-1)}(G^{(0)}(\dots), \dots, G^{(t-2)}(\dots))$ is (e, δ', ϵ) -region fault-robust with respect to the fault set $\mathcal{F}$, $\delta' = 1 - (1 - \delta)^t$, and the regions $\{R_k^{(j)} \mid j \in [t], k \in [N^{(j)}]\}$.

Proof. Let $G^{(0)}, \dots, G^{(t-1)}$ be as in the statement, and define $G^{(\text{comp})}$ as the expression $G^{(t-1)}(G^{(0)}(\cdot), \dots, G^{(t-2)}(\cdot))$. For each $j \in [t]$, let $R_0^{(j)}, \dots, R_{N^{(j)}-1}^{(j)}$ be the disjoint regions of $G^{(j)}$. Fix any tuple of valid (polynomial) input sharings to the composed gadget, and consider any fault attack $T \in \mathcal{A}(\mathcal{F})$ such that for every $j \in [t]$ and every $r \in [N^{(j)}]$ we have $|T|_{R_r^{(j)}} \leq e$. Here, $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ guarantees that the faults in each region are independent and can be split. Hence, we can decompose T into disjoint sub-attacks $T = \bigcup_{j=0}^{t-1} T^{(j)}$ with $T^{(j)} = \bigcup_{r=0}^{N^{(j)}-1} T_r^{(j)}$, where $T_r^{(j)} \subseteq R_r^{(j)}$ and $|T_r^{(j)}| \leq e$ for all j, r .

Outputs of the First Layer $G^{(0)}, \dots, G^{(t-2)}$. For each $j \in [t-1]$, apply (e, δ, ϵ) -region fault robustness of $G^{(j)}$ to its valid inputs $(\mathbf{F}_0^{(j)}, \dots, \mathbf{F}_{l^{(j)}}^{(j)})$ and joint (e, ϵ) -fault vectors $(\xi_0 \dots, \xi_{l^{(j)}}^{(j)})$ under the attack $T^{(j)}$:

$$(\mathbf{Y}'^{(j)} + \boldsymbol{\omega}'^{(j)}) \leftarrow T^{(j)}[G^{(j), R_i}] (\mathbf{F}_0^{(j)} + \xi_0 \dots \mathbf{F}_{l^{(j)}}^{(j)} + \xi_{l^{(j)}}^{(j)}).$$

By Definition 11, $\omega^{(j)}$ is a $(s^{(j)}, \epsilon)$ -fault vector with probability $1 - \delta$, where $s^{(j)} \leq \max_{r \in [N^{(j)}]} |T_r^{(j)}| \leq e$. Moreover, for each $j \in [t - 1]$, the distribution $\mathcal{D}_{\omega^{(j)}}$ described in Definition 5 only depends on the internal randomness of $G^{(j)}$ and by assumption, the internal randomness of the gadgets $G^{(j)}, j \in [t - 1]$ are independent of each other. Hence, the fault vectors $(\omega^{(j)})_{j \in [t-1]}$ are joint $((s^{(j)})_{j \in [t-1]}, \epsilon)$ -fault vectors with probability $(1 - \delta)^{t-1}$.

Inputs to the Final Gadget $G^{(t-1)}$. Every input sharing to $G^{(t-1)}$ is itself the valid sharing $\mathbf{Y}'^{(j)}$ plus an (e, ϵ) -fault vector $\omega^{(j)}$ since by assumption 2, the gadget $G^{(t-1)}$ takes as inputs the concatenation of all outputs of $G^{(0)}, \dots, G^{(t-2)}$ of the form $\mathbf{Y}^* = (\mathbf{Y}'^{(j)} + \omega^{(j)})_{j \in [t-1]}$.

We can now apply the (e, δ, ϵ) -region fault robustness of $G^{(t-1)}$ under the attack $T^{(t-1)}$ since the per-region budget $|T_r^{(t-1)}| \leq e$ is satisfied for each r and the inputs are faulted by joint $((s^{(j)})_{j \in [t-1]}, \epsilon)$ -fault vectors with probability $(1 - \delta)^{t-1}$, as $(\mathbf{Y}^{(t-1)} + \omega^{(t-1)}) \leftarrow T^{(t-1)}[G^{(t-1), R_{(t-1)}}](\mathbf{Y}^*)$.

By Definition 11, it holds again with probability $(1 - \delta)$ that its outputs equal the valid output $\mathbf{Y}^{(t-1)}$ plus $(s^{(t-1)}, \epsilon)$ -fault vector $\omega^{(t-1)}$ which only depends on the internal randomness of gadget $G^{(t-1)}$ and where $s^{(t-1)} \leq \max_{r \in [N^{(t-1)}]} |T_r^{(t-1)}|$.

Conclusion. Putting the failure probabilities together, we get $1 - \delta' = (1 - \delta)^t$. We have shown that for any attack T respecting the per-region bound e on each $\mathbf{R}_r^{(j)}$, the composed gadget $G^{(\text{comp})}$ maps any valid inputs $(\mathbf{F}_0^{(j)}, \dots, \mathbf{F}_{l^{(j)}-1}^{(j)})$ plus joint (e, ϵ) -fault vectors $(\xi_0 \dots, \xi_{l^{(j)}-1}^{(j)})$ with probability $1 - \delta'$ to an output that can be described as the sum of the original output $\mathbf{Y}^{(t-1)}$ plus $(s^{(t-1)}, \epsilon)$ -fault vector $\omega^{(t-1)}$ only dependent on the internal randomness of gadget $G^{(t-1)}$ and therefore only dependent on the internal randomness of gadget $G^{(\text{comp})}$, with $s^{(t-1)} \leq \max_{r \in [N^{(t-1)}]} |T_r^{(t-1)}| \leq \max_{j \in [t], r \in [N^{(j)}]} |T_r^{(j)}|$. Hence, $G^{(\text{comp})}$ is (e, δ', ϵ) -region fault robust with respect to $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ and the region sets are $\{\mathbf{R}_r^{(j)}\}_{j \in [t], r \in [N^{(j)}]}$, as claimed.

Region fault robustness allows securely composing gadgets or even larger circuits. The following lemma gives the general security bounds for region fault-robust circuits.

Lemma 8. *Let G be an (e, δ, ϵ) -region fault-robust gadget (or circuit), and assume that $n - e > d$. Then the probability that the output encoding is valid but encodes the wrong secret is at most $\delta + (1 - \delta) \cdot (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$.*

Proof. Let $\mathbf{Y} + \omega$ be the output encoding of the faulted computation. Under the failure probability δ of fault robustness, we cannot claim anything about the validity of the encoding. This introduces the additive factor δ in the lemma bound. When failure does not happen, i.e., with probability $1 - \delta$, then ω is an (e, ϵ) -fault vector. This is a valid encoding for an incorrect value if and only if the polynomial underlying the encoding $\mathbf{Y} + \omega$ is valid and $\mathbf{Y} + \omega$ decodes to

$\tilde{y} \neq y$, where y is the decoding of $\mathbf{Y}$. Let $\boldsymbol{\omega} = \mathbf{v} + \mathbf{v}''$ be the decomposition of fault vector $\boldsymbol{\omega}$ according to Definition 9. Notably, $\boldsymbol{\omega} \neq 0$, as it changes the secret underlying $\mathbf{Y}$. Moreover, $\mathbf{v}''$ must equal the randomized component $\bar{\mathbf{v}}$. This is because of the fundamental lemma (*cf.* Lemma 1): $\boldsymbol{\omega} = \mathbf{v}$ would satisfy $0 < \text{weight}(\boldsymbol{\omega}) \leq e$, meaning that the underlying polynomial would have degree greater than $n - e > d$, and thus be invalid. Given that $\mathbf{v}'' = \bar{\mathbf{v}}$, validity is only possible when $p_h^{(\bar{\mathbf{v}})} = -p_h^{(\mathbf{v})}$ for every $h = d + 1, \dots, n$. As all these coefficients in $p^{(\bar{\mathbf{v}})}$ are chosen independently from D_ϵ , this happens with probability at most $(|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$. This concludes the proof.

4 Circuit Fault-Invariance

The notion of fault-invariance was introduced by Berndt et al. in [12] and has been used to obtain combined resilience for masked circuits. Informally, fault invariance holds when the effect of a fault in an arbitrary location of the circuit can be modeled by a fault on the inputs or the outputs. This eases the analysis of faults significantly. Looking ahead, the fault-invariance property is key for general deductions, as we can show that any circuit that is (S)NI and fault-invariant can be perfectly simulated in the presence of faults. Fault-invariance can be analyzed for both the circuit and the leakage simulator. The work of Berndt et al. considered the fault-invariance of circuits, assuming that it is equivalent to simulator invariance. However, in Supp. D we show that this assumption does not hold in general. Namely, the fault-invariance presented by Berndt et al. does not imply that the simulator itself is fault-invariant, i.e., that the simulator can perfectly simulate a faulted circuit by considering faulted input or output elements. However, this mismatch did not affect the security of the gadgets presented in the original paper [12], as both, the circuits and the (S)NI simulators chosen in their proofs satisfy fault-invariance. A detailed discussion of the limitations of fault-invariance defined in [12], along with an explanation of why the original gadgets remain unaffected, is provided in Supp. I.

To remedy this small gap, we introduce the notion of *circuit fault-invariance*. In a nutshell, circuit fault-invariance consists of a generalized version of the fault-invariance defined by Berndt et al., together with a linearity property of the fault propagation. Circuit fault-invariance is used directly in the security proofs of our gadgets and involves a new proof technique. Intuitively, circuit fault invariance implies that the output of a probe in a faulted circuit can be reconstructed by faulting the probed value and/or any tuple of previous wires.

For the notions and our proof technique, we rely on graph theory. Standard definitions and notation from graph theory are restated in Supp. C. More concretely, a directed acyclic graph (DAG) $\mathbf{G} = (V, E)$ represents circuit $\mathbf{C}^R$ for uniformly i.i.d. eagerly sampled randomness $R = (R_0, \dots, R_m)$.

We are interested in the following situation: Suppose that $\mathbf{G}$ consists of two subgraphs $\mathbf{G}_1 = (V_1, E_1)$ and $\mathbf{G}_2 = (V_2, E_2)$ such that the computations of $\mathbf{G}_1$ are performed before $\mathbf{G}_2$. We now want to analyze how a fault in $\mathbf{G}_1$ might propagate to $\mathbf{G}_2$. Throughout this section, we thus fix such a partition $\mathbf{G}_1$ and $\mathbf{G}_2$, where

a subset of output gates of G_1 are input gates of G_2 , as well as a topological ordering $\preceq$ on V . Note that G_1 and G_2 do not need to correspond to gadgets. Additionally, by saying *probe* $\hat{p}$, we refer to the probe of the wire $\hat{p} \in E$, while the result of the probe, i.e., the value of the probed wire, is denoted by $\lambda(\hat{p})$. By analyzing this situation, we can define a more generalized version of fault-invariance that allows us to push faults towards earlier values or the outputs. To define this properly, we need two ingredients when considering a fault on an edge: First, we need to find the parts of the graph that lead to the probed edge (which we call the *probing subgraph*) and then define the possible positions where we aim to push a fault to (which we call the *partial subgraph*). The reason we have to take probes into account when pushing faults forward or toward the outputs is that we must ensure that the locations to which the faults are pushed yield values that remain consistent during the simulation of different probes.

Note that, when seeing C^R as a directed graph, any fault is performed on an edge $(\hat{u}, \hat{v}) \in E$ and not on the vertex $\hat{u}$ itself. E.g., considering a gate with more than one output vertex, we assume either to have a faulted input, which leads to the outputs being faulted too, or a fault can occur in one output at a time, which corresponds to a gadget with a fault injected during the *transportation of the information* via the edge. Intuitively, the probing subgraph $G^{\hat{p}}$ consists of all input vertices as well as all vertices V^{in} and edges that lie on a path from an input to the probe edge $\hat{p}$.

Definition 12 (Subgraph with respect to probe $\hat{p}$ (probing subgraph)). Let $G = (V, E)$ denote a directed acyclic graph and the probe $\hat{p} = (\hat{u}, \hat{v}) \in E$. Then, the subgraph of (V, E) with respect to a probe $\hat{p}$ (or *probing subgraph with respect to $\hat{p}$*) is the graph $G^{\hat{p}} = (V^{\hat{p}}, E^{\hat{p}}) = (V^{\text{in}}, \emptyset) \cup P_{V^{\text{in}} \rightarrow \hat{u}}$. Additionally, the output vertices and edges of $G^{\hat{p}}$ are defined as $V^{\text{out}} = \{v \in V^{\hat{p}} \mid \exists (v, v') \in E \setminus E^{\hat{p}}\}$ and $E^{\text{out}} = \{(v, v') \in E \setminus E^{\hat{p}} \mid v \in V^{\hat{p}}, v' \notin V^{\hat{p}}\}$.

We can also extend this definition such that for a set $\hat{p}_0, \dots, \hat{p}_{\ell-1}$ of probes, we define $G^{(\hat{p}_0, \dots, \hat{p}_{\ell-1})} := \bigcup_{i=0}^{\ell-1} G^{\hat{p}_i}$.

Looking ahead, we cannot simply propagate a fault to a previous value within the graph, as this might not be consistent with previous calculations. This is the reason why we define partial subgraphs with respect to previous probes. Namely, we define the partial subgraph $\bar{G}^{\hat{p} \setminus (p_0, \dots, p_{\ell-1})}$ regarding probe $\hat{p}$ and all its preceding probes $(p_0, \dots, p_{\ell-1})$. It is obtained from the probing subgraph $G^{\hat{p}}$ by removing all paths contained in the union of probing subgraphs $G^{(p_0, \dots, p_{\ell-1})}$, except for the final edge leading into $G^{\hat{p}}$. In other words, the outputs of $G^{(p_0, \dots, p_{\ell-1})}$ can be used as inputs for partial graph $\bar{G}^{\hat{p} \setminus (p_0, \dots, p_{\ell-1})}$.

Definition 13 (Partial subgraph with respect to probes). Let $G = (V, E)$ be a DAG and $\hat{p} = (\hat{u}, \hat{v}) \in E$ denote a probed wire. Furthermore, let $p_0, \dots, p_{\ell-1} \in E$ such that for $p_i = (v_i, v'_i)$ it holds true that $v_i \preceq \hat{u}$ for all i . Let G^{p_i} , and $G^{\hat{p}}$ denote the subgraphs of G corresponding to $p_i, i \in [\ell]$ and $\hat{p}$, respectively. Then, the partial subgraph $\bar{G}^{\hat{p} \setminus (p_0, \dots, p_{\ell-1})}$ corresponding to $G^{\hat{p}}$ without $G^{(p_0, \dots, p_{\ell-1})}$ includes

all the vertices and edges of $G^{\hat{p}} \setminus G^{(p_0, \dots, p_{\ell-1})}$ and all the vertices in $G^{(p_0, \dots, p_{\ell-1})}$ that have edges in $G^{\hat{p}}$ leading outside of $G^{(p_0, \dots, p_{\ell-1})}$, i.e.,

$$\overline{G}^{\hat{p} \setminus (p_0, \dots, p_{\ell-1})} := \left(G^{\hat{p}} \setminus G^{(p_0, \dots, p_{\ell-1})} \right) \cup \left(\{v \in \bigcup_{i \in [\ell]} V^{p_i} \mid \exists (v, v') \in E^{\hat{p}} \setminus \bigcup_{i \in [\ell]} E^{p_i}\}, \emptyset \right).$$

In the following, we denote by $G^{\hat{p}}(\mathbf{x})$ the values on the output wires, i.e., for each $e \in E^{\hat{p}_{\text{out}}}$, the value $\lambda(e)$ is output by $G^{\hat{p}}(\mathbf{x})$. Thus, the subgraph $G^{\hat{p}}(\mathbf{x})$ with respect to the probe $\hat{p}$ outputs $\lambda(\hat{p})$ together with some auxiliary data, which corresponds to the values on the remaining output wires of $G^{\hat{p}}$. As all input vertices V^{in} belong to $G^{(p_0, \dots, p_{\ell-1})}$ and $G^{\hat{p} \setminus (p_0, \dots, p_{\ell-1})}$ also contains the edges going into $G^{\hat{p}}$, we have $\overline{G}^{\hat{p} \setminus (p_0, \dots, p_{\ell-1})}(G^{(p_0, \dots, p_{\ell-1})}(\mathbf{x})) = G^{\hat{p}}(\mathbf{x})$. Notably, we treat the internal randomness R used by the gadget as an input to the graph.

When “pushing” faults to previous values, one needs to be careful: It is relatively simple to explain any faults by arbitrarily changing previous values. However, to model such arbitrary changes, we would need to increase the fault threshold e significantly. We thus only allow changes to previous values that can be modeled without such a threshold increase. More formally, we only allow *input-independent element-wise bijective function*. These are element-wise bijective functions $y(\cdot): \mathbb{F}^* \rightarrow \mathbb{F}^*$ whose constants are independent of the gadgets’ inputs, and only depend on the structure of the gadget and the injected faults.

Definition 14 (Circuit fault-invariance). Let $\ell \in \mathbb{N}$. Let $\mathbf{R}$ denote uniformly i.i.d. eagerly sampled randomness. Let $\mathbf{x}$ denote any input vector. Let G be a gadget represented as a directed acyclic graph. Let p denote any wire (or edge) in G , let G^p denote its probing subgraph and let $T[G^p]$ denote the corresponding faulted probing subgraph. If $\forall \hat{p} \in E, \forall p_0, \dots, p_{\ell-1} \preccurlyeq \hat{p} \in E: \exists$ an input-independent element-wise bijective function $y: \mathbb{F}^* \rightarrow \mathbb{F}^*$ and $\exists$ fault functions ξ of $\mathcal{F}$ such that $T[G^p(\mathbf{R}, \mathbf{x})] = \xi(\overline{G}^{\hat{p} \setminus (p_0, \dots, p_{\ell-1})}(y(T[G^{(p_0, \dots, p_{\ell-1})}(\mathbf{R}, \mathbf{x})]))$, then we call G circuit fault-invariant with respect to fault set $\mathcal{F}$.

We refer to Supp. F for the full formal proofs of Lemma 9 and Corollary 1.

Lemma 9. Let G be a gadget that is $d(S)NI$ and circuit fault-invariant with respect to $\mathcal{F}^+$. Then the gadget G is $d\text{-fr}(S)NI$ with respect to $\mathcal{F}^+$.

Corollary 1 ([12, Cor.2]). If a gadget G is $d\text{-fr}(S)NI$ and circuit fault-invariant with respect to $\mathcal{F}^+$, it is $d\text{-wfr}(S)NI$ with respect to $\mathcal{F}^{\text{ind}}$.

5 Our Refresh Gadget

We construct a refresh gadget that re-randomizes the high-order coefficients of the input sharing for nonzero input faults. This is an essential property for our region fault robustness.

In a nutshell, we propagate any non-zero higher-order coefficient $c_{k>d}$ of the input $F_0 + \xi_0$ to all higher-order coefficients $[d+1, n]$ in the output, and

additionally ensure that the resulting higher-order coefficients are randomized by multiplying with random values during this process. Technically, we achieve this by constructing a matrix A with respect to a parity check matrix H , as well as a special matrix to re-distribute random values, which consists of the inverse Vandermonde matrix V^{-1} , as well as a random matrix R .

In more detail, consider the shares $\left(R_i^{(k)}\right)_{i \in [n]}$ for $k = d+1, \dots, n-1$ with respect to a polynomial $r^{(k)}(X) = r_{d+1}^{(k)}X^{d+1} + \dots + r_{n-1}^{(k)}X^{n-1}$, where $r_i^{(k)}, i \in [d+1, n]$ are sampled uniformly random from $\mathbb{F}$. Let $\mathbf{F} = (F_0, \dots, F_{n-1})$ denote the input vector and let $f(X) = f_0 + \dots + f_{n-1}X^{n-1}$ denote the corresponding polynomial. We construct a polynomial with coefficients

$$\mathbf{s} = \mathbf{f} + \begin{pmatrix} r_0^{(d+1)} \cdot f_{d+1} \\ \vdots \\ r_{n-1}^{(d+1)} \cdot f_{d+1} \end{pmatrix} + \dots + \begin{pmatrix} r_0^{(n-1)} \cdot f_{n-1} \\ \vdots \\ r_{n-1}^{(n-1)} \cdot f_{n-1} \end{pmatrix},$$

where $r_0^{(k)} = \dots = r_d^{(k)} = 0$ for $k \in [d+1, n]$. Notably, the first $d+1$ coefficients of s equal those of f . Moreover, if $\mathbf{f}$ is a *valid* encoding, then $\mathbf{s}$ equals $\mathbf{f}$, as

$$\mathbf{s} = \mathbf{f} + \begin{pmatrix} r_0^{(d+1)} \cdot 0 \\ \vdots \\ r_{n-1}^{(d+1)} \cdot 0 \end{pmatrix} + \dots + \begin{pmatrix} r_0^{(n-1)} \cdot 0 \\ \vdots \\ r_{n-1}^{(n-1)} \cdot 0 \end{pmatrix} = \mathbf{f}.$$

On the other hand, if there exists exactly one coefficient $f_k \neq 0$ for $k > d$, then

$$\mathbf{s} = \mathbf{f} + \begin{pmatrix} r_0^{(d+1)} \cdot f_{d+1} \\ \vdots \\ r_{n-1}^{(d+1)} \cdot f_{d+1} \end{pmatrix} + \dots + \begin{pmatrix} r_0^{(n-1)} \cdot f_{n-1} \\ \vdots \\ r_{n-1}^{(n-1)} \cdot f_{n-1} \end{pmatrix} = \mathbf{f} + \begin{pmatrix} 0 \cdot f_k \\ \vdots \\ 0 \cdot f_k \\ r_{d+1}^{(k)} \cdot f_k \\ \vdots \\ r_{n-1}^{(k)} \cdot f_k \end{pmatrix}$$

results in a polynomial of degree up to $n-1$, where the higher-order coefficients $s_{d+1}, \dots, s_{n-1}$ are indistinguishable from chosen uniformly random, since $r_i^{(k)}$ are chosen uniformly random for $i \in [d+1, n]$. This extends likewise to more than one non-zero high-order coefficient.

We detail the construction of the matrix A that transforms the shares $\mathbf{F}$ for coefficients $\mathbf{f}$ to the shares $\mathbf{S}$ for the coefficients $\mathbf{s}$, namely $\mathbf{S} = A \cdot \mathbf{F}$, in the following. Let $R_i^{(k)} = \sum_{j>d} r_j^{(k)} x_i^j$ with $r_j^{(k)}$ uniformly random denote the i -th share of a polynomial with coefficients $r_0^{(k)} = 0, \dots, r_d^{(k)} = 0, r_{d+1}^{(k)}, \dots, r_{n-1}^{(k)}$ and

define $M^{(k)}$ s.t.

$$M^{(k)} = \begin{pmatrix} R_0^{(k)} \cdot V_{k,0}^{-1} & \dots & R_0^{(k)} \cdot V_{k,n-1}^{-1} \\ \vdots & & \vdots \\ R_{n-1}^{(k)} \cdot V_{k,0}^{-1} & \dots & R_{n-1}^{(k)} \cdot V_{k,n-1}^{-1} \end{pmatrix} = \begin{pmatrix} R_0^{(k)} \cdot V_{k,:}^{-1} \\ \vdots \\ R_{n-1}^{(k)} \cdot V_{k,:}^{-1} \end{pmatrix}.$$

$$M^{(k)} \cdot \mathbf{F} = \begin{pmatrix} R_0^{(k)} \cdot V_{k,:}^{-1} \\ \vdots \\ R_{n-1}^{(k)} \cdot V_{k,:}^{-1} \end{pmatrix} \cdot \mathbf{F} = \begin{pmatrix} \sum_{i=0}^{n-1} R_0^{(k)} \cdot V_{k,i}^{-1} \cdot F_i \\ \vdots \\ \sum_{i=0}^{n-1} R_{n-1}^{(k)} \cdot V_{k,i}^{-1} \cdot F_i \end{pmatrix}$$

$$= \begin{pmatrix} R_0^{(k)} \cdot \sum_{i=0}^{n-1} V_{k,i}^{-1} \cdot F_i \\ \vdots \\ R_{n-1}^{(k)} \cdot \sum_{i=0}^{n-1} V_{k,i}^{-1} \cdot F_i \end{pmatrix} = \begin{pmatrix} R_0^{(k)} \cdot f_k \\ \vdots \\ R_{n-1}^{(k)} \cdot f_k \end{pmatrix} = \begin{pmatrix} \sum_{j>d} r_j^{(k)} \cdot x_0^j \cdot f_k \\ \vdots \\ \sum_{j>d} r_j^{(k)} \cdot x_{n-1}^j \cdot f_k \end{pmatrix}.$$

Note that $\sum_{j>d} r_j^{(k)} \cdot x_i^j \cdot f_k$ denotes the i -th share of a polynomial with coefficient $r_j^{(k)} \cdot f_k$ for $j > d$ and 0 for $j \leq d$. Finally, we define $A = \text{Id} + \sum_{k=d+1}^{n-1} M^{(k)}$. We use this construction of A in Algorithm 1. Furthermore, this observation

Algorithm 1 P-Refresh

Input : Non-negative parameters n, d, e s.t. $n \geq d + e + 1$, $V^{-1} = \text{Van}_{[n] \times [n]}^{-1}(x_0, \dots, x_{n-1})$ and degree- d encoding of secret f_0 as $(F_i)_{i \in [n]}$, for distinct non-zero $x_0, \dots, x_{n-1}$.

Output: Degree- d encoding of secret $s_0 = f_0$ in shares $(S_i)_{i \in [n]}$.

```

1.1 initialize  $(A_{i,j})_{i,j \in [n]}$  as  $\text{Id}_{n \times n}$  and  $(S_i)_{i \in [n]}$  as  $\mathbf{0}_n$ 
1.2 for  $k \in [d+1, n]$  do
1.3   initialize  $(R_i^{(k)})_{i \in [n]}$  as  $\mathbf{0}_n$ 
1.4   for  $j \in [d+1, n]$  do
1.5     Sample  $r_j^{(k)} \leftarrow \mathbb{F}$ 
1.6     for  $i \in [n]$  do
1.7        $R_i^{(k)} \leftarrow R_i^{(k)} + r_j^{(k)} x_i^j$ 
1.8 for  $i \in [n]$  do
1.9    $(Z_{i,0}, \dots, Z_{i,n-1}) \leftarrow \mathbf{ZEnc}_n^d$ 
1.10  for  $j \in [n]$  do
1.11     $A_{j,i} \leftarrow A_{j,i} + \sum_{k \in [d+1, n]} R_j^{(k)} \cdot V_{k,i}^{-1}$ 
1.12     $Z_{i,j} \leftarrow Z_{i,j} + A_{j,i} F_i$ 
1.13     $S_j \leftarrow S_j + Z_{i,j}$ 
1.14 return  $(S_i)_{i \in [n]}$ 

```

gives us correctness of the Refresh gadget **P-Refresh** (Lemma 10), and error propagation properties (Lemma 11).

Lemma 10 (Correctness of P-Refresh). *Let $\mathbf{F} = (F_0, \dots, F_{n-1})$ be a valid sharing that corresponds to the polynomial $f(X) = f_0 + \dots + f_d X^d$, i.e., $F_i = f(x_i) \forall i \in [n]$, where $x_i, i \in [n]$ denote pairwise distinct support points that are non-zero. Moreover, let **ZEnc** be a correct algorithm that outputs n shares of a zero-encoding, i.e., shares corresponding to a polynomial of degree up to d encoding the secret 0. Then, Algorithm 1 outputs a sharing $(S_i)_{i \in [n]}$, such that the corresponding polynomial $s(X)$, fulfils $s(X) = s_0 + \dots s_d \cdot X^d$ and $s_0 = f_0$.*

Lemma 11 (Correctness of Error-Propagation). *Let $(F_i)_{i \in [n]}$ denote a non-valid sharing, i.e., a sharing corresponding to a polynomial $f(X) = f_0 + \dots + f_{n-1} X^{n-1}$ where $\exists i \in [d+1, n-1] : f_i \neq 0$ (and with respect to the support points $x_0, \dots, x_{n-1}$). On input $(F_i)_{i \in [n]}$ Algorithm 1 outputs a sharing S_i with respect to an up to degree- $n-1$ polynomial $s(X)$, whose coefficients $s_{d+1}, \dots, s_{n-1}$ are indistinguishable from chosen uniformly at random.*

The proofs of Lemma 10 and Lemma 11 proceed by algebraic reasoning on the above definitions. We refer to Supp. G.1 for the full formal proofs due to space constraints. Next, we show region fault robustness of **P-Refresh**.

Lemma 12 (P-Refresh is (e, δ, ϵ) -region fault-robust). *Let **ZEnc** be (e, δ, ϵ) -fault robust with respect to the fault set $\mathcal{F}^{+1}$. Consider the gadget **P-Refresh** described in Algorithm 1 as a single region. Then **P-Refresh** is (e, δ', ϵ) -region fault-robust with respect to $\mathcal{F}^+$, with $\delta' = 1 - (1 - \delta)^n + 2(|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$.*

Proof. Let ξ be an (e, ϵ) -fault vector, and let T be a fault attack on internal and output wires such that $|T| \leq e$. We want to prove that the output fault ω defined as $\mathbf{Y} + \omega \equiv T[\mathbf{P-Refresh}^R](\mathbf{F} + \xi)$ is a (s, ϵ) -fault vector independent of the input except with probability $1 - \delta'$, where $s \leq |T|$. Moreover, we want to prove that the distribution $\mathcal{D}_\omega$ only depends on the internal randomness R . We distinguish four types of faults:

- *linear fault*: one fault that affects only one output share.
- *quasi-linear fault*: one internal fault affects up to one row of the intermediate matrix $A_{[i,:]}, i \in [n]$. We will prove later that each such fault still affects only one output share, and is thus linear.
- *randomness faults*: denotes faults that are injected into the sampling of random elements.
- *non-linear fault*: a fault that affects more than one output share.

Now, we consider all possible faults that may occur within the gadget **P-Refresh**.

- Group 1 consists of input faults on the valid input $\mathbf{F} = (F_i)_{i \in [n]}$. We need to distinguish two cases:
 - a) $(F_i)_{i \in [n]} + (\xi_i^F)_{i \in [n]} \neq (F_i)_{i \in [n]}$ and is of degree up to d . We call this event **BAD₁**. By the same argument used for proving Lemma 8, $\xi = (\xi_i)_{i \in [n]}$ being a (e, ϵ) -fault vector implies that $\Pr[\mathbf{BAD}_1] \leq (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$.

¹ An instantiation of a $(e, 0, 0)$ -fault robust zero-encoding can be found in Supp. B

- b) Otherwise, we classify these faults as non-linear.
- Group 2 includes all faults performed on the randomness $r_j^{(k)}$ (line 5). These faults are classified as randomness faults. Since $r_j^{(k)}$ is uniformly distributed in $\mathbb{F}$ and additive faults are input-independent, $r_j^{(k)} + \xi r_j^{(k)}$ remains uniformly distributed for any faults $\xi r_j^{(k)} \in \mathcal{F}^+$.
 - Group 3 considers all faults in line 3 and 7, i.e., all faults injected during the calculation of $R_i^{(k)} = \sum_{j \in [n]} r_j^{(k)} x_i^j$. Faults injected into and within the calculation of $R_i^{(k)}$ are of quasi-linear type, since any fault in $R_i^{(k)}$ propagates to the i -th row $A_{[i,:]}$.
 - Group 4 denotes the internal and output faults of the i -th execution of **ZEnc**, i.e. $(Z_{i,0}, \dots, Z_{i,n-1}) \leftarrow \mathbf{ZEnc}^{(i)}$ (line 10). As the zero-encodings $(Z_{i,j})_{j \in [n]}$ are only used once during a share-wise addition in line 13, the faults propagate in a share-wise manner to the output. According to the fault robustness of **ZEnc**, $e_{\mathbf{ZEnc}}^{(i)}$ internal or output faults within the $\mathbf{ZEnc}^{(i)}$ result in the injection of a $(s^{(i)}, \epsilon)$ -fault vector $\mathbf{v}^{(i)}$ in the output of $\mathbf{ZEnc}^{(i)}$, except with probability δ , where $s^{(i)} \leq e_{\mathbf{ZEnc}}^{(i)}$. We call the event where $\mathbf{v}^{(i)}$ is not a $(s^{(i)}, \epsilon)$ -fault vector $\text{BAD}_2^{(i)}$, and therefore $\Pr[\text{BAD}_2^{(i)}] \leq \delta$. Let BAD_2 be the event where at least one of the events $(\text{BAD}_2^{(i)})_{i \in [n]}$ happens. Since the refresh evaluations are independent of each other and additive faults are wire-independent, so are the vectors $\mathbf{v}^{(i)}$, and therefore $\Pr[\text{BAD}_2] = 1 - \prod_{i \in [n]} (1 - \Pr[\text{BAD}_2^{(i)}]) \leq 1 - (1 - \delta)^n$. Notably, a non-zero $(e_{\mathbf{ZEnc}}^{(i)}, \epsilon)$ -fault vector $\mathbf{v}^{(i)}$ maintains validity only with probability $\leq (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$. We call BAD_3 the event where all the **ZEnc** output faults $\mathbf{v}^{(i)}$ are valid $(s^{(i)}, \epsilon)$ -fault vectors, and there exists at least one $\mathbf{v}^{(i)} \neq 0$. This event satisfies $\Pr[\text{BAD}_3] \leq (1 - \Pr[\text{BAD}_2])(|\mathbb{F}|^{-1} + \epsilon)^{n-d-1} \leq (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$.
 - Group 5 denotes all additive faults injected in lines 1, 8, 12, 13, and 14, i.e., faults injected during the initialization and calculations of matrix A , $A_{j,i} = \text{Id}_{j,i} + \sum_{k \in [d+1, n]} R_j^{(k)} V_{k,i}^{-1}$, the calculation of output shares $(S_i)_{i \in [n]}$, $S_i = \sum_{j \in [n]} Z_{j,i} + A_{i,j} \cdot F_j$, and during the return of the final outputs (line 15). Since any additive fault propagates share-wise to the output S_i for $i \in [n]$, this group consists of linear faults.

Quasi Linear Faults. Notably, any quasi-linear fault propagates to at most one fault in the output, because

$$S_i = \sum_{j \in [n]} Z_{j,i} + A_{i,j} \cdot F_j = \sum_{j \in [n]} \left(Z_{j,i} + \left(\text{Id}_{i,j} + \sum_{k \in [d+1, n]} R_i^{(k)} V_{k,j}^{-1} \right) \cdot F_j \right)$$

and hence, one fault in $R_i^{(k)}$ propagates to at most all elements in the row $A_{[i,:]}$, but the elements in this row are summed up to obtain one output share S_i . Moreover, the i -th row of matrix A only affects the i -th output share S_i .

Ruling Out the Bad Events. We now put all the bad events together to provide an upper bound δ' to the failure probability. We call **BAD** the event where at least one among $\text{BAD}_1, \text{BAD}_2, \text{BAD}_3$ happens. By union bound,

$$\begin{aligned} \Pr[\text{BAD}] &\leq \Pr[\text{BAD}_1] + \Pr[\text{BAD}_2] + \Pr[\text{BAD}_3] \\ &\leq (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1} + 1 - (1 - \delta)^n + (|F|^{-1} + \epsilon)^{n-d-1} \\ &= 1 - (1 - \delta)^n + 2(|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}. \end{aligned}$$

Therefore, we set $\delta' = 1 - (1 - \delta)^n + 2(|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$. In the following paragraphs, we will always assume that **BAD** has not happened.

Linear Faults and Deterministic Part of v . We have $\sum_{i \in [n]} e_{\mathbf{ZEnc}}^{(i)}$ total faults within all the **ZEnc** evaluations, and $|T| - \sum_{i \in [n]} e_{\mathbf{ZEnc}}^{(i)}$ in the remaining internal wires. Now, the **ZEnc** faults contribute to the deterministic part of each **ZEnc** output fault vector $\mathbf{v}^{(i)}$ on at most $e_{\mathbf{ZEnc}}^{(i)}$ shares, which propagate share-wise to the output. There are at most $|T| - \sum_{i \in [n]} e_{\mathbf{ZEnc}}^{(i)}$ linear faults in the rest of the circuit. This means that all the linear faults contribute to the deterministic part of the output in at most $\sum_{i \in [n]} e_{\mathbf{ZEnc}}^{(i)} + |T| - \sum_{i \in [n]} e_{\mathbf{ZEnc}}^{(i)} = |T|$ shares.

Non-Linear Faults and Randomness Faults. It is left to consider the type of non-linear faults as well as the randomness faults. By correctness of the gadget, the coefficients $s_m, m \in [n]$ of the polynomial corresponding to the output shares $(S_i)_{i \in [n]}$ are calculated as

$$\begin{aligned} s_m &= \sum_{i=0}^{n-1} (S_i \cdot V_{m,i}^{-1}) = \sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} Z_{j,i} \cdot V_{m,i}^{-1} \right) + \sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} \text{Id}_{i,j} \cdot F_j \cdot V_{m,i}^{-1} \right) \\ &\quad + \sum_{i=0}^{n-1} \sum_{k=d+1}^{n-1} \sum_{\ell=d+1}^{n-1} \left(r_{\ell}^{(k)} + \xi_{\ell}^{r^{(k)}} \right) x_i^{\ell} f_k \cdot V_{m,i}^{-1} \\ &= \sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} Z_{j,i} \cdot V_{m,i}^{-1} \right) + \sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} \text{Id}_{i,j} \cdot F_j \cdot V_{m,i}^{-1} \right) + \sum_{k=d+1}^{n-1} \left(r_m^{(k)} + \xi_m^{r^{(k)}} \right) f_k. \end{aligned}$$

If $r_m^{(k)}$ is faulted but the input is not, then $f_k = 0$ for every $k \geq d+1$, and therefore the fault does not affect $(S_i)_{i \in [n]}$. If there is a non-linear fault on $(F_i)_{i \in [n]}$, then $(F_i)_{i \in [n]} + (\xi_i^F)_{i \in [n]}$ interpolates to a polynomial of degree greater than d . Thus, the corresponding polynomial $f(X)$ must have at least a non-zero high-order coefficient f_{k^*} , i.e. $k^* > d$. Recall that $r_m^{(k)}$ are sampled uniformly from $\mathbb{F}$ and are independent of each other for all $m \in [d+1, n], k \in [d+1, n]$. Moreover, $r_j^{(k)}$ are sampled uniformly at random from $\mathbb{F}$. Hence, also $r_j^{(k)} + \xi_j^{r^{(k)}}$ is uniformly distributed for all $\xi_j^{r^{(k)}} \in \mathcal{F}^+$, for $r_j^{(k)} \leftarrow \mathbb{F}$. It follows that all e coefficients $s_{d+1}, \dots, s_{n-1}$ are uniformly distributed and independent of each other,

and their distribution only depends on the internal randomness sampled in line 5. It remains to consider the contribution of the non-deterministic components of each **ZEnc**'s output fault vector. Since **ZEnc** is assumed to be fault robust, for all $i \in [n]$ it holds true that the non-deterministic component $\mathbf{v}^{(i)''}$ is either zero or corresponds to a polynomial that is indistinguishable from a polynomial whose coefficients with indices in $d+1, \dots, n-1$ are ϵ -indistinguishable from uniform and only depend on the internal randomness of the gadget. These faults propagate linearly to the output $(S_i)_{i \in [n]}$ and are independent of any randomness used outside their respective **ZEnc**. All these non-deterministic components sum up to the non-deterministic part of the output fault vector. Since the random coins sampled in line 5 and each **ZEnc** execution are independent of each other, Lemma 24 guarantees that their sum corresponds to a polynomial whose coefficients with degree $> d$ are ϵ -indistinguishable from uniform.

We continue to prove privacy and circuit fault invariance to ensure that probes do not reveal information even in the presence of faults.

Lemma 13. *Let $t \leq d$. Assume that **ZEnc** outputs a polynomial encoding of zero which is t -wise independent and uniformly distributed. Then the gadget presented in Algorithm 1 is t -SNI.*

Lemma 14. *If **ZEnc** is circuit fault-invariant with respect to $\mathcal{F}^{+2}$, then the gadget presented in Algorithm 1 is circuit fault-invariant with respect to $\mathcal{F}^+$.*

The proofs of Lemma 13 and Lemma 14 are close to the original proofs in [12] but rely on our adapted circuit fault-invariance instead. They appear in Supp. G.2 and Supp. G.3, respectively.

6 Multiplication Gadget

We begin the construction of our adapted LaOla gadget by defining the matrix $\hat{A}$ which re-randomizes the higher-order coefficients of our output in case they are non-zero, and performs a degree-reduction otherwise.

6.1 Generalization of Matrix A

We define the matrix $\hat{A}_{i,j}$ similar to A in **P-Refresh** (cf. Alg. 1). Recall that we defined $A_{i,j}$ as

$$\text{Id}_{i,j} + \sum_{k=d+1}^{n-1} M_{i,j}^{(k)} = \text{Id}_{i,j} + \sum_{k=d+1}^{n-1} \left(R_i^{(k)} \cdot V_{k,j}^{-1} \right) = \text{Id}_{i,j} + \sum_{k=d+1}^{n-1} \sum_{\ell=d+1}^{n-1} r_\ell^{(k)} x_i^\ell \cdot V_{k,j}^{-1}$$

where $r_\ell^{(k)} \leftarrow \mathbb{F}$ for all $\ell \in [d+1, n], k \in [d+1, n]$. However, using $\hat{A}$, we aim to decrease the degree of F_i and re-randomize *all* coefficients of the polynomial

² An instantiation of such a zero-encoding can be found in Supp. B.

whenever one fault is present. We denote the degree *before* the degree reduction by d_{in} and the degree we aim to have after the degree reduction by d_{out} . Please note that $d_{\text{in}}, d_{\text{out}}, n$ are fixed for a choice of multiplication, e.g., for the LaOla multiplication we consider $d_{\text{in}} = d, d_{\text{out}} = \frac{d}{2}$ (cf. [12]). Hence, we can treat $d_{\text{in}}, d_{\text{out}}$ and n as public parameters.

Now, we describe the differences made to the generation of matrix A in **P-Refresh** in greater detail. We change the shares $R_j^{(k)}$ to be shares of polynomials that have random coefficients $r_{d_{\text{out}}}^{(k)}, \dots, r_{n-1}^{(k)}$ and $r_0^{(k)}$. Then, we proceed similar to **P-Refresh** (cf. Alg. 1), namely we add $R_j^{(k)} \cdot f_k$ to share F_j for any $f_k \neq 0, k \in [d_{\text{in}} + 1, n]$ and $\forall j \in [n]$. In the fault-robustness proof of **AdaptedSplitRed** (cf. Alg. 3) we will argue that doing so, the underlying polynomial consists of zero-encodings (with independent and random coefficients $r_1, \dots, r_{d_{\text{out}}}$) added to a polynomial with coefficients $\sum_{k=d_{\text{in}}+1}^{n-1} r_i^k \cdot f_k$ for $i \in [d_{\text{out}} + 1, n] \cup \{0\}$. Hence, the output polynomial is a random polynomial of degree up to $n - 1$, in case of a fault in the input to the multiplication. The detailed proof is provided in Theorem 1.

We use the $\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})}$ to represent the matrix that reduces the degree of shares from degree d_{in} to d_{out} . Altogether, we define

$$\hat{A}_{i,j} := \lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} + \sum_{k=d_{\text{in}}}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1}, \quad (1)$$

where $\hat{R}_i^{(k)} := \sum_{\ell=d_{\text{out}}+1}^{n-1} r_\ell^{(k)} \cdot x_i^\ell + r_0^{(k)}$ and $r_\ell^{(k)} \leftarrow_{\$} \mathbb{F}, \forall k \in [d_{\text{in}}, n], \ell \in [d_{\text{out}}+1, n] \cup \{0\}$. We summarize the calculation of $\hat{A}$ with respect to $d_{\text{in}}, d_{\text{out}}, n$ in Algorithm 2. The computation of the degree reduction matrix $\lambda_{i,j}^{(d \rightarrow \frac{d}{2})}$ can be found in Supp. K.

Algorithm 2 $\text{Comp}\hat{A}^{(d_{\text{in}}, d_{\text{out}}, n)}$

Input : Parameters $d_{\text{in}}, d_{\text{out}}, n \in \mathbb{N}$.

Output: Matrix $\hat{A}$.

```

2.1 for  $k \in [d_{\text{in}} + 1, n]$  do
2.2   for  $j \in [d_{\text{out}} + 1, n] \cup \{0\}$  do
2.3      $r_j^{(k)} \leftarrow_{\$} \mathbb{F}$ 
2.4     for  $i \in [n]$  do
2.5        $\hat{R}_i^{(k)} \leftarrow \hat{R}_i^{(k)} + r_j^{(k)} \cdot x_i^j$ 
2.6 for  $i \in [n]$  do
2.7   for  $j \in [n]$  do
2.8      $\bar{R}_{i,j} \leftarrow \sum_{k=d_{\text{in}}+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1}$ 
2.9  $(\hat{A}_{i,j})_{i,j \in [n]} \leftarrow (\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})})_{i,j \in [n]} + (\bar{R}_{i,j})_{i,j \in [n]}$ 
2.10 return  $(\hat{A}_{i,j})_{i,j \in [n]}$ 

```

The above explanation provides the high-level ideas for Lemma 15 and 16. We defer the proofs to Supp. H due to space constraints.

Lemma 15 (Correctness of $\hat{A}$). *Let $(F_i)_{i \in [n]}$ be a valid encoding with respect to a degree d_{in} polynomial and secret $f(0)$. Moreover, let $\hat{A}$ be defined as in Algorithm 2 with respect to parameters $d_{\text{in}}, d_{\text{out}}$ and n . Then, $S_i = \sum_{j=0}^{n-1} \hat{A}_{i,j} \cdot F_j$ corresponds to a polynomial s with $s(0) = f(0)$ and $\deg(s) \leq d_{\text{out}}$.*

Lemma 16 (Re-randomization by $\hat{A}$). *Let $(F_i)_{i \in [n]}$ be an encoding with respect to a polynomial of degree d' , with $d' \in [d_{\text{in}} + 1, n]$. Let $\hat{A}$ be defined as in Algorithm 2 with respect to $d_{\text{in}}, d_{\text{out}}, n$. Then, $S_i = \sum_{j=0}^{n-1} \hat{A}_{i,j} \cdot F_j$ corresponds to a polynomial with random coefficients $s_0, s_{d_{\text{out}}+1}, \dots, s_{n-1}$.*

6.2 Adapted LaOla Multiplication

In Algorithm 3, we give the adapted SplitRed algorithm. Changes made in comparison to [12] are highlighted in teal. Note that the original **SplitRed** algorithm in [12] also outputs sharings of degree up to $n - 1$, for more details see Supp. L. **SWMult** and **SWComp** are gadgets from [12] where only share-wise operations are involved, and are restated in Supp. H for completeness.

Lemma 17 (Correctness of AdaptedSplitRed $_{\hat{A}}$ (cf. Alg. 3)). *Let $n, d \in \mathbb{N}$ be even numbers³. Let $(F_i)_{i \in [n]}$ denote an encoding with respect to a polynomial f of degree d and let $\left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})}\right)_{i \in [n], j \in [n]}$ be a matrix such that $\left(\sum_{j=0}^{n-1} \lambda_{i,j}^{(d \rightarrow \frac{d}{2})} \cdot F_j\right)_{i \in [n]}$ yields a polynomial encoding corresponding to a degree- $\frac{d}{2}$ polynomial with constant term $f(0)$. Moreover, let $\hat{A}$ be the output of **CompA** $(d, \frac{d}{2}, n)$. On input $(F_i)_{i \in [n]}$, **AdaptedSplitRed** (cf. Alg. 3) outputs two sharings $(F'_i)_{i \in [n]}$ and $(F''_i)_{i \in [n]}$ corresponding to polynomials f', f'' respectively, such that $f' + f''$ is of degree $\frac{d}{2}$ and $(f' + f'')(0) = f(0)$.*

We now show the SNI property of the multiplication gadget. Since the proofs of the following lemmas and Theorem 1 are close to the original proofs in [12], we refer to Supp. H for the formal proofs.

Lemma 18. *Let $t \leq d$. Assume that **ZEnc** outputs a polynomial encoding of zero which is t -wise independent and uniformly distributed. Then, the multiplication gadget **Mult** – **LaOla** depicted in Algorithm 4 is t -SNI.*

The proof appears in Supp. H.1.

Lemma 19 (Fault invariance of **SWMult and **SWComp** [12]).** ***SWMult** is circuit fault-invariant with respect to $\mathcal{F}^+$. If **sZEnc** is circuit fault-invariant with respect to $\mathcal{F}^+$, then **SWComp** is circuit fault-invariant with respect to $\mathcal{F}^+$.*

³ For simplicity, we present the gadget assuming the parameters are even. However, it can be adapted to handle arbitrary (including odd) parameter values as well.

Algorithm 3 AdaptedSplitRed $\hat{A}$ for $n = d + e + 1$ and even d, n .

Input : Degree- d shares of $\mathbf{v} = (v_0, \dots, v_d)$ as $(F_i)_{i \in [n]}$.

Output: Shares $(F'_i), (F''_i)_{i \in [n]}$ s.t. $f(0) = f'(0) + f''(0)$, $\tilde{f} := f' + f''$ has degree $\frac{d}{2}$.

```

3.1 for  $j \in [\frac{n}{2}]$  do
3.2    $(\hat{Z}_i^j)_{i \in [n]} \leftarrow \mathbf{ZEnc}_n^d$ 
3.3 for  $j \in [\frac{n}{2}]$  do
3.4    $(\tilde{Z}_i^j)_{i \in [n]} \leftarrow \mathbf{ZEnc}_n^{\frac{d}{2}}$ 
3.5    $(Z_i^j)_{i \in [n]} \leftarrow (\tilde{Z}_i^j)_{i \in [n]} + (\hat{Z}_i^j)_{i \in [n]}$ 
3.6 for  $j \in [\frac{n}{2}]$  do
3.7   for  $i \in [n]$  do
3.8      $\mathcal{F}_i^j \leftarrow \hat{A}_{i,j} \cdot F_j$ 
3.9      $\mathcal{F}_i^j \leftarrow \mathcal{F}_i^j + Z_i^j$ 
3.10 for  $j \in [\frac{n}{2}]$  do
3.11    $(F'_i)_{i \in [n]} \leftarrow (F'_i)_{i \in [n]} + (\mathcal{F}_i^j)_{i \in [n]}$ 
3.12 for  $j \in [\frac{n}{2}]$  do
3.13    $(\tilde{Z}_i^{j+\frac{n}{2}})_{i \in [n]} \leftarrow \mathbf{ZEnc}_n^{\frac{d}{2}}$ 
3.14    $(Z_i^{j+\frac{n}{2}})_{i \in [n]} \leftarrow (\tilde{Z}_i^{j+\frac{n}{2}})_{i \in [n]} - (\hat{Z}_i^j)_{i \in [n]}$ 
3.15 for  $j \in [\frac{n}{2}]$  do
3.16   for  $i \in [n]$  do
3.17      $\mathcal{F}_i^{j+\frac{n}{2}} \leftarrow \hat{A}_{i,j+\frac{n}{2}} \cdot F_{j+\frac{n}{2}}$ 
3.18      $\mathcal{F}_i^{j+\frac{n}{2}} \leftarrow \mathcal{F}_i^{j+\frac{n}{2}} + Z_i^{j+\frac{n}{2}}$ 
3.19 for  $j \in [\frac{n}{2}]$  do
3.20    $(F''_i)_{i \in [n]} \leftarrow (F''_i)_{i \in [n]} + (\mathcal{F}_i^{j+\frac{n}{2}})_{i \in [n]}$ 
3.21 return  $(F'_i)_{i \in [n]}, (F''_i)_{i \in [n]}$ 

```

Lemma 20. Assume that $\mathbf{ZEnc}$ is circuit fault-invariant with respect to $\mathcal{F}^+$. Then the composition of $\mathbf{Comp}\hat{A}$ and **AdaptedSplitRed** is also circuit fault-invariant with respect to $\mathcal{F}^+$.

The proofs of Lemma 19 and Lemma 20 can be found in Supp. H.2.

Theorem 1 (Region fault robustness of Mult – LaOla). Assume that $\mathbf{ZEnc}$ is $(e, \delta^{\mathbf{ZEnc}} = 0, \epsilon^{\mathbf{ZEnc}} = 0)$ -threshold fault robust with respect to the fault set $\mathcal{F}^+$. Consider the gadget **Mult – LaOla** described in Algorithm 4 as a single region. Then, **Mult – LaOla** fulfills (e, δ', ϵ') -region⁴ fault robustness with respect to fault set $\mathcal{F}^+$, where $\delta' \leq \frac{4}{|\mathbb{F}|^{n-d-1}}$ and $\epsilon' \lesssim \frac{n}{|\mathbb{F}|}$.

The proof of Theorem 1 is deferred to Supp. H.3. Notably, $\mathbf{ZEnc}$ can be instantiated as the circuit fault invariant, $(e, 0, 0)$ -region fault robust and SNI gadget described in Supp. B.

⁴ The number of regions for **Mult – LaOla** is 1.

Algorithm 4 Mult – LaOla for $n = d + e + 1$, with even d, n .

Input : Degree- d shares of $\mathbf{v} = (v_i)_{i \in [d]}$ as $(F_i)_{i \in [n]}$ and $\mathbf{v}' = (v'_i)_{i \in [d]}$ as $(G_i)_{i \in [n]}$.

Output: Degree- d shares of $(v_0 \cdot v'_0, \dots, v_{k-1} \cdot v'_{k-1})$ as $(Q_i)_{i \in [n]}$.

```

4.1  $\hat{A}_1 \leftarrow \text{Comp}\hat{A}(d, \frac{d}{2}, n)$ 
4.2  $((F'_i)_{i \in [n]}, (F''_i)_{i \in [n]}) \leftarrow \text{AdaptedSplitRed}_{\hat{A}_1}((F_i)_{i \in [n]})$ 
4.3  $\hat{A}_2 \leftarrow \text{Comp}\hat{A}(d, \frac{d}{2}, n)$ 
4.4  $((G'_i)_{i \in [n]}, (G''_i)_{i \in [n]}) \leftarrow \text{AdaptedSplitRed}_{\hat{A}_2}((G_i)_{i \in [n]})$ 
4.5  $(H^0_i)_{i \in [n]} \leftarrow \text{SWMult}((F'_i)_{i \in [n]}, (G'_i)_{i \in [n]})$ 
4.6  $(H^1_i)_{i \in [n]} \leftarrow \text{SWMult}((F'_i)_{i \in [n]}, (G''_i)_{i \in [n]})$ 
4.7  $(H^2_i)_{i \in [n]} \leftarrow \text{SWMult}((F''_i)_{i \in [n]}, (G'_i)_{i \in [n]})$ 
4.8  $(H^3_i)_{i \in [n]} \leftarrow \text{SWMult}((F''_i)_{i \in [n]}, (G''_i)_{i \in [n]})$ 
4.9  $(Q_i)_{i \in [n]} \leftarrow \text{SWComp}((H^0_i)_{i \in [n]}, (H^1_i)_{i \in [n]}, (H^2_i)_{i \in [n]}, (H^3_i)_{i \in [n]})$ 
4.10 return  $(Q_i)_{i \in [n]}$ 

```

Failure probabilities To conclude, we get with Lemma 12 and Theorem 1 that **P-Refresh** is fault robust with failure probability $\delta = 1 - (1 - \delta_{\text{ZEnc}})^n + 2(|\mathbb{F}|^{-1} + \epsilon_{\text{ZEnc}})^{n-d-1} = 2(|\mathbb{F}|^{-1})^e$ and **Mult – LaOla** is fault robust with failure probability $\delta' \leq \frac{4}{|\mathbb{F}|^{n-d-1}}$ for $\delta_{\text{ZEnc}} = \epsilon_{\text{ZEnc}} = 0$. For the c -wise sequential composition $\delta_c = 1 - (1 - \delta)^c$ by Lemma 7. This allows us to compute an upper bound p on the overall failure probability according to Lemma 8.

7 Towards Formal Verification

The focus of this work are profound algebraic proofs that support the security of **P-Refresh** and **Mult – LaOla** for arbitrary choices of valid security parameters d, e, n , support points, and arbitrary fault-invariant **ZEnc** algorithms.

Formal verification tools can be used to verify concrete instances of gadgets for specific security parameters d, e, n . However, no tool adequately supports polynomial masking: The sole tools **Ironmask**⁺ and **Verica**⁺ verify for concrete, inlined support points and in Boolean field $\mathbb{F}_2$ only [10]. Using them would (a) require new front-ends that bit-blast implementations in $\mathbb{F}_{2^k}$ to $\mathbb{F}_2$; resulting in a significant blow-up in circuit size and verification time and (b) provide limited assurance specific to chosen parameters since enumeration of all parameters is intractable [27]. Furthermore, these tools are incapable of verifying fault robustness with failure probabilities $\delta > 0$ and therefore leave a central contribution of this work unaddressed. However, the recent tool **eVer** [27] specifically addresses verification of side-channel security for arbitrary fields $\mathbb{F}_{p^k}$ and symbolic support points. In ongoing work, **eVer** is extended to verify fault robustness with failure probabilities and (weak) fault-resilient (S)NI of **P-Refresh** and **Mult – LaOla**, which requires novel methods to circumvent above mentioned limitations.

References

1. Althaus, H., Leake, R.J.: Inverse of a finite-field vandermonde matrix (corresp.). *IEEE Trans. Inf. Theory* **15**(1), 173 (1969). <https://doi.org/10.1109/TIT.1969.1054253>
2. Andrychowicz, M., Dziembowski, S., Faust, S.: Circuit compilers with $o(1/\log(n))$ leakage rate. In: Fischlin, M., Coron, J. (eds.) *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Vienna, Austria, May 8-12, 2016, Proceedings, Part II. *Lecture Notes in Computer Science*, vol. 9666, pp. 586–615. Springer (2016). https://doi.org/10.1007/978-3-662-49896-5_21
3. Arnold, P., Berndt, S., Eisenbarth, T., Faust, S., Gourjon, M., Andresen, J., Landthaler, E., Micheli, E., Orlt, M., Pauls, P., Wirschem, K., Zhao, L.: UP TO 50% OFF: efficient implementation of polynomial masking. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2026**(1), – (2026). <https://doi.org/10.46586/TCHES.V2026.I1.->
4. Arnold, P., Berndt, S., Eisenbarth, T., Orlt, M.: Polynomial sharings on two secrets: Buy one, get one free. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2024** (2024)
5. Barthe, G., Belaïd, S., Dupressoir, F., Fouque, P., Grégoire, B., Strub, P.: Verified proofs of higher-order masking. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I. *Lecture Notes in Computer Science*, vol. 9056, pp. 457–485. Springer (2015). https://doi.org/10.1007/978-3-662-46800-5_18
6. Barthe, G., Belaïd, S., Dupressoir, F., Fouque, P., Grégoire, B., Strub, P., Zucchini, R.: Strong non-interference and type-directed higher-order masking. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, Vienna, Austria, October 24-28, 2016. pp. 116–129. ACM (2016). <https://doi.org/10.1145/2976749.2978427>
7. Béguinot, J., Cheng, W., Guilley, S., Rioul, O.: Formal security proofs via doebelin coefficients: - optimal side-channel factorization from noisy leakage to random probing. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VI. *Lecture Notes in Computer Science*, vol. 14925, pp. 389–426. Springer (2024). https://doi.org/10.1007/978-3-031-68391-6_12, https://doi.org/10.1007/978-3-031-68391-6_12
8. Belaïd, S., Cassiers, G., Rivain, M., Taleb, A.R.: Unifying freedom and separation for tight probing-secure composition. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference*, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. *Lecture Notes in Computer Science*, vol. 14083, pp. 440–472. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_15
9. Belaïd, S., Feldtkeller, J., Güneysu, T., Guinet, A., Richter-Brockmann, J., Rivain, M., Sasdrich, P., Taleb, A.R.: Formal definition and verification for combined random fault and random probing security. In: Chung, K., Sasaki, Y. (eds.) *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security*, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. *Lecture Notes in Computer Science*, vol. 15490, pp. 167–200. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_6

10. Belaïd, S., Feldtkeller, J., Güneysu, T., Guinet, A., Richter-Brockmann, J., Rivain, M., Sasdrich, P., Taleb, A.R.: Formal definition and verification for combined random fault and random probing security. *IACR Cryptol. ePrint Arch.* p. 757 (2024). <https://eprint.iacr.org/2024/757>
11. Ben-Or, M., Goldwasser, S., Wigderson, A.: Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In: Simon, J. (ed.) *Proceedings of the 20th Annual ACM Symposium on Theory of Computing*, May 2-4, 1988, Chicago, Illinois, USA. pp. 1–10. ACM (1988). <https://doi.org/10.1145/62212.62213>
12. Berndt, S., Eisenbarth, T., Faust, S., Gourjon, M., Orlt, M., Seker, O.: Combined fault and leakage resilience: Composability, constructions and compiler. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. Lecture Notes in Computer Science*, vol. 14083, pp. 377–409. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_13
13. Brian, G., Faonio, A., Obremski, M., Ribeiro, J., Simkin, M., Skórski, M., Venturi, D.: The mother of all leakages: How to simulate noisy leakages via bounded leakage (almost) for free. *IEEE Trans. Inf. Theory* **68**(12), 8197–8227 (2022). <https://doi.org/10.1109/TIT.2022.3193848>
14. Cassiers, G., Dhooghe, S., Moos, T., Saha, S., Standaert, F.: Fly away: Lifting fault security through canaries and the uniform random fault model. *IACR Commun. Cryptol.* **2**(3), 19 (2025). <https://doi.org/10.62056/AN-49QXQI>, <https://doi.org/10.62056/an-49qxqi>
15. Dehbaoui, A., Dutertre, J., Robisson, B., Tria, A.: Electromagnetic transient faults injection on a hardware and a software implementations of AES. In: Bertoni, G., Gierlichs, B. (eds.) *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography*, Leuven, Belgium, September 9, 2012. pp. 7–15. IEEE Computer Society (2012). <https://doi.org/10.1109/FDTC.2012.15>
16. Dhooghe, S., Nikova, S.: My gadget just cares for me - how NINA can prove security against combined attacks. In: Jarecki, S. (ed.) *Topics in Cryptology - CT-RSA 2020 - The Cryptographers' Track at the RSA Conference 2020, San Francisco, CA, USA, February 24-28, 2020, Proceedings. Lecture Notes in Computer Science*, vol. 12006, pp. 35–55. Springer (2020). https://doi.org/10.1007/978-3-030-40186-3_3
17. Dobraunig, C., Eichlseder, M., Groß, H., Mangard, S., Mendel, F., Primas, R.: Statistical ineffective fault attacks on masked AES with fault countermeasures. In: Peyrin, T., Galbraith, S.D. (eds.) *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 11273, pp. 315–342. Springer (2018). https://doi.org/10.1007/978-3-030-03329-3_11
18. Duc, A., Dziembowski, S., Faust, S.: Unifying leakage models: From probing attacks to noisy leakage. *J. Cryptol.* **32**(1), 151–177 (2019). <https://doi.org/10.1007/s00145-018-9284-1>
19. Dumont, M., Lisart, M., Maurine, P.: Electromagnetic fault injection : How faults occur. In: *2019 Workshop on Fault Diagnosis and Tolerance in Cryptography, FDTC 2019, Atlanta, GA, USA, August 24, 2019*. pp. 9–16. IEEE (2019). <https://doi.org/10.1109/FDTC.2019.00010>

20. Dziembowski, S., Faust, S., Skorski, M.: Noisy leakage revisited. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9057, pp. 159–188. Springer (2015). https://doi.org/10.1007/978-3-662-46803-6_6
21. Faust, S., Ortl, M., Wirschem, K., Zhao, L.: All-you-can-compute: Packed secret sharing for combined resilience. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2025**(2), 420–459 (2025). <https://doi.org/10.46586/TCHES.V2025.I2.420-459>
22. Feldtkeller, J., Güneysu, T., Moos, T., Richter-Brockmann, J., Saha, S., Sasdrich, P., Standaert, F.: Combined private circuits - combined security refurbished. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023*, Copenhagen, Denmark, November 26-30, 2023. pp. 990–1004. ACM (2023). <https://doi.org/10.1145/3576915.3623129>
23. Feldtkeller, J., Richter-Brockmann, J., Sasdrich, P., Güneysu, T.: CINI MINIS: domain isolation for fault and combined security. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022*, Los Angeles, CA, USA, November 7-11, 2022. pp. 1023–1036. ACM (2022). <https://doi.org/10.1145/3548606.3560614>
24. Feldtkeller, J., Richter-Brockmann, J., Sasdrich, P., Güneysu, T.: Combined threshold implementation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2024**(4), 307–334 (2024). <https://doi.org/10.46586/TCHES.V2024.I4.307-334>
25. Franklin, M.K., Yung, M.: Communication complexity of secure computation (extended abstract). In: Kosaraju, S.R., Fellows, M., Wigderson, A., Ellis, J.A. (eds.) *Proceedings of the 24th Annual ACM Symposium on Theory of Computing*, May 4-6, 1992, Victoria, British Columbia, Canada. pp. 699–710. ACM (1992). <https://doi.org/10.1145/129712.129780>
26. Gandolfi, K., Mourtel, C., Olivier, F.: Electromagnetic analysis: Concrete results. In: Koc, Ç.K., Naccache, D., Paar, C. (eds.) *Cryptographic Hardware and Embedded Systems - CHES 2001, Third International Workshop*, Paris, France, May 14-16, 2001, Proceedings. Lecture Notes in Computer Science, vol. 2162, pp. 251–261. Springer (2001). https://doi.org/10.1007/3-540-44709-1_21
27. Gourjon, M., Ortl, M., Pauls, P., Treff, A.: eVer: Universal and automated verification of side-channel security for additive, inner product, polynomial and general code-based masking. *Cryptology ePrint Archive*, Paper 2026/208 (2026), <https://eprint.iacr.org/2026/208>
28. Ishai, Y., Sahai, A., Wagner, D.A.: Private circuits: Securing hardware against probing attacks. In: Boneh, D. (ed.) *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference*, Santa Barbara, California, USA, August 17-21, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2729, pp. 463–481. Springer (2003). https://doi.org/10.1007/978-3-540-45146-4_27
29. Kocher, P.C.: Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In: Koblitz, N. (ed.) *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference*, Santa Barbara, California, USA, August 18-22, 1996, Proceedings. Lecture Notes in Computer Science, vol. 1109, pp. 104–113. Springer (1996). https://doi.org/10.1007/3-540-68697-5_9
30. Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener, M.J. (ed.) *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference*, Santa Barbara, California, USA, August 15-19, 1999, Proceedings.

- Lecture Notes in Computer Science, vol. 1666, pp. 388–397. Springer (1999). https://doi.org/10.1007/3-540-48405-1_25
31. Prest, T., Goudarzi, D., Martinelli, A., Passelègue, A.: Unifying leakage models on a rényi day. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I. Lecture Notes in Computer Science, vol. 11692, pp. 683–712. Springer (2019). https://doi.org/10.1007/978-3-030-26948-7_24
 32. Richter-Brockmann, J., Feldtkeller, J., Sasdrich, P., Güneysu, T.: VERICA - verification of combined attacks automated formal verification of security against simultaneous information leakage and tampering. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2022**(4), 255–284 (2022). <https://doi.org/10.46586/TCHES.V2022.I4.255-284>
 33. Seker, O., Fernandez-Rubio, A., Eisenbarth, T., Steinwandt, R.: Extending glitch-free multiparty protocols to resist fault injection attacks. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2018**(3), 394–430 (2018). <https://doi.org/10.13154/TCHES.V2018.I3.394-430>
 34. Shamir, A.: How to share a secret. *Commun. ACM* **22**(11), 612–613 (1979). <https://doi.org/10.1145/359168.359176>
 35. Tollec, S., Hadzic, V., Nasahl, P., Asavoe, M., Bloem, R., Couroussé, D., Heydemann, K., Jan, M., Mangard, S.: Fault-resistant partitioning of secure cpus for system co-verification against faults. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* **2024**(4), 179–204 (2024). <https://doi.org/10.46586/TCHES.V2024.I4.179-204>, <https://doi.org/10.46586/tches.v2024.i4.179-204>
 36. Toprakhisar, D., Nikova, S., Nikov, V.: Combined stability: Protecting against combined attacks. *IACR Cryptol. ePrint Arch.* p. 1692 (2025), <https://eprint.iacr.org/2025/1692>
 37. Zussa, L., Dutertre, J., Clédière, J., Tria, A.: Power supply glitch induced faults on FPGA: an in-depth analysis of the injection mechanism. In: *2013 IEEE 19th International On-Line Testing Symposium (IOLTS)*, Chania, Crete, Greece, July 8–10, 2013. pp. 110–115. IEEE (2013). <https://doi.org/10.1109/IOLTS.2013.6604060>

A Alternative Definition of Fault Robustness

In this section, we propose an alternative definition of fault robustness that more closely resembles the one of Berndt et al. [12], and discuss its relation with the one in Definition 10. For simplicity, we only consider gadgets with two inputs and one output here.

Definition 15 (((e, ϵ) –fault robustness, alternative). *Let G be a gadget with one output sharing and up to two input sharings. Then G is (e, ϵ) –fault-robust with respect to $\mathcal{F}$ if for any couple of valid (polynomial) sharings $\mathbf{F}$ and $\mathbf{H}$, then the output $\mathbf{Y} \leftarrow G^R(\mathbf{F}, \mathbf{H})$, $R \leftarrow^* U$, is also valid. For any couple of vectors $\boldsymbol{\xi}^F, \boldsymbol{\xi}^H$ and for any $T \in \mathcal{A}(\mathcal{F})$ with $|T| \leq e$, and resulting $\mathbf{Y} + \boldsymbol{\omega} + \boldsymbol{\omega}' \leftarrow T[G^R](\mathbf{F} + \boldsymbol{\xi}^F, \mathbf{H} + \boldsymbol{\xi}^H)$ it holds that there are numbers $s_1 \in \mathbb{N}$ and $s_2 \in \mathbb{N}$ with $s_1 + s_2 \leq |T|$ such that:*

- i) $\text{weight}(\boldsymbol{\omega}) \leq s_1$ (linear part)
- ii) and $\boldsymbol{\omega}'$ is either equal to $\boldsymbol{\xi}^F + \boldsymbol{\xi}^H$ or $\boldsymbol{\omega}'$ is equal to $\bar{\boldsymbol{\omega}}'$, where $\bar{\boldsymbol{\omega}}'$ is produced by the following random experiment: Let $p_{\bar{\boldsymbol{\omega}}'} \in \mathbb{F}_q[X]$ be a polynomial with coefficients $p_i, i \in [n]$ that are chosen such that at least $e + 1 - s_2$ of the coefficients $p_{d+1}, \dots, p_{n-1}$ are chosen independently of each other and from a distribution D_ϵ that is ϵ –indistinguishable to the uniform distribution. Besides, $\boldsymbol{\omega}'$ is independent of T when $R \leftarrow^* U$. (non-linear part)

If $\epsilon = 0$, we also write e –fault robustness instead of (e, ϵ) –fault robustness. We call D_ϵ the non-linear fault distribution.

In the following proposition, we show that Definition 15 implies Definition 10 so long as $e + 1 - s_2 = n - d - 1$.

Proposition 1. *Let G be a gadget with one output sharing and up to two input sharings. Let G be (e, ϵ) –fault robust w.r.t. $\mathcal{F}$ according to Definition 15 where the distribution $\mathcal{D}_v$ of $\boldsymbol{\omega}'$ only depends on the distribution of the input faults, $\mathcal{D}_{\xi_i}, i \in [2]$, and the internal randomness of the gadget. Moreover, $e + 1 - s_2 = n - d - 1$. Then G is also (e, δ, ϵ) –fault robust w.r.t. $\mathcal{F}$ according to Definition 10, with $\delta = (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$.*

Proof. To prove this result, we divide all the possible fault attacks into three different cases.

Case 1: inputs unfaulted, internal wires unfaulted. If $\boldsymbol{\xi}_0 = \boldsymbol{\xi}_1 = 0^n$ are $(0, 0)$ –joint fault vectors and the gadget is unfaulted, then Definition 15 ensures that the output is valid too. We can then represent the output as $G^R((\mathbf{F}_i)_{i \in [l]}) + \mathbf{v}$ where $\mathbf{v} = 0$ is a $(0, 0)$ –fault vector, and thus Definition 10 holds. Notably, there is no need to argue about the distribution of the high-order coefficients of the output fault vector, as we here consider $\mathbf{v}'' = 0$ instead of $\bar{\mathbf{v}}$.

Case 2: inputs faulted but valid, internal wires unfaulted. Let ξ_0, ξ_1 be (e, ϵ) -joint fault vectors, assume that at least one among them is non zero, and that the gadget's internal wires are unfaulted. Assume w.l.o.g. that $\xi_0 \neq 0^n$. Then, with probability at most $(|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$, the vector ξ_0 corresponds to a valid encoding. In this case, we cannot guarantee that the output is faulted with a (s, ϵ) -fault vector, and therefore count this event within the failure probability δ .

Case 3: else. Let T be a fault attack on the internal wires of G , and assume that ξ_0, ξ_1 are joint $((e_i)_{i \in [2]}, \epsilon)$ -fault vectors with $|T| + e_0 + e_1 \leq e$. Moreover, assume that case 1 and case 2 do not happen, i.e. the inputs are faulted and become invalid encodings and/or the internal wires are faulted. Then, G being (e, ϵ) -fault robust according to Definition 15 implies that

$$Y + \omega + \omega' \leftarrow T[G^R](F + \xi_0, H + \xi_1)$$

with

1. $\text{weight}(\omega) \leq s_1$
2. ω' is either equal to $\xi_0 + \xi_1$ or to $\bar{\omega}'$ whose underlying polynomial has $e + 1 - s_2 = n - d - 1$ randomized high-order coefficients.

Now $\text{weight}(\omega) \leq s_1$ implies that ω is an s_1 -deterministic fault vector. If $\omega' = \xi_0 + \xi_1$, then ω' is an $(e_0 + e_1, \epsilon)$ -fault vector,⁵ which implies that $\omega + \omega'$ is a $(e_0 + e_1 + s_1 \leq e, \epsilon)$ -fault vector, making Definition 10 hold. If $\omega' = \bar{\omega}$, then $\omega + \omega'$ is a (e, ϵ) -fault vector by definition. Furthermore, in the latter case, ω' only depends on $\mathcal{D}_{\xi_i}$, $i \in [2]$ and the internal randomness of the gadget by assumption of G being fault robust according to definition 15.

This concludes the proof.

We highlight that the gadgets presented in [21,3] were adapted to fulfill the requirement of $e + 1 - s_2 = n - d + 1$ and hence the assumptions for Proposition 1 are met.

We now consider a multi-input version of Definition 15.

Definition 16. Let G be a gadget with one output sharing and up to k input sharings. Then G is (e, ϵ) -fault robust with respect to $\mathcal{F}$ if for any tuple of valid (polynomial) sharings $F_i, i \in [k]$, then the output $Y \leftarrow G^R((F_i)_{i \in [k]}), R \leftarrow^s U$, is also valid. For any tuple of vectors $\xi^{F_i}, i \in [k]$ and for any $T \in \mathcal{A}(\mathcal{F})$ with $|T| \leq e$, and resulting $Y + \omega + \omega' \leftarrow T[G^R]((F_i + \xi^{F_i})_{i \in [k]})$ it holds that there are numbers $s_1, s_2 \in \mathbb{N}$ with $s_1 + s_2 \leq |T|$ such that

- i) $\text{weight}(\omega) \leq s_1$ (linear part)
- ii) and ω' is either equal to $\sum_{i \in [k]} \xi^{F_i}$ or ω' is equal to $\bar{\omega}'$, where $\bar{\omega}'$ is produced by the following random experiment: Let $p_{\bar{\omega}'} \in \mathbb{F}_q[X]$ be a polynomial with coefficients $p_i, i \in [n]$ that are chosen such that at least $e + 1 - s_2$ of the coefficients $p_{d+1}, \dots, p_{n-1}$ are chosen independently of each other and from a distribution $\mathcal{D}_{\omega'}$ that is ϵ -indistinguishable to the uniform distribution.

⁵ Note that here ϵ is preserved under sum because joint fault vectors have independent high-order coefficients.

We are able to show composability of Definition 16.

Notably, e -linear fault robustness is not included in Definition 16, as we do not allow a linear combination of elements there.

Lemma 21. *Let $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ be a wire-independent fault set. Let G' be an (e, ϵ) -fault robust gadget with respect to $\mathcal{F}$ and k inputs and one output. Moreover, let the distribution $\mathcal{D}_v$ of ω' (case ii) of Definition 16) only depend on the distribution of the input faults, $\mathcal{D}_{\omega_i + \omega'_i}, i \in [k]$, and the internal randomness of the gadget. For each $i \in [k]$, let G_i be an (e, ϵ) -fault robust gadget with respect to $\mathcal{F}$ and l_i inputs and one output. Furthermore, let the distribution $\mathcal{D}_{\omega'_i}$ of ω'_i (case ii) of Definition 16) only depend on the distribution of the input faults and the internal randomness of the gadget. Moreover, let each gadget use independent internal randomness and independent inputs. Then*

$$G(\cdot) = G'(G_0(\cdot), \dots, G_{k-1}(\cdot))$$

is (e, ϵ) -fault robust with respect to $\mathcal{F}$ except with probability $\delta = 1 - \left(1 - \frac{1}{|\mathbb{F}|^{n-d-1}}\right)^k$.

Proof. For each $i \in [k]$, let $\mathbf{F}_j^i, j \in [l_i]$ denote valid (polynomial) input sharings. Let $\boldsymbol{\xi}^{\mathbf{F}_j^i}$ denote (arbitrary) fault vectors. Consider any attack vector $T \in \mathcal{A}(\mathcal{F})$ with $|T| \leq e$.

Since $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$, we can decompose T into disjoint sub-attacks

$$T = T' \cup \bigcup_{i \in [k]} T_i$$

where T_i acts inside G_i and T' acts inside the core gadget G' , hence $|T| = |T'| + \sum_{i \in [k]} |T_i|$. According to Definition 16, we need to show that for $\mathbf{Y} + \omega + \omega' \leftarrow T[G^R]((\mathbf{F}_i + \boldsymbol{\xi}^{\mathbf{F}_i})_{i \in [k]})$ there exists $s_1, s_2 \in \mathbb{N}$ such that $s_1 + s_2 \leq |T|$ and

- i) $\text{weight}(\omega) \leq s_1$ (linear part)
- ii) and ω' is either equal to $\sum_{i \in [k]} \boldsymbol{\xi}^{\mathbf{F}_i}$ or ω' is equal to $\overline{\omega'}$, where $\overline{\omega'}$ is produced by the following random experiment: Let $p_{\overline{\omega'}} \in \mathbb{F}_q[X]$ be a polynomial with coefficients $p_i, i \in [n]$ that are chosen such that at least $e + 1 - s_2$ of the coefficients $p_{d+1}, \dots, p_{n-1}$ are chosen independently of each other and from a distribution $\mathcal{D}_\epsilon$ that is ϵ -indistinguishable to the uniform distribution and only dependent on $R \leftarrow_s U$.

Preprocessing Stage. Since for each $i \in [k]$ it holds that $|T_i| \leq e$, we can apply the (e, ϵ) -fault robustness of $G_i, i \in [k]$. Thus, there exist $s_1^i, s_2^i \in \mathbb{N}$ such that $s_1^i + s_2^i \leq |T_i|$ with

$$\mathbf{Y}_i + \omega_i + \omega'_i \leftarrow T_i[G_i^{R_i}]((\mathbf{F}_j^i + \boldsymbol{\xi}^{\mathbf{F}_j^i})_{j \in [l_i]})$$

such that

- i) $\text{weight}(\omega_i) \leq s_1^i$ (linear part)

- ii) and ω'_i is either equal to $\sum_{j \in [l_i]} \xi^{F_j^i}$ or ω'_i is equal to $\overline{\omega'}_i$, where $\overline{\omega'}_i$ is produced by the following random experiment: Let $p_{\overline{\omega'}_i} \in \mathbb{F}_q[X]$ be a polynomial with coefficients $p_j^{(i)}, j \in [n]$ that are chosen such that at least $e+1-s'_2$ of the coefficients $p_{d+1}^{(i)}, \dots, p_{n-1}^{(i)}$ are chosen independently of each other and from a distribution $\mathcal{D}_{\omega'_i}$. By assumption, these distributions are ϵ -indistinguishable to the uniform distribution and only dependent on the corresponding input fault vectors and the internal randomness of the gadget.

Let BAD^i denote the event the vector $\omega_i + \omega'_i \neq 0$ corresponds to a polynomial of degree up to d . Notably, this event BAD^i occurs in the case that the high order coefficients corresponding to ω'_i equal the high order coefficients corresponding to ω_i , and thus with probability $\frac{1}{|\mathbb{F}|^{n-d-1}}$ for each $i \in [k]$. Moreover, by assumption, $p_{\overline{\omega'}_i}$ is $\mathcal{D}_{\omega'_i}$ distributed, where the distributions are independent of each other.

Core Stage. By assumption, the outputs $\mathbf{Y}_i + \omega_i + \omega'_i$ of $G_i, i \in [k]$ are the inputs of G' . We consider the case where none of the events $\text{BAD}^i, i \in [k]$ occurs. Since $|T'| \leq e$, we can use the (e, ϵ) -fault robustness of G' . Thus, there exist numbers s'_1, s'_2 such that $s'_1 + s'_2 \leq |T'|$ such that

$$\mathbf{Y}' + \mathbf{v} + \mathbf{v}' \leftarrow T'[G'^{R'}]((\mathbf{Y}_i + \omega_i + \omega'_i)_{i \in [k]})$$

with attack vector T' , the output can be written as the valid polynomial sharing $\mathbf{Y}' \leftarrow G'^{R'}((\mathbf{Y}_i)_{i \in [k]})$ plus $\mathbf{v}, \mathbf{v}'$ such that

- i) $\text{weight}(\mathbf{v}) \leq s'_1$ (linear part)
 ii) and $\mathbf{v}'$ is either equal to $\sum_{i \in [k]} (\omega_i + \omega'_i)$ or $\mathbf{v}'$ is equal to $\overline{\mathbf{v}'}$, where $\overline{\mathbf{v}'}$ is produced by the following random experiment: Let $p_{\overline{\mathbf{v}'}} \in \mathbb{F}_q[X]$ be a polynomial with coefficients $p'_j, j \in [n]$ that are chosen such that at least $e+1-s'_2$ of the coefficients $p'_{d+1}, \dots, p'_{n-1}$ are chosen independently of each other and from a distribution $\mathcal{D}_{\mathbf{v}'}$ that is ϵ -indistinguishable to the uniform distribution and only dependent on $R' \leftarrow_s U$ and the distribution of the input fault vectors $\mathcal{D}_{\omega_i + \omega'_i}$.

Again, if $\mathbf{v}'$ is equal to $\overline{\mathbf{v}'}$, then with probability $\text{BAD}^k = \frac{1}{|\mathbb{F}|^{n-d-1}}$, $\mathbf{v}'$ corresponds to a polynomial of degree at most d .

Conclusion. Let BAD denote the event that at least one of the events $\text{BAD}^i, i \in [k+1]$ happens. It holds true that

$$\Pr[\text{BAD}] = 1 - \prod_{i \in [k+1]} (1 - \Pr[\text{BAD}^i]) \leq 1 - \left(1 - \frac{1}{|\mathbb{F}|^{n-d-1}}\right)^{k+1}.$$

We set $\Pr[\text{BAD}] = \delta$ to be the failure probability and assume for the following cases, that this event does not happen. Thus, with probability $1 - \delta$, one of the following cases occurs.

Case 1: Assume $\mathbf{v}'$ is equal to $\overline{\mathbf{v}'}$, which is produced by the random experiment. In this case, we set $\omega := \mathbf{v}$, $\omega' := \mathbf{v}'$, $s_1 := s'_1$ and $s_2 := s'_2$. By assumption, $s'_1 + s'_2 \leq |T'| \leq |T|$. Moreover, $\text{weight}(\omega) = \text{weight}(\mathbf{v}) \leq s'_1 = s_1$ and $\omega' = \mathbf{v}'$

is indistinguishable from a sharing with coefficients $p'_j, j \in [n]$ that are chosen such that at least $e + 1 - s'_2$ of the coefficients $p'_{d+1}, \dots, p'_{n-1}$ are chosen independently of each other and from a distribution $\mathcal{D}_{\mathbf{v}'}$ that is ϵ -indistinguishable to the uniform distribution and only dependent on the distribution of the input faults $\mathcal{D}_{\omega'_i}, i \in [k]$ and the internal randomness $R' \leftarrow_{\$} U$. Notably, by fault robustness of $G_i, i \in [k]$ the distributions $\mathcal{D}_{\omega'_i}$ are only dependent of the internal randomness of the (sub)gadget and the input fault distributions $\mathcal{D}_{\xi^{F_j^i}}$. By assumption, the internal randomness of the (sub)gadgets is independent. Thus, in this case all requirements are met and G is (e, ϵ) fault robust.

Case 2: In this case, $\mathbf{v}' = \sum_{i \in [k]} \omega_i + \sum_{i \in [k]} \omega'_i$. We set $\omega := \sum_{i \in [k]} \omega_i + \mathbf{v}$. Hence, we obtain

$$\text{weight}(\omega) = \text{weight}\left(\sum_{i \in [k]} \omega_i + \mathbf{v}\right) \leq \sum_{i \in [k]} \text{weight}(\omega_i) + \text{weight}(\mathbf{v}) \leq \sum_{i \in [k]} s_1^i + s'_1$$

Now, setting $s_1 = \sum_{i \in [k]} s_1^i + s'_1$, we get $\text{weight}(\omega) \leq s_1$. Moreover, $\omega' := \sum_{i \in [k]} \omega'_i$. We set $s_2 := \sum_{i \in [k]} s_2^{(i)}$ and highlight that $s_1 + s_2 = \sum_{i \in [k]} s_1^i + s'_1 + \sum_{i \in [k]} s_2^{(i)} \leq \sum_{i \in [k]} |T_i| + |T'| = |T|$.

Case 2.a) At least one of the ω'_i , say ω'_{i^*} , is produced by the random experiment, such that at least $e + 1 - s_2^{(i^*)}$ of the coefficients $p_{d+1}^{(i^*)}, \dots, p_{n-1}^{(i^*)}$ are chosen independent of each other and from a distribution $\mathcal{D}_{\epsilon}^{(i^*)}$ that only depends on the distribution of the input faults, $\mathcal{D}_{\xi^{F_j^{i^*}}}$ and the internal randomness R_{i^*} of the gadget G_{i^*} . By linearity of the sharings, and the distribution of $\overline{\omega}^{(i^*)}$ being only dependent of $\mathcal{D}_{\xi^{F_j^{i^*}}}$ and R_{i^*} , which is independent of the other gadgets, also ω' is equal to $\overline{\omega}'$, where $\overline{\omega}'$ is produced by the random experiment: where the polynomial $p_{\overline{\omega}'} \in \mathbb{F}_q[X]$ has coefficients $p_i, i \in [n]$ that are chosen such that at least $e + 1 - s_2^{(i^*)} \geq e + 1 - s$ of the coefficients $p_{d+1}, \dots, p_{n-1}$ are chosen independently of each other and from a distribution $\mathcal{D}_{\mathbf{v}'}$ that is ϵ -indistinguishable to the uniform distribution and only dependent on $R \leftarrow_{\$} U$ and $\mathcal{D}_{\xi^{F_j^{i^*}}}$.

Case 2.b) Otherwise, $\omega' = \sum_{i \in [k]} \omega'_i = \sum_{i \in [k]} \sum_{j \in [l_i]} \xi^{F_j^i}$.

This finishes the proof.

B Instantiation of ZEnc

As an instantiation of our zero-encodings **ZEnc**, we can use the zero-encoding from [12]. We restate the algorithm in the following and provide proofs for fault-invariance as well as fault-robustness. For correctness and the SNI proof, we refer to [12].

Algorithm 5 $\mathbf{ZEnc}_{n,d}$ for $n = d + e + 1$

Output: Randomized (n,d) -encoding of zero $(Z_i)_{i \in [n]}$

```

5.1 initialize  $(Z_i)_{i \in [n]}$ 
5.2  $(r_1, \dots, r_d) \leftarrow \mathbb{R}^d$ 
5.3 for  $i \in [n]$  do
5.4   for  $j \in \{1, \dots, d\}$  do
5.5      $Z_i \leftarrow Z_i + r_j \lambda_{i,j}$ 
5.6 return  $(Z_i)_{i \in [n]}$ 

```

Notably, Algorithm 5 does not take an input.

Lemma 22. *Algorithm 5 is circuit fault-invariant with respect to fault set $\mathcal{F}^+$.*

Proof. An adversary can inject a fault either in line 5.2 or line 5.5. A fault injected in line 5.2 corresponds to a fault injected to the internal randomness of the gadget. Since an addition is an element-wise bijective function and randomness gates are considered as inputs of our graph, this satisfies fault-invariance. By commutativity of addition, any fault injected to line 5.5 can be linearly pushed to the output. This finishes the proof.

Lemma 23. *Algorithm 5 is $(e, 0, 0)$ -region fault robust with respect to $\mathcal{F}^+$ and $N = 1$ regions.*

Proof. Since Algorithm 5 does not take any inputs, by definition of region fault robustness, we aim to show the following: For any fault attack $T \in \mathcal{A}(\mathcal{F}^+)$ on internal and output wires with $|T| \leq e$, there exists a $(s, 0)$ -fault vector $\mathbf{v}$, whose distribution only depends on the internal randomness of the gadget, such that it holds with probability 1 that

$$\mathbf{ZEnc}^R() + \mathbf{v} \equiv T[\mathbf{ZEnc}^R]()$$

where $s \leq |T|$ and the output distributions are induced by the randomness R .

Faults in Line 5.2. Let ξ be a fault injected by the adversary, then the faulted element $r_i, i \in [1, d]$ equals to $r_i + \xi$. Since the elements $r_j, j \in [1, d]$ are chosen uniformly at random and independent of each other, and ξ is a wire-independent fault, $r_i + \xi$ is also uniformly distributed and remains independent of $r_j, j \neq i$. Thus, this fault is ineffective, independent of the secret zero, and does not change the distribution of the output.

Faults in Line 5.5. Let ξ_i^j be a fault injected to $Z_i^{(j)}$, i.e. the j -th iteration on Z_i , in Line 5.5. Then, by commutativity of addition, we can write any fault injected until iteration j_0 as the unfaulted value $Z_i^{(j_0)}$ plus the sum $\sum_{j \leq j_0} \xi_i^j$, where $\xi_i^j = 0$ in case no fault was injected. Thus, for each $i \in [n]$, we obtain the faulted output $G_i + \sum_{j \in [1, d]} \xi_i^j$. Setting $\sum_{j \in [1, d]} \xi_i^j = \mathbf{v}_i$ (where $\xi_{i'}^{j'} = 0$ in case no fault was injected in iteration $i' \in [n], j' \in [1, d]$) we can write any fault injected in line 5.5 as the unfaulted value Z_i plus a fault $\mathbf{v}_i$.

Conclusion. Overall we obtain

$$(Z_i)_{i \in [n]} + \mathbf{v} \equiv T[\mathbf{ZEnc}^R](\cdot)$$

where $(Z_i)_{i \in [n]} \leftarrow \mathbf{ZEnc}^R(\cdot)$. Since we only consider linear faults, and $|T| \leq e$, we obtain $\text{weight}(\mathbf{v}) \leq |T|$. Hence, $\mathbf{v}$ is an e -deterministic fault vector, and thus an $(e, 0)$ -fault vector by definition. Since the above calculations hold with probability $1 - \delta = 1$ and the output distributions are induced by the randomness R , this finishes the proof.

C Background on Graph Theory

Since we model a circuit C as a directed acyclic graph, where wires are modeled as edges and gates are modeled as vertices, we recall some necessary definitions.

Definition 17 (directed graph). A directed graph G is a finite, nonempty set $V = V(G)$, along with a set $E = E(G) \subseteq V \times V$, the edge set.

In order to model our circuits as graphs, we define paths, predecessors and a cycle for a directed graph as follows.

Definition 18. Let $G = (V, E)$ be a directed graph with ℓ vertices. A sequence of edges $E^{(P)} := \{e_0, \dots, e_{k-1}\} \subseteq E$ with $e_i = (u_i, v_i), i \in [k]$ is called a walk in the graph G if there exists a corresponding sequence of vertices $V^{(P)} = \{v_0^{(P)}, \dots, v_{k-1}^{(P)}\} \subseteq V$ such that $v_i = u_{i+1}$ for all $i \in [k]$. We denote $P := (V^{(P)}, E^{(P)})$. Furthermore, $E^{(P)}$ is called a path from v_0 to v_{k-1} , if no vertex appears twice in $E^{(P)}$. We will also sometimes interpret walks and paths as a sequence of vertices together with the actual walk (path) $E^{(P)}$ interchangeably, and thus denote a walk (path) as $P = (V^{(P)}, E^{(P)})$.

The set of all paths from u to v is denoted as $P_{u \rightarrow v}$. Likewise, $P_{U \rightarrow v}$ for a set U denotes all paths from all $u \in U$ to v .

Let $u, v \in V$. We call u a predecessor of v , iff there exists a path from u to v .

A graph (V, E) contains a cycle if there exists a vertex $v \in V$ such that there exists a walk P from v to v . If a graph does not contain any cycle, it is called acyclic.

The in-degree $\deg_{in}(v)$ of v is the number of edges going into v , i.e., $\deg_{in}(v) := |\{u \mid (u, v) \in E\}|$. Similarly, the out-degree $\deg_{out}(u)$ is the number of outgoing edges, i.e., $\deg_{out}(u) := |\{v \mid (u, v) \in E\}|$.

A vertex v is an input, if $\deg_{in}(v) = 0$, and an output, if $\deg_{out}(v) = 0$. The set of all input vertices is denoted as V^{in} and the set of output vertices as V^{out} .

Moreover, since we consider directed acyclic graphs, we can define a topological sorting as follows.

Definition 19. A topological sort of a directed acyclic graph $G = (V, E)$ is a linear ordering $\preceq$ of V such that if $(u, v) \in E$ then $u \preceq v$.

Two vertices $u, v \in V$ are incomparable if there are two topological sorts $\preceq_1$ and $\preceq_2$ such that $u \preceq_1 v$ and $v \preceq_2 u$, i.e., if $P_{u \rightarrow v} = P_{v \rightarrow u} = (\emptyset, \emptyset)$.

Note that the sorting is not unique as the graph does not need to be connected and nodes on the same layer are interchangeable.

Using these definitions, we can model our arithmetic circuits as directed acyclic graphs in a straightforward way: Every addition gate and every multiplication gate corresponds to a vertex v with in-degree 2 (for the operands). Every randomness gate is a special input vertex with in-degree 0. Now, if G is such a DAG, a given set of values $\lambda(v) \in \mathbb{F}$ for all input vertices $v \in V^{\text{in}}$ gives a complete set of values $\lambda(u)$ for all vertices $u \in V$ in the graph in a straightforward manner. For every wire $e = (u, v) \in E$, we also associate the label $\lambda(e) := \lambda(u)$ with it. The set of all such labels that correspond to output vertices V^{out} of the graph G is denoted by $G(\lambda)$, where $\lambda: V^{\text{in}} \rightarrow \mathbb{F}$. To distinguish between labels for normal inputs $\mathbf{x}$ and values $\mathbf{R}$ for the randomness gates, we also write $G(\mathbf{x}, \mathbf{R})$.

Modeling Circuits as Graphs. We consider a small circuit, only consisting of a share-wise addition. We can model this as depicted in Figure 1. Namely, we consider gates as edges. For our toy example with $n = 1$, it follows that we consider two input edges, l_1, l_2 , one addition $\oplus$, and one output gate, O . We obtain

$$G = (V, E) = (\{l_1, l_2, \oplus, O\}, \{(l_1, \oplus), (l_2, \oplus), (\oplus, O)\}).$$

Now, defining an input F and randomness R , we can *evaluate* our graph on these values by following the edges and performing the calculations depicted on the vertices. For simplicity, we will write $G(R, F)$ in this case, assuming inputs ordered according to a topological order.

Using this model, we can also consider circuits that are faulted, i.e., $T[C^R(F)]$. To do so, we additionally need to model the faults. We represent a fault by a fault-function $\xi_e \in \mathcal{F} \cup \{\text{id}\}$, $e \in E$, attached to the edge $e \in |E|$ the fault occurs at. As an example, let us consider Figure 1. A fault ξ injected for example to the input l_1 that is used during the addition, we set $\xi_{(l_1, \oplus)} = \xi$. Note that $\{\text{id}\}$ denotes that no fault was injected, in case this is clear from the context, we do not write this explicitly, i.e., in this case the corresponding edge does not contain a weight. This results in the graph depicted on the right-hand side in Figure 1. We obtain $T[C^R(F)] = (V_1, E_1)$ where $V_1 = \{l_1, l_2, \oplus, O\}$ and $E_1 = \{((l_1, \oplus), \xi_{(l_1, \oplus)}), ((l_2, \oplus), \xi_{(l_2, \oplus)}), ((\oplus, O), \xi_{(\oplus, O)})\}$. We can see E_1 as E together with some *weights* on the edges.

Similar to before, we use $T[G(R, F)]$ to denote the evaluation of R, F with respect to the faulted gadget $T[G]$.

Since we consider wire-probes, a probe p is denoted as a wire (edge) and reveals the value transported on this wire, i.e., for a gadget $G = (V, E)$, a probe $p = (v, v') \in E$ reveals the element output by the gate (vertex) v .

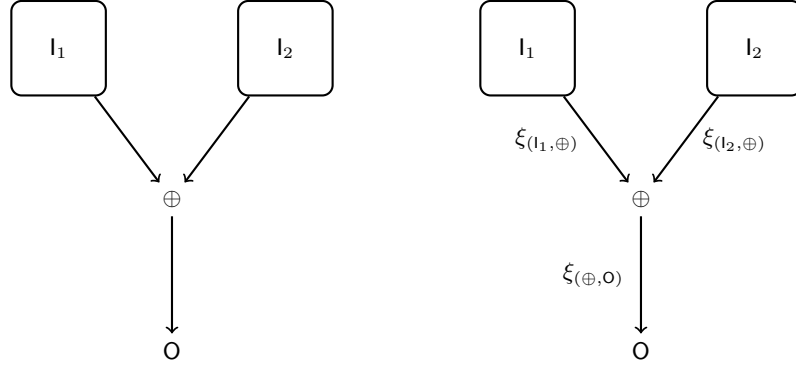


Fig. 1. Circuit C with $n = 1$ consisting of an addition on the left hand side is modeled as graph $(V, E) = (\{l_1, l_2, \oplus, O\}, \{(l_1, \oplus), (l_2, \oplus), (\oplus, O)\})$. On the right hand side, we draw the (possibly faulty) circuits instantiation, $T[C^R(F)]$, with $T[C^R(F)] = (V_1, E_1)$ and $V_1 = \{l_1, l_2, \oplus, O\}$ and $E_1 = \{((l_1, \oplus), \xi_{(l_1, \oplus)}), ((l_2, \oplus), \xi_{(l_2, \oplus)}), ((\oplus, O), \xi_{(\oplus, O)})\}$.

D Example: How to Simulate a Faulted Circuit

In this section, we consider a toy circuit example and use it to show two main intuitions concerning fault invariance. First, we show why the proof of Berndt et al. [12] fails on this specific circuit. Then, we use it to illustrate our circuit partition intuition for the proofs of Lemma 9 and Corollary 1.

Circuit. We consider a circuit with three inputs l_0, l_1, l_2 and two outputs O_0, O_1 , which is shown in Figure 2.

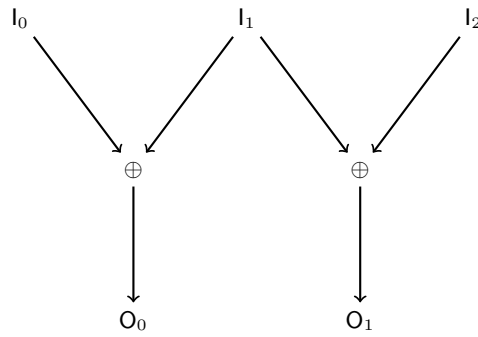


Fig. 2. Instantiation of circuit C consisting of two additions, three inputs l_0, l_1, l_2 and, two outputs O_0, O_1 .

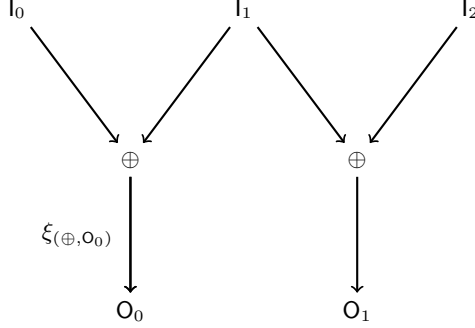


Fig. 3. Instantiation of faulty circuit $T[C]$ consisting of two additions, three inputs l_0, l_1, l_2 and, two outputs O_0, O_1 .

Faulted Circuit. Suppose the edge between the first addition and O_0 is faulted with additive fault $\xi_{(\oplus, O_0)}$, as shown in Figure 3. Notably, all identity functions on the unfaulted wires are omitted for readability.

Simulation Issue from [12]. Without loss of generality, to simulate the probes p_0, p_1 on edges coming out of the outputs O_0, O_1 of the faulty circuits, the fault $\xi_{(\oplus, O_0)}$ can be pushed into inputs l_0 or l_1 , i.e., simulator S_0 computes $p_0 = (l_0 + \xi_{(\oplus, O_0)}) + (l_1)$ or $p_0 = (l_0) + (l_1 + \xi_{(\oplus, O_0)})$. Besides, simulator S_1 can simulate probe p_1 by computing $p_1 = l_1 + l_2$. However, the issue arises when the input l_1 is the randomness and the fault is pushed into input l_1 . The reason is that probe p_0 can be simulated as $p_0 = l_0 + (R_0 + \xi_{(\oplus, O_0)})$ and probe p_1 can be simulated $p_1 = R_1 + l_2$, where R_0, R_1 are uniformly random values. But if $R_0 + \xi_{(\oplus, O_0)} \neq R_1$, then we cannot argue that there exists a composed simulator S consisting of S_0 and S_1 that can jointly simulate both probes, p_0, p_1 , because the value at node l_1 used for each simulation by S_0 and S_1 are different. Besides, solely fixing the randomness, i.e. setting $l_1 = R_0$ for R_0 being chosen uniformly at random, does not fix this issue, as in this case $R_0 + \xi_{(\oplus, O_0)} \neq R_0$ as long as $\xi_{(\oplus, O_0)} \neq 0$.

Simulation using the ideas from Section 4. To fix the above inconsistent simulation issues caused by node l_1 , Section 4 proposed a graph-based method to construct a composed simulator for both probes p_0, p_1 .

In the first step, we construct the probe-subgraphs for each probe. For probe p_0 , we construct probe-subgraph G^{p_0} as shown in Figure 4. Similarly, we construct probe-subgraph G^{p_1} shown in Figure 5.

In the second step, we construct the common-subgraph $\tilde{G}_0$ for both probe-subgraphs G^{p_0}, G^{p_1} shown in Figure 6. This is done by attaching a probe index set to each node as shown in Figure 6. Then, the index sets are ordered (using the ordering of the graph). We begin the index set containing all input nodes and set them to be a common subgraph. Afterwards, following the ordering of the index sets, each node corresponding to the next probe index set, together

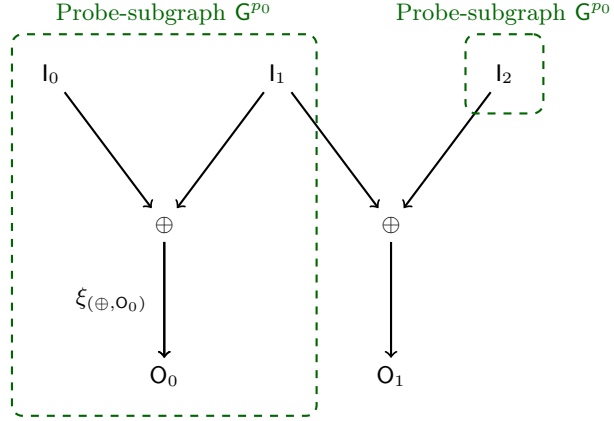


Fig. 4. Instantiation of faulty circuit $T[C]$ and probe-subgraph G^{p_0} .

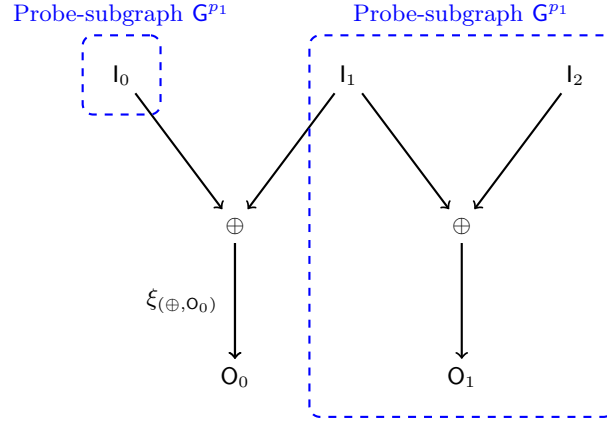


Fig. 5. Instantiation of faulty circuit $T[C]$ and probe-subgraph G^{p_1} .

with the previous common-subgraphs output, is defined to be the next common subgraph.

Now, we construct simulators S_0, S_1 for each probe p_0, p_1 based on the probe index set. For probe p_0 , the simulator S_0 uses the input value at node l_0, l_1 and fault $\xi_{(\oplus, O_0)}$, which can also be written as the faulty output of the common-subgraph $\xi(\tilde{G}_0(l_0, l_1, I_2)) = (l_0, l_1 + \xi_{(\oplus, O_0)}, l_2)$. For probe p_1 , the simulator S_1 uses the input value at node l_1, l_2 , which can also be written as the non-faulted output of the common-subgraph $\tilde{G}_0(l_0, l_1, l_2) = (l_0, l_1, l_2)$. The key point here is that both simulators S_0, S_1 use the (faulty) output from common-subgraph $\tilde{G}_0(l_0, l_1, l_2)$. In this case, we can compose S_0, S_1 to build a composed simulator S_{S_0, S_1} for both probe p_0, p_1 and avoid the inconsistency issues from [12]. Generally, by constructing common-subgraph for each layer of the circuit, we can push the faults layer-wise to the inputs, which can be sharing or randomness.

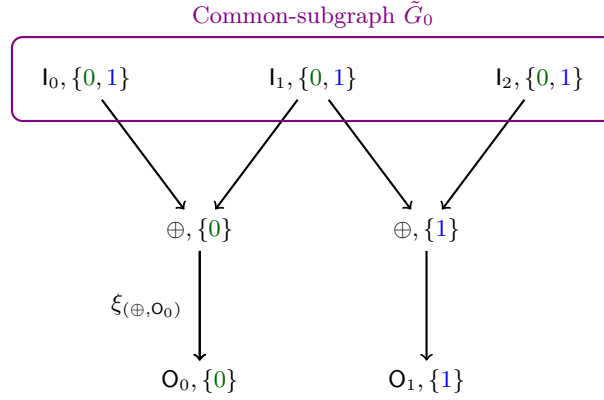


Fig. 6. Instantiation of faulted circuit $T[C]$ with respect to vertices in V' (containing both, vertices and index sets) and common-subgraph $\tilde{G}_0$. Notably, the index sets are defined as follows: $\{0, 1\} = \tilde{I}_0$, $\{0\} = \tilde{I}_1$ and $\{1\} = \tilde{I}_2$

E Proofs for Section 3

We begin by stating a Lemma that is needed for the proofs of Section 3.

Lemma 24. *Let $\mathcal{U}$ denote the uniform distribution. Let $\mathcal{D}_1, \mathcal{D}_2$ be two distributions such that $\Delta(\mathcal{D}_1, \mathcal{U}) \leq \epsilon$, $\Delta(\mathcal{D}_2, \mathcal{U}) \leq \epsilon$ and $\mathcal{D}_1, \mathcal{D}_2$ are independent of each other. Then $\Delta(\mathcal{D}_1 + \mathcal{D}_2, \mathcal{U}) \leq \epsilon$.*

Proof.

$$\begin{aligned}
\Delta(\mathcal{D}_1 + \mathcal{D}_2, \mathcal{U}) &= \frac{1}{2} \sum_x |\Pr(\mathcal{D}_1 + \mathcal{D}_2 = x) - \Pr(\mathcal{U} = x)| \\
&= \frac{1}{2} \sum_x \left| \sum_y \Pr(\mathcal{D}_1 + \mathcal{D}_2 = x, \mathcal{D}_1 = y) - \Pr(\mathcal{U} = x) \right| \\
&= \frac{1}{2} \sum_x \left| \sum_y \Pr(\mathcal{D}_2 = x - y, \mathcal{D}_1 = y) - \Pr(\mathcal{U} = x) \right| \\
&= \frac{1}{2} \sum_x \left| \sum_y (\Pr(\mathcal{D}_2 = x - y) \cdot \Pr(\mathcal{D}_1 = y)) - \Pr(\mathcal{U} = x) \right|
\end{aligned}$$

where the last equation follows by independence of the distributions. Since $\sum_y \Pr(\mathcal{D}_1 = y) = 1$, it follows that

$$\begin{aligned}
&\frac{1}{2} \sum_x \left| \sum_y (\Pr(\mathcal{D}_2 = x - y) \cdot \Pr(\mathcal{D}_1 = y)) - \Pr(\mathcal{U} = x) \right| \\
&= \frac{1}{2} \sum_x \left| \sum_y (\Pr(\mathcal{D}_1 = y) [\Pr(\mathcal{D}_2 = x - y) - \Pr(\mathcal{U} = x)]) \right|
\end{aligned}$$

Furthermore, by triangular inequality, we obtain

$$\begin{aligned}
&\frac{1}{2} \sum_x \left| \left(\sum_y \Pr(\mathcal{D}_1 = y) \right) [\Pr(\mathcal{D}_2 = x - y) - \Pr(\mathcal{U} = x)] \right| \\
&\leq \frac{1}{2} \sum_x \sum_y \Pr(\mathcal{D}_1 = y) |[\Pr(\mathcal{D}_2 = x - y) - \Pr(\mathcal{U} = x)]|
\end{aligned}$$

Last but not least, we obtain

$$\begin{aligned}
&\frac{1}{2} \sum_x \sum_y \Pr(\mathcal{D}_1 = y) |[\Pr(\mathcal{D}_2 = x - y) - \Pr(\mathcal{U} = x)]| \\
&= \frac{1}{2} \sum_y \Pr(\mathcal{D}_1 = y) \sum_x |[\Pr(\mathcal{D}_2 = x - y) - \Pr(\mathcal{U} = x)]| \\
&= \frac{1}{2} \sum_y \Pr(\mathcal{D}_1 = y) 2\Delta(\mathcal{D}_2, \mathcal{U}) \\
&= \frac{1}{2} \cdot 1 \cdot 2\Delta(\mathcal{D}_2, \mathcal{U}) \\
&= \Delta(\mathcal{D}_2, \mathcal{U}) \\
&\leq \epsilon
\end{aligned}$$

Putting all equations together, we obtain $\Delta(\mathcal{D}_1 + \mathcal{D}_2, \mathcal{U}) \leq \epsilon$, which finishes the proof.

E.1 Proof Preservation of e -linear fault robustness, Lemma 2

Proof (Preservation of e -linear fault robustness, Lemma 2). Let $G = G_2 \circ G_1$ be the composition of two e -linear fault-robust gadgets with respect to $\mathcal{F}$, where the output of G_1 is an input to G_2 . We need to show that G is e -linear fault robust according to Definition 7. Let T be an attack on the internal and output wires of G such that $T \in \mathcal{A}(\mathcal{F})$ and $|T| \leq e$. Since $\mathcal{F} \subset \mathcal{F}^{\text{ind}}$, we can split this attack into T_1 and T_2 corresponding, respectively, to the internal and output faults on G_1 and G_2 . Hence, it holds that $T[G] = T_2[G_2] \circ T_1[G_1]$ and $|T| = |T_1| + |T_2|$. Clearly, $|T| \leq e$ implies that $|T_1|, |T_2| \leq e$.

Step 1: Faults inside G_1 . By Definition 7, for any valid inputs to G_1 , any injected fault vectors $\xi_0, \dots, \xi_{l-1}$ on its inputs, and any attack T_1 of size $|T_1| \leq e$ to the internal and output wires of G_1 , the output of G_1 takes the form

$$Y' + \omega + \omega'$$

where:

- i) $\text{weight}(\omega) \leq |T_1|$ (*internal linear part*)
- ii) ω' is equal to a linear combination of $\xi_0, \xi_1, \dots$ (*input linear part*)

Step 2: Faults inside G_2 . Now, G_2 takes the possibly faulty output of G_1 as one of its inputs. The injected fault vectors on any further inputs of G_2 are denoted by $\zeta_0, \dots, \zeta_{l'-1}$. By e -linear fault robustness of G_2 , for any additional attack T_2 to the internal and output wires with size $|T_2| \leq e$, its output can be expressed as

$$Y + \eta + \eta'$$

where:

- i) $\text{weight}(\eta) \leq |T_2|$ (*internal linear part*)
- ii) η' is equal to a linear combination of $\omega + \omega', \zeta_0, \dots, \zeta_{l'-1}$ (*input linear part*)

Since η'_i is a linear combination of $\omega + \omega'$ and $\zeta_0, \dots, \zeta_{l'-1}$, we can further split this into $\hat{\eta}$ and $\hat{\eta}'$ such that $\eta' = \hat{\eta} + \hat{\eta}'$, $\hat{\eta}$ is a linear combination of ω , and $\hat{\eta}'$ is a linear combination of ω' and $\zeta_0, \dots, \zeta_{l'-1}$.

We know that $\text{weight}(\omega) \leq |T_1|$ by Step 1, and hence $\text{weight}(\hat{\eta}) \leq |T_1|$. Further, $\hat{\eta}'$ is a linear combination of ω' and $\zeta_0, \dots, \zeta_{l'-1}$, where ω' is a linear combination of the ξ_i s. Consequently each $\hat{\eta}'$ can be described as a linear combination of ξ_i s and ζ_j s.

Step 3: Combining both steps. The final output of the composed gadget can be described as

$$Y + (\eta + \hat{\eta}) + \hat{\eta}'$$

As $\hat{\eta}'_i$ is a linear combination of ξ_i 's and ζ_j 's, it only remains to prove that every $\theta = (\eta + \hat{\eta})$ satisfies $\text{weight}(\theta) \leq |T|$. This follows the internal parts of G_1 and G_2 , which respectively ensure that the inequalities $\text{weight}(\eta) \leq |T_2|$ and $\text{weight}(\hat{\eta}) \leq \text{weight}(\omega) \leq |T_1|$ hold, and from the identity $|T| = |T_1| + |T_2|$.

E.2 Proof that linear fault robust gadgets are threshold fault robust, Lemma 3

Proof (linear fault robust gadgets are threshold fault robust, Lemma 3). Let l be the (arbitrary) number of input sharings of G , and let $(F_0, F_1, \dots, F_{l-1})$ be any tuple of valid input sharings such that

$$Y \leftarrow G(F_0, F_1, \dots, F_{l-1}).$$

Further, let $T \in \mathcal{A}(\mathcal{F})$ be any fault attack with at most $|T| \leq e$ faults on the internal and output wires of the composed gadget G and $\xi_0, \xi_1, \dots, \xi_{l-1}$ be any tuple of joint $((e_i)_{i \in [l]}, \epsilon)$ -fault vectors such that

$$(Y + v) \equiv T[G^R]((F_i + \xi_i)_{i \in [l]}).$$

Step 1: Using linear fault robustness. Since G is an e -linear fault robust gadget with respect to $\mathcal{F}$, we can rewrite $v = \omega + \omega'$ such that

- i) $\text{weight}(\omega) \leq |T|$
- ii) ω' is equal to a linear combination of $\xi_0, \dots, \xi_{l-1}$.

Moreover, as $\xi_0, \dots, \xi_{l-1}$ are assumed to be joint $((e_i)_{i \in [l]}, \epsilon)$ -fault vectors, we can decompose each $\xi_i = \xi'_i + \xi''_i$, such that the ξ'_i are joint $(e_i)_{i \in [l]}$ -deterministic fault vectors and each ξ''_i is either zero or an encoding $\bar{\xi}_i$ such that the coefficients with indices $d+1 \dots, n-1$ of the underlying polynomial are ϵ -indistinguishable from uniform and independent of the other vectors.

Thus, we split the linear combination ω' of $\xi_0, \dots, \xi_{l-1}$ into $\omega' = \zeta' + \zeta''$, where ζ' denotes the part of the linear combination containing the ξ'_i and ζ'' denotes the linear combination of ξ''_i .

Step 2: Combining. For $v' := \omega + \zeta'$ by triangular inequality and definition of joint deterministic fault vectors, it holds true that $\text{weight}(v') = \text{weight}(\omega + \zeta') \leq \text{weight}(\omega) + \text{weight}(\zeta') \leq |T| + \sum_{i \in [l]} e_i$. Finally, we set $v'' := \zeta''$. Recall that ζ'' is a linear combination of the ξ''_i , which are either zero or an encoding $\bar{\xi}_i$ such that the coefficients with indices $d+1 \dots, n-1$ of the underlying polynomial are ϵ -indistinguishable from uniform only dependent on the distribution of the high order coefficients of the input faults $\mathcal{D}_{\xi_i}$. Importantly, the $\bar{\xi}_i$ are ϵ -indistinguishable from uniform and independent of each other. By Lemma 24, the high-order coefficients of the linear-combination ζ'' are ϵ -indistinguishable from uniform. Moreover, it only depends on the distribution the high order coefficients of the input faults $\mathcal{D}_{\xi_i}$.

Since $v = v' + v''$, this finishes the proof.

E.3 Proof of Composition of Threshold Fault Robustness, Lemma 4

Proof (Composition of Threshold Fault Robustness, Lemma 4). For all $i \in [k]$, fix any valid inputs $(F_i^j)_{j \in [l_i]}$ and any attack vector T with $|T| \leq e$ that can fault internal and output wires in the composition G . Since $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$, we can

decompose T into disjoint sub-attacks $T = T' \cup \bigcup_{i \in [k]} T_i$, where T' acts inside G' and T_i acts inside G_i .

According to the definition of (e, δ, ϵ) -threshold fault robustness, we need to show for

$$\mathbf{Y} + \boldsymbol{\omega} \leftarrow T[G^R] \left(\mathbf{F}_0^0 + \boldsymbol{\xi}_0^0, \dots, \mathbf{F}_0^{l_0} + \boldsymbol{\xi}_0^{l_0}, \mathbf{F}_1^0 + \boldsymbol{\xi}_1^0, \dots \right)$$

and joint $\left((e_i^j)_{j \in [l_i], i \in [k]}, \epsilon \right)$ -fault vectors $(\boldsymbol{\xi}_i^j)_{j \in [l_i], i \in [k]}$ that unless with probability $\delta' = 1 - (1 - \delta)^{k+1}$, $\boldsymbol{\omega}$ is a (s, ϵ) -fault vector whose distribution $\mathcal{D}_{\boldsymbol{\omega}}$ only depends on $\mathcal{D}_{\boldsymbol{\xi}_i^j}$ and the internal randomness of the gadget, and with $s \leq |T| + \sum_{i \in [k]} \sum_{j \in [l_i]} e_i^j$.

Preprocessing Stage: Since the $(\boldsymbol{\xi}_i^j)_{j \in [l_i]}$ are joint fault vectors for every $i \in [k]$, and since it holds true that $|T_i| + \sum_{j \in [l_i]} e_i^j \leq e$, we can apply the (e, δ, ϵ) -threshold fault robustness of G_i for each $i \in [k]$. Thus, for all $i \in [k]$, with probability $1 - \delta$ the output of G_i with attack vector T_i can be written as

$$\mathbf{Y}_i' + \boldsymbol{\xi}_i' \leftarrow T_i[G_i^{R_i}] \left((\mathbf{F}_i^j + \boldsymbol{\xi}_i^j)_{j \in [l_i]} \right)$$

where $\mathbf{Y}_i' \leftarrow G_i^{R_i} \left((\mathbf{F}_i^j)_{j \in [l_i]} \right)$ and $\boldsymbol{\xi}_i'$ is a (s_i, ϵ) -fault vector.

Notably, $(\boldsymbol{\xi}_i')_{i \in [k]}$ are joint fault vectors, because for each $i \in [k]$ the distributions $\mathcal{D}_{\boldsymbol{\xi}_i'}$ of the high order coefficient only depends on the distributions $\mathcal{D}_{\boldsymbol{\xi}_i^j}$, $j \in [l_i]$ and the internal randomness of the gadget G_i ; moreover, $(\boldsymbol{\xi}_i^j)_{j \in [l_i], i \in [k]}$ are joint (e, δ, ϵ) -fault vectors and each gadget uses independent randomness.

Core Stage: The k preprocessed outputs $\mathbf{Y}_i' + \boldsymbol{\xi}_i'$ are the inputs of G' . Since $|T'| + \sum_{i \in [k]} s_i \leq e$, the (e, δ, ϵ) -threshold fault robustness of G' under attack T' with $s \leq |T'|$ yields with probability $1 - \delta$ that

$$\mathbf{Y} + \boldsymbol{\omega} \leftarrow T'[G'^R] \left((\mathbf{Y}_i' + \boldsymbol{\xi}_i')_{i \in [k]} \right)$$

where $\boldsymbol{\omega}$ is a (s, ϵ) -fault vector whose high-order coefficient distribution $\mathcal{D}_{\boldsymbol{\omega}}$ only depends on $\mathcal{D}_{\boldsymbol{\xi}_0'}, \dots, \mathcal{D}_{\boldsymbol{\xi}_{k-1}'}$ and the internal randomness of gadget G' , where $s \leq e$.

Conclusion: Since $\boldsymbol{\omega}$ is an (s, ϵ) -fault vector whose distribution $\mathcal{D}_{\boldsymbol{\omega}}$ depends only on the distributions $\mathcal{D}_{\boldsymbol{\xi}_i^j}$, $i \in [k], j \in [l_i]$ and the internal randomness of the gadget, it is left to determine the probability δ' . If we combine the intermediate independent fail probabilities, i.e. calculate the probability that at least one output of an intermediate gadget fails, we obtain an overall failure probability of $\delta' = 1 - (1 - \delta)^{k+1}$. This concludes the proof.

E.4 Proof of Composition of Threshold and Region Fault Robustness, Lemma 6

Proof (Composition of threshold and region fault robustness, Lemma 6). Recall that we consider the composition $G = G'((G_i(\cdot))_{i \in [k]})$, where G' is an (e, δ, ϵ) -threshold fault robust gadget with respect to $\mathcal{F}$ and for all $i \in [k]$, it holds true that G_i is an (e, δ, ϵ) -region fault robust.

In the $N = 1$ setting, the single region is the whole composed circuit. Fix any valid inputs $\mathbf{F}_i^j, i \in [k], j \in [l_i]$ and any attack vector T into disjoint sub-attacks

$$T = T' \cup \bigcup_{i \in [k]} T_i$$

where T_i acts inside the preprocessor gadget G_i and T' acts inside the core gadget G' : hence $|T| = |T'| + \sum_{i \in [k]} |T_i|$.

According to the definition of $(\frac{\epsilon}{2}, \delta', \epsilon)$ -region fault robustness, we need to show for

$$\mathbf{Y} + \boldsymbol{\omega} \leftarrow T[G^R] \left((\mathbf{F}_i^0 + \boldsymbol{\xi}_i^0, \dots, \mathbf{F}_i^{l_i-1} + \boldsymbol{\xi}_i^{l_i-1})_{i \in [k]} \right)$$

and joint $(\frac{\epsilon}{2}, \epsilon)$ -fault vectors $(\boldsymbol{\xi}_i^j)_{i \in [k], j \in [l_i]}$ that with probability $\delta' = 1 - (1 - \delta)^{k+1}$ $\boldsymbol{\omega}$ is an $(\frac{\epsilon}{2}, \epsilon)$ -fault vector whose distribution $\mathcal{D}_{\boldsymbol{\omega}}$ is only dependent on the internal randomness of the gadget G .

Preprocessing Stage. Since for each $i \in [k]$ it holds that the $(\boldsymbol{\xi}_i^j)_{j \in [l_i]}$ are joint $(\frac{\epsilon}{2}, \epsilon)$ -fault vectors, and it holds that $|T_i| + \frac{\epsilon}{2} \leq e$, we can apply the (e, δ, ϵ) -region fault robustness of G_i for each $i \in [k]$: the output of G_i can be written with probability $1 - \delta$ as the valid (polynomial) sharing $\mathbf{Y}'_i$ plus an (s_i, ϵ) -fault vector $\boldsymbol{\xi}'_i$ with $s_i \leq |T_i|$

$$\mathbf{Y}'_i + \boldsymbol{\xi}'_i \leftarrow T_i[G_i^{R_i}] \left((\mathbf{F}_i^j + \boldsymbol{\xi}_i^j)_{j \in [l_i]} \right),$$

whose distribution $\mathcal{D}_{\boldsymbol{\xi}'_i}$ only depends on the internal randomness of gadget G_i . Notably, $(\boldsymbol{\xi}'_i)_{i \in [k]}$ are joint $((s_i)_{i \in [k]}, \epsilon)$ -fault vectors because $\mathcal{D}_{\boldsymbol{\xi}'_i}$ only depend on the internal randomness of the gadget G_i , which is independent from the randomness in $G_j, i \neq j$, by assumption.

Core Stage. By assumption, the k preprocessed outputs $\mathbf{Y}'_i + \boldsymbol{\xi}'_i$ are the inputs of G' . Since $|T'| + \sum_{i \in [k]} s_i \leq |T'| + \sum_{i \in [k]} |T_i| \leq \frac{\epsilon}{2}$, the (e, δ, ϵ) -threshold fault robustness of G' under attack T' results in

$$\mathbf{Y} + \boldsymbol{\omega} \leftarrow T'[G'^{R'}] (\mathbf{Y}'_0 + \boldsymbol{\xi}'_0, \dots)$$

with $s \leq |T'| + \sum_{i \in [k]} s_i \leq \frac{\epsilon}{2}$ such that it holds with probability $1 - \delta$ that $\boldsymbol{\omega}$ is a (s, ϵ) -fault vector, i.e. an $(\frac{\epsilon}{2}, \epsilon)$ -fault vector, whose distribution $\mathcal{D}_{\boldsymbol{\omega}}$ only depends on the internal randomness of G' and $\mathcal{D}_{\boldsymbol{\xi}'_i}, i \in [k]$.

Conclusion. Since for all $i \in [k]$ it holds that $\mathcal{D}_{\boldsymbol{\xi}'_i}$ only depends on the internal randomness of G_i and the randomness of each gadget is independent of the other gadgets, $\mathcal{D}_{\boldsymbol{\omega}}$ only depends on the internal randomness of the composed gadget G . Moreover, we can combine the intermediate independent failure probabilities, i.e. with probability $\delta' = 1 - (1 - \delta)^{k+1}$, we get precisely the requirement of $(\frac{\epsilon}{2}, \delta', \epsilon)$ -region fault robustness for $N = 1$: for any attack of size at most $\frac{\epsilon}{2}$ on the (single) region, the output fault vector is an $(\frac{\epsilon}{2}, \epsilon)$ -fault vector. This concludes the proof.

F Proofs for Section 4

Proof (Proof Sketch of Lemma 9). In a nutshell, the proof can be described as follows: We first *separate*⁶ the gadget G into probing subgraphs G^{p_i} for probes $p_0, \dots, p_{d'-1}$. Then, we build a set of components (or common subgraphs) for the probing subgraphs as follows: The components $\tilde{G}_i$ denote the overlapping parts of the different subgraphs, such that an intermediate calculation (a component) that is needed for probes p_1 and p_2 is calculated only once, and can be used to obtain both p_1 and p_2 , instead of needing to recompute this intermediate calculation (a component) twice, i.e., for p_1 and p_2 individually. Looking ahead, these components can be constructed by appending index sets $\tilde{I}_v$ to each vertex $v \in V$, where $v, \tilde{I}_v$ indicates that $v \in G^{p_\ell}$ for each $\ell \in \tilde{I}_v$. Then, each component corresponds to the vertices sharing an index set⁷. Thus, each of the subgraphs can be constructed out of these components, while the components do not share any wires (edges) or any intermediate gates (vertices). Hence, building the probe-graphs out of these components, we do not use any wire more than once in our calculations (the output gates of the components, however, can be used as often as needed).

Now, circuit fault-invariance comes into place. Each of these components $\tilde{G}_i$ can be written as an unfaulted component G_i . Moreover, due to (S)NI, each unfaulted component G_i is perfectly simulatable. Thus, we consider only the simulators of unfaulted components G_i *stacked on top of each other* where the functions ξ_i, y_i defined in the circuit fault-invariance act as *glue* in between, defining our desired wfr-(S)NI simulator S' . Since any component itself is only needed to be simulated once and can be reused afterwards, and we consider a simulator with arbitrary but fixed randomness $\tilde{R}$, our simulator S' stays consistent. Furthermore, by allowing the output vertices of the components to be used more than once, we are able to model the possibility of having one gate with several wires, which may have different fault functions.

For an illustrative explanation of the subgraphs, components and index sets, we refer to Supp. D.

Proof (Lemma 9). To prove d -fr(S)NI, we want to construct a simulator S' with output $\mathbf{p}$ such that for every $d' \leq d$ tuple of probes in the faulted circuit we have $\mathbf{p} = (p_0, \dots, p_{d'-1})$. W.l.o.g., we assume that these probes are ordered topologically, i.e., $p_i \preceq p_{i+1}$ for all $i \in [d' - 1]$. Let f' denote the way $T[G]$ computes the intermediate values $\lambda(\mathbf{p}) = (\lambda(p_0), \dots, \lambda(p_{d'-1}))$. We aim to show that

$$\Pr_{\mathbf{R} \leftarrow \mathbb{F}^r} [f'(\mathbf{R}, \mathbf{x}) = \lambda(\mathbf{p})] = \Pr_{\tilde{\mathbf{R}} \leftarrow \mathbb{F}^{\tilde{r}}} [S'(\tilde{\mathbf{R}}, \mathbf{a}) = \lambda(\mathbf{p})] \quad (2)$$

⁶ Here, “separate” means that after the separation operation, the resulting subgraphs are conceptually distinguished but may still share overlapping vertices.

⁷ For technical reasons we will need to add a few more vertices to be able to write each probe graph G^{p_i} as a composition of components. The details are deferred to the full proof (cf. paragraph *Constructing Common Subgraphs*).

where $\mathbf{x}$ denotes the circuits inputs and $\mathbf{R}$ denotes the $r \in \mathbb{N}$ random elements used by the circuit. Moreover, $\mathbf{a} \subset \mathbf{x}$ denotes the inputs of the simulator, where $\mathbf{a}$ contains at most d' elements of each input sharing. $\tilde{\mathbf{R}}$ denotes the $\tilde{r} \in \mathbb{N}$ random elements used by the simulator. We can write the left-hand side by making the components explicit using the definition of probing subgraph $\mathbf{G}^{p_i}$:

$$\Pr_{\mathbf{R} \leftarrow \mathbb{F}^r} [f'(\mathbf{R}, \mathbf{x}) = \lambda(\mathbf{p})] = \Pr_{\mathbf{R} \leftarrow \mathbb{F}^r} [T[\mathbf{G}^{p_0}(\mathbf{R}, \mathbf{x})]_0 = \lambda(p_0), \dots, T[\mathbf{G}^{p_{d'-1}}(\mathbf{R}, \mathbf{x})]_0 = \lambda(p_{d'-1})]. \quad (3)$$

Now, we separate the graphs into subgraphs, such that each component of the graph is only computed once. We can do so by using a notion similar to maximal common subgraphs as follows.

Constructing Common Subgraphs. In this paragraph we build a set of components (or common subgraphs) for the probing subgraphs, such that each intermediate calculation that is needed for more than one probe is calculated exactly once and can be reused for all probes using this component.

We consider our graph $T[\mathbf{G}] = (V, E)$ as well as the probing subgraphs $T[\mathbf{G}^{p_i}] = (V^{p_i}, E^{p_i}), i \in [d']$. Let us initialize $\mathbf{G}' = (V', E') := T[\mathbf{G}]$, which we will then transform by extending all gates (vertices) to also include index sets. For each gate $v \in V$, we consider the index set I_v , initialized as an empty set. For all $i \in [d']$, if $v \in V^{p_i}$, then we add the index i to the index set corresponding to vertex v , i.e., $I_v \leftarrow I_v \cup \{i\}$. Then, we add the index sets to the corresponding vertices, i.e., $V' \leftarrow \{(v, I_v) | v \in V\}$.

Next, we consider subgraphs with respect to the different index sets. Without loss of generality, we consider $\tilde{I} = \bigcup_{v \in V} \{I_v\}$ as the set of index sets and sort its elements according to their size, i.e., $\forall \tilde{I}_i, \tilde{I}_j \in \tilde{I} = \{\tilde{I}_0, \dots, \tilde{I}_{|\tilde{I}|}\}$ it holds that if $i < j$ then $|\tilde{I}_i| \geq |\tilde{I}_j|$.

Claim. For the index sets $\tilde{I}_i, \tilde{I}_j$, the following holds true: If $|\tilde{I}_i| > |\tilde{I}_j|$, then for all elements $v^{(i)}, v^{(j)}$ with $(v^{(i)}, \tilde{I}_i) \in V'$ and $(v^{(j)}, \tilde{I}_j) \in V'$, it holds that they are either incomparable or there exists a path $P_{v^{(i)} \rightarrow v^{(j)}} \neq (\emptyset, \emptyset)$.

Proof. Assume by contradiction that the claim is wrong, i.e., there exist $(v^{(i)}, \tilde{I}_i) \in V'$ and $(v^{(j)}, \tilde{I}_j) \in V'$ such that $|\tilde{I}_i| > |\tilde{I}_j|$ but they are neither incomparable nor there exists a path from $v^{(i)}$ to $v^{(j)}$. This implies that there exists an opposite path, i.e. $P_{v^{(j)} \rightarrow v^{(i)}} \neq (\emptyset, \emptyset)$. Since $|\tilde{I}_i| > |\tilde{I}_j|$, there exists an index ℓ such that $\ell \in \tilde{I}_i$ and $\ell \notin \tilde{I}_j$, i.e., $v^{(i)} \in V^{p_\ell}$ but $v^{(j)} \notin V^{p_\ell}$. Additionally, we know that there exists a path between $v^{(j)}$ and $v^{(i)}$. However, if edge p_ℓ is reachable from $v^{(i)}$, then by first following the path from $v^{(j)}$ to $v^{(i)}$, edge p_ℓ must also be reachable from $v^{(j)}$. As this would imply that $v^{(j)} \in V^{p_\ell}$, the contradiction follows.

We now focus on constructing our graphs as follows. First, let the subgraph $\tilde{\mathbf{G}}_0 = (\tilde{V}_0, \tilde{E}_0)$ with $\tilde{V}_0 = \{v | (v, \tilde{I}_0) \in V'\}$ and $\tilde{E}_0 = \{(v, \dot{v}) \in E' | v, \dot{v} \in \tilde{V}_0\}$. Moreover, we define its output nodes to be the set $\tilde{V}_0^{\text{out}} = \{v \in \tilde{V}_0 : \exists (v, \dot{v}) \in E \setminus \tilde{E}_0\}$. W.l.o.g., we assume that the calculation $\tilde{\mathbf{G}}_0(\mathbf{R}, \mathbf{x})$ returns all values on the wires $(v, \dot{v}) \in E \setminus \tilde{E}_0$ for $v \in \tilde{V}_0$ with respect to inputs $\mathbf{R}, \mathbf{x}$.

For all $i \in [\tilde{I}] \setminus \{0\}$, we denote the set $\tilde{V}_i^* = \{v | (v, \tilde{I}_i) \in V'\}$ and define a set of previously used vertices that might be needed as inputs later as $\tilde{V}_i^h = \{v | \exists (v, \tilde{v}) \in E' : v \in \tilde{V}_{i-1} \text{ and } \tilde{v} \notin \tilde{V}_{i-1}\}$. Then, we define $\tilde{G}_i = (\tilde{V}_i, \tilde{E}_i)$ for $\tilde{V}_i := \tilde{V}_i^* \cup \tilde{V}_i^h$ where the last set corresponds to adding the needed input vertices (gates) that are previous output vertices (gates), and $\tilde{E}_i := \{(v, \tilde{v}) \in E' | v, \tilde{v} \in \tilde{V}_i\}$. Notably, $\forall i \in [d'] : \bigcup_{j \text{ s.t. } i \in \tilde{I}_j} \tilde{G}_j = T[G^{p_i}]$ by construction of the index sets.

We can now rewrite the probing subgraphs with respect to the graphs $\tilde{G}_i, i \in [\tilde{I}]$: For all $i \in [d']$, there exists an order of the set $J_i = \{j | i \in \tilde{I}_j\}$ such that

$$\tilde{G}_{\ell_{|J_i|-1}^i} \left(\tilde{G}_{\ell_{|J_i|-2}^i} \left(\dots \left(\tilde{G}_{\ell_0^i}(\mathbf{R}, \mathbf{x}) \right) \right) \right) = T[G^{p_i}(\mathbf{R}, \mathbf{x})], \quad (4)$$

where $\ell_j^i \in [\tilde{I}], j \in [J_i]$ and the index of ℓ, i , is used to highlight the correspondence to probe p_i . Thus, each faulted probing subgraph $T[G^{p_i}(\mathbf{R}, \mathbf{x})]$ can be written in terms of the components $\tilde{G}_i$ as desired.

Using The Common Subgraphs. Using equation 4, we can rewrite the right-hand side of Equation 3 as follows

$$\Pr_{\mathbf{R} \leftarrow \mathbb{F}^r} [T[G^{p_0}(\mathbf{R}, \mathbf{x})]_0 = \lambda(p_0), \dots, T[G^{p_{d'-1}}(\mathbf{R}, \mathbf{x})]_0 = \lambda(p_{d'-1})] \quad (5)$$

$$= \Pr_{\mathbf{R} \leftarrow \mathbb{F}^r} [\tilde{G}_{\ell_{|J_0|-1}^0} \left(\dots \left(\tilde{G}_{\ell_0^0}(\mathbf{R}, \mathbf{x}) \right) \right) = \lambda(p_0), \dots, \tilde{G}_{\ell_{|J_{d'-1}|-1}^{d'-1}} \left(\dots \left(\tilde{G}_{\ell_0^{d'-1}}(\mathbf{R}, \mathbf{x}) \right) \right) = \lambda(p_{d'-1})]. \quad (6)$$

Before we begin to apply circuit fault-invariance on the subgraphs $\tilde{G}_{\ell_j^i}$ to simulate them, we highlight the following: By definition of the index sets, for all $i_0, i_1 \in [d'], j_0 \in [J_{i_0}], j_1 \in [J_{i_1}]$, one of the following holds true. By definition, each component $\tilde{G}_{\ell_j^i}$ corresponds to exactly one index set $\tilde{I}_{\ell_j^i}$. Recall that the index sets $\tilde{I}_k, k \in [\tilde{I}]$ are disjoint by construction. Now, each component $\tilde{G}_{\ell_j^i}$ contains the vertices corresponding to one index set $\tilde{I}_{\ell_j^i}$ together with previous output vertices. Thus, we can only obtain an overlapping of the graphs, if one of the following holds: either two components are identical, which might happen as one component can be used in different calculations, or two components share input/output vertices. The latter arises because, by construction of the components, we include previous output nodes to our structure, even if the corresponding vertices are not contained in the index set, which allows these elements to overlap. Formally, this yields the following cases:

- $\tilde{G}_{\ell_{j_0}^{i_0}} = \tilde{G}_{\ell_{j_1}^{i_1}},$
- $\tilde{G}_{\ell_{j_0}^{i_0}} \cap \tilde{G}_{\ell_{j_1}^{i_1}} = (\tilde{V}_{\ell_{j_0}^{i_0}}^{\text{out}}, \emptyset) = (\tilde{V}_{\ell_{j_1}^{i_1}}^{\text{in}}, \emptyset),$
- $\tilde{G}_{\ell_{j_0}^{i_0}} \cap \tilde{G}_{\ell_{j_1}^{i_1}} = (\tilde{V}_{\ell_{j_1}^{i_1}}^{\text{out}}, \emptyset) = (\tilde{V}_{\ell_{j_0}^{i_0}}^{\text{in}}, \emptyset),$
- $\tilde{G}_{\ell_{j_0}^{i_0}} \cap \tilde{G}_{\ell_{j_1}^{i_1}} = (\emptyset, \emptyset),$

$$- \tilde{G}_{\ell_{j_0}^{i_0}} \cap \tilde{G}_{\ell_{j_1}^{i_1}} = (\tilde{V}_{\ell_{j_0}^{i_0}}^{\text{in}} \cap \tilde{V}_{\ell_{j_1}^{i_1}}^{\text{in}}, \emptyset).$$

where $\tilde{V}_{\ell_{j_*}^{i_*}}^{\text{in}}$ denotes the set of input vertices as defined in Definition 18. Thus, either the graphs are the same or do not share an edge. Moreover, two components $\tilde{G}_i, \tilde{G}_j$ share at most their output and input vertices.

Construction of Simulator S' . We now have all the necessary insights to construct the simulator S' as follows. Let $N = \{p_0, \dots, p_{d'-1}\}$ denote the set of elements that need to be simulated, which is initialized as the probes. In other words, N contains all *tags* of values that need to be simulated. As the elements of N will always match exactly one intermediate edge (wire) each in $T[G]$, we assume that the information on which probe matches which wire in the graph is given inherently, meaning that w.l.o.g. we assume to know the location of each probe within our graph. Please note that the elements $(v_i, v'_i) = p_i, i \in [d']$ correspond to output edges (i.e., v_i denotes an output gate) of the graphs $G_{\ell_{j_i-1}^i}, i \in [d]$, respectively. Moreover, any inputs we need to compute for $\tilde{G}_{\ell_j^i}$ are outputs of the subgraph $\tilde{G}_{\ell_{j-1}^i}, 1 \leq j \leq |J_i| - 1$.

Recall that we consider the graphs $\tilde{G}_{\ell_j^i}$ in ascending topological order and denote them as $\tilde{G}_m, m \in [|\tilde{I}|]$ to only compute each evaluation of a graph once.

The simulation now follows a hybrid argument: Using a sequence of small steps, the subgraphs are replaced by simulators, which are then shown to be perfectly simulated. For this, in a first step we construct S' by *going down the stairs*, i.e. we consider the furthest component from the inputs, $\tilde{G}_{|\tilde{I}|-1}$ and go step by step to the input vertices in $\tilde{G}_0$. In more detail, we substitute the actual computation of $\tilde{G}_m$ by a simulator S_m , starting from $m = |\tilde{I}| - 1$ and ending at $m = 0$, where circuit fault-invariance acts as glue between the simulators $S_m, m \in [|\tilde{I}|]$. Doing so, we always include the inputs that a simulator needs to perfectly simulate the desired elements in the set N to keep track of the elements that are needed by the sub-simulators. In the second step, after substituting all actual circuits with simulators, we show that this actually simulates all of the necessary elements $p_0, \dots, p_{d'-1}$. Here, we start from $m = 0$ and end at $m = |\tilde{I}| - 1$ and call this step *going up the stairs and simulate*. The randomness needed by the simulators is denoted as $\tilde{\mathbf{R}}$ and initialized as $\emptyset$.

Step 1: Going down the stairs. We start from the base case $m = |\tilde{I}| - 1$, but keep the notation general, as the induction step will contain the same ideas. If there exists no element $p' \in N$ such that $p' \in \tilde{G}_m^{\text{out}}(\mathbf{b}^{(m)})$ is an output of the graph $\tilde{G}_m$ with respect to inputs $\mathbf{b}^{(m)}$, we continue with $m \rightarrow m - 1$ if $m > 0$ and stop otherwise. (Notably, this case does not happen for the base case $m = |\tilde{I}| - 1$.) Otherwise, we simulate all elements $p' \in N$ where $p' \in \tilde{G}_m^{\text{out}}(\mathbf{b}^{(m)})$ is an output of the graph $\tilde{G}_m$ with respect to inputs $\mathbf{b}^{(m)}$. By circuit fault-invariance of G , we can write $\tilde{G}_m(\mathbf{b}^{(m)})$ with respect to the unfaulted graph G_m corresponding to $\tilde{G}_m$ as

$$\tilde{G}_m(\mathbf{b}^{(m)}) = \xi_m \left(G_m \left(\mathbf{y}_m \left(\mathbf{b}^{(m)} \right) \right) \right)$$

where ξ_m denotes several fault functions from $\mathcal{F}^+$ and y_m denotes an element-wise bijective function whose coefficients are solely dependent on the gadget structure and the injected faults.

Let S denote a (S)NI simulator of the unfaulted graph G . We define a simulator S_m of $G_m \subseteq G$ as follows: S_m acts as S for all probes p_i in G_m , taking any vertices (or calculations) needed from $G \setminus G_m$, as well as its internal randomness $\tilde{R}$ as inputs. In other words, S_m denotes the restriction of the simulator S to G_m . We denote the randomness the simulator S_m needs additionally to $\tilde{R}$ as $\tilde{R}_m = \tilde{R}_m(\mathbf{R}, \mathbf{x})$, where $S_m(\tilde{R}_m(\mathbf{R}, \mathbf{x}), \mathbf{b}_a^{(m)}) = G_m(\mathbf{b}^{(m)})$ and consider these values as inputs. We show that the simulator for the faulted circuit does not use more inputs than the unfaulted circuit. We begin by considering how many inputs are needed to compute the (additional) randomness the simulator uses.

On the Randomness S_m needs. Notably, $\tilde{R}_m$ is considered as an input to the simulator S_m (and thus S , respectively). We set $\tilde{R} \leftarrow \tilde{R} \cup \tilde{R}_m$. Faults can be pushed to the random values, since they are considered as inputs. By definition of S_i as parts of the overall simulator S , the simulators S_i simulate subgraphs that do not overlap except for input and output wires. Thus the used randomness stays consistent within the simulators. Moreover, $\tilde{R}(\mathbf{R}, \mathbf{x})$ remains independent of the inputs $\mathbf{x}''$ of G that are not taken by the simulator for $\mathbf{R} \leftarrow_s \mathbb{F}^r$, as we can simulate the effect of the faults as follows: Let $\tilde{R}_m^{(i)}(\mathbf{R}, \mathbf{x}) \in \tilde{R}_m(\mathbf{R}, \mathbf{x})$ be one random value used for the simulation of $\tilde{G}_m$, and let $G^{\tilde{R}_m^{(i)}}$ and $\bar{G}^{\tilde{R}_m^{(i)}}$ denote the unfaulted probe-graph, and faulted probe-graph respectively, corresponding to the element the simulator simulates by calculating $\tilde{R}_m^{(i)}$. Applying circuit fault-invariance yields $\tilde{R}_m^{(i)}(\mathbf{R}, \mathbf{x}) = \left[\bar{G}^{\tilde{R}_m^{(i)}}(\mathbf{R}, \mathbf{x}) \right]_0 = \left[\xi_m^{(i)} \left(G^{\tilde{R}_m^{(i)}}(y_m^{(i)}(\mathbf{R}, \mathbf{x})) \right) \right]_0$ for some bijective functions $y_m^{(i)}$ and fault functions $\xi_m^{(i)}$. Moreover, G is (S)NI, thus for any inputs $\mathbf{R}, \tilde{\mathbf{x}}$, the element $\left[G^{\tilde{R}_m^{(i)}}(\mathbf{R}, \tilde{\mathbf{x}}) \right]_0$ is perfectly simulatable for $\mathbf{R} \leftarrow_s \mathbb{F}^r$. Furthermore, by assumption on the (S)NI simulator S_m , the element $\left[G^{\tilde{R}_m^{(i)}}(\mathbf{R}, \tilde{\mathbf{x}}) \right]_0$ (corresponding to the unfaulted graph) does not depend on the inputs $\tilde{\mathbf{x}}'' \subseteq \tilde{\mathbf{x}}$ the simulator does not hold for $\mathbf{R} \leftarrow_s \mathbb{F}^r$, i.e., the randomness used by the simulator of the unfaulted circuit only depends on the simulator's selection of inputs. Since $y_m^{(i)}$ is an input independent bijective function, and $\xi \in \mathcal{F}^+$ is a wire-independent additive fault function, also $\left[\xi_m^{(i)} \left(G^{\tilde{R}_m^{(i)}}(y_m^{(i)}(\mathbf{R}, \mathbf{x})) \right) \right]_0 = \tilde{R}_m^{(i)}(\mathbf{R}', \mathbf{x}')$ is independent of the inputs $\mathbf{x}'' \subseteq \mathbf{x}'$ the simulator does not hold, where $(\mathbf{R}', \mathbf{x}') = (y_m^{(i)}(\mathbf{R}, \mathbf{x}), \mathbf{R} \leftarrow_s \mathbb{F}^r)$. Moreover, if $\mathbf{R}$ is chosen independently and uniformly at random, due to bijectivity of $y(\cdot)$ so is $\mathbf{R}'$.

Hence, S_m can perfectly simulate the elements of G_m that are needed to simulate $p_0, \dots, p_{d'-1}$ with inputs that are calculated from at most t elements per input sharing of G . Importantly, the simulation of the functions ξ_m, y_m does not increase the number of needed input elements for each of the S_i , $i \in [|\tilde{I}|]$ and

hence not the overall number of needed input elements.

On the Inputs S_m Takes. Let $b_a^{(m)} \in \mathbf{b}^{(m)}$ denote the input elements the simulator S_m needs to simulate $p' \in \tilde{\mathbf{G}}_m^{\text{out}}(\mathbf{b}^{(m)}) \cap N$. Notably, by definition of S_m and $\tilde{\mathbf{R}}$, the simulators stay consistent within their use of randomness. We highlight that simulator S_m uses randomness $\tilde{\mathbf{R}}$ by writing $S_m^{\tilde{\mathbf{R}}}$. Moreover, since ξ_m and $\mathbf{y}_m$ are element-wise and their coefficients are only dependent on the internal structure of the gadget, as well as the fault functions, these functions are perfectly simulatable as well. It follows that

$$\Pr_{\mathbf{R}, \tilde{\mathbf{R}} \leftarrow \mathbb{F}^*} [\mathcal{Y}_{\ell_0^0}^{j_0-1} (\dots (\mathcal{Y}_{\ell_0^0}(\mathbf{R}, \mathbf{x})))] = \lambda(p_0), \dots \mid \mathcal{Y}_{\ell_j^i}(\cdot) = \left[\begin{array}{l} \xi_{m^*} \left(\mathbf{G}_{\ell_j^i}(\mathbf{y}_{m^*}(\cdot)) \right) \text{ if } \tilde{\mathbf{G}}_{\ell_j^i} = \tilde{\mathbf{G}}_{m^*} \text{ with } m^* \geq m+1 \\ \xi_{m^*} \left(S_{m^*}^{\tilde{\mathbf{R}}_{|\tilde{I}|-1}}(\mathbf{y}_{m^*}(\cdot)) \right) \text{ otherwise} \end{array} \right] \quad (7)$$

$$= \Pr_{\mathbf{R}, \tilde{\mathbf{R}} \leftarrow \mathbb{F}^*} [\mathcal{Y}_{\ell_0^0}^{j_0-1} (\dots (\mathcal{Y}_{\ell_0^0}(\mathbf{R}, \mathbf{x})))] = \lambda(p_0), \dots \mid \mathcal{Y}_{\ell_j^i}(\cdot) = \left[\begin{array}{l} \xi_{m^*} \left(\mathbf{G}_{\ell_j^i}(\mathbf{y}_{m^*}(\cdot)) \right) \text{ if } \tilde{\mathbf{G}}_{\ell_j^i} = \tilde{\mathbf{G}}_{m^*} \text{ with } m^* \geq m \\ \xi_{m^*} \left(S_{m^*}^{\tilde{\mathbf{R}}}(\mathbf{y}_{m^*}(\cdot)) \right) \text{ otherwise} \end{array} \right]. \quad (8)$$

For all input elements $b_a^{(m)} \in \mathbf{b}^{(m)}$ the simulator S_m needs to simulate $p' \in \tilde{\mathbf{G}}_m^{\text{out}}(\mathbf{b}^{(m)}) \cap N$, we set $N \leftarrow N \cup \{b_a^{(m)}\}$. Moreover, we set $\tilde{\mathbf{R}} \leftarrow \tilde{\mathbf{R}} \cup \tilde{\mathbf{R}}_m$.

This concludes the base case and the induction step is exactly the same as the base case. Thus, we can continue as follows: If $m > 0$, we now continue with $m \rightarrow m-1$. Otherwise, we stop.

Step 2: Going up the stairs and simulate. We begin with our base case $m = 0$.

While N denotes the tags of the probes, as well as the tags of inputs needed for the simulator S_i of component $\tilde{\mathbf{G}}_i, i \in [|\tilde{I}|]$, we will set $\mathcal{H}$ to contain the actual values of these probes and (intermediate) inputs of S_i . First, we initialize the set of simulated input elements as $\mathcal{H} = \emptyset$.

Note that $\tilde{\mathbf{G}}_0$ consists of at least all input vertices of $\mathbf{G}$, as by definition of the probe-graphs, every probe-graph contains all input vertices. Hence, one of the graphs built from the index sets consists of all input vertices as well. Due to topological ordering, this graph needs to be $\tilde{\mathbf{G}}_0$. Moreover, all remaining components $\mathbf{G}_i, i \in [|\tilde{I}|] \setminus \{0\}$ consider the outputs of one of the previous graphs $\tilde{\mathbf{G}}_j, j \in [|\tilde{I}|], j < i$ as inputs. Since the simulator $S_i, i \in [|\tilde{I}|]$ is constructed using the (S)NI simulator S , together with functions $\mathbf{y}_m, \xi_m$ that do not increase the number of needed inputs, S_0 takes at most t elements per input sharing. Since we aim to simulate d' probes in total, $\tilde{\mathbf{G}}_0$ needs at most d' inputs.

We denote the inputs $\tilde{\mathbf{G}}_0$ takes from $\mathbf{x}$ as $\mathbf{a} \subseteq \mathbf{x}$. Any needed randomness $\mathbf{R}$ can be provided without limitation on the number, as these elements are assumed to be uniformly distributed (cf. Step 1).

Now, we only need to consider outputs of $\tilde{G}_0$ that are needed as input for following computation, therefore we focus on $v \in V_0^{\text{out}} \cap N$. For any output node of $v \in V_0^{\text{out}} \cap N$, $\tilde{G}_0 = (\tilde{V}_0, \tilde{E}_0)$, we can simulate this element as follows: Recall that due to circuit fault-invariance of G , we can write $\tilde{G}_0(\mathbf{R}, \mathbf{x}) = \xi_0(G_0(\mathbf{y}_0(\mathbf{R}, \mathbf{x})))$, where any element $v \in V_0^{\text{out}} \cap N$ is perfectly simulatable by S_0 using at most d' elements per input sharing. We simulate the required elements set $\mathcal{H} \leftarrow \mathcal{H} \cup \{\lambda(v) : v \in V_0^{\text{out}} \cap N\}$.

We continue with case $m - 1 \rightarrow m$. We consider $\tilde{G}_m$ and aim to simulate all elements $\mathbf{v}^{(m)} \in \tilde{V}_m^{\text{out}} \cap N$. Due to the topological ordering of $G_i, i \in [|\tilde{I}|]$ and definition of N , any input that is needed to simulate $\mathbf{v}^{(m)} \in \tilde{V}_m^{\text{out}} \cap N$ via S_m has already been simulated and is part of the set $\mathcal{H}$. Since ξ_m and $\mathbf{y}_m$ are element-wise and their coefficients are only dependent on the internal structure of the gadget, as well as the fault functions, these functions can be perfectly simulated. Since $\mathbf{y}_m$ is an element-wise function, we can simulate its evaluation on the input element $\mathbf{b}_a^{(m)} \in \mathbf{b}^{(m)}$ the simulator S_m needs before we feed this input to S_m . Now, obtaining $\mathbf{v}^{(m)} \leftarrow S_m(\mathbf{b}_a^{(m)})$, we simulate the effect of the function ξ_m on the elements $\mathbf{v}^{(m)}$, abusing the notation, we denote this by $\xi_m(\mathbf{v}^{(m)})$ for simplicity. This simulation is perfect, as $\xi^{(m)} \in \mathcal{F}^+$ is assumed to be public. Now, we set $\mathcal{H} \leftarrow \mathcal{H} \cup \{\xi_m(\mathbf{v}^{(m)})\}$.

If $m \leq |\tilde{I}| - 1$, we continue with $m \rightarrow m + 1$. Otherwise, $|\mathcal{H}| = |N|$ and we stop. Now, by construction of $\mathcal{H}$ and N , the desired elements $p_0, \dots, p_{d'} \in \mathcal{H}$ are perfectly simulated using the input elements $\mathbf{a}$. This finishes the proof.

Proof (Corollary 1). We aim to show that $T[G]$ can be perfectly simulated with $d' + |T|$ shares of each input sharing for any fault attack $T \in \mathcal{A}(\mathcal{F}^{\text{ind}})$ with $|T| + d' \leq d$. Moreover, let $p_{d'}, \dots, p_{|T|+d'-1}$ denote the wires (edges) where faults $\tilde{\xi}_i \in \mathcal{F}^{\text{ind}}, i \in [d', |T| + d']$ are injected. Looking ahead, our reduction needs the faulted values as probes, which is why we include these values as probes here. By assumption, we know the fault vectors $\tilde{\xi}_i \in \mathcal{F}^{\text{ind}}, i \in [d', |T| + d']$. Computing $a_i := \tilde{\xi}_i(p_i) - p_i$, we can define the additive fault function $\xi'_i(x) := x + a_i$ for each $i \in [d', |T| + d']$.

Let $G_0, \dots, G_{\tilde{I}-1}$ be the subgraphs of G w.r.t. $p_0, \dots, p_{d'-1}, p_{d'}, \dots, p_{|T|+d'-1}$ constructed as in the paragraph Constructing Common Subgraphs of Lemma 9. Similar to the proof of Lemma 9, we can write

$$\tilde{G}_{\ell_0^0|_{J_0|-1}} \left(\dots \left(\tilde{G}_{\ell_0^0}(\mathbf{R}, \mathbf{x}) \right) \right) = \lambda(p_0), \dots, \tilde{G}_{\ell_{|J_{d'-1}|-1}^{d'-1}} \left(\dots \left(\tilde{G}_{\ell_0^{d'-1}}(\mathbf{R}, \mathbf{x}) \right) \right) = \lambda(p_{d'-1}).$$

We begin by proceeding analogously to Step 1 of Lemma 9, and only differ in Step 2 as follows. We begin by $m = 0$ and consider the probes $p_i = (v'_i, v_i)$: If for all $i \in [d', |T| + d'] \nexists v'_i \in \tilde{V}_m$, in other words, if no fault was injected into component $\tilde{G}_m$, then proceed as denoted in Step 2 of Lemma 9. Otherwise, simulate the unfaulted circuit G_m to obtain all values $\lambda(p_i), i \in [d', |T| + d']$ such that $v'_i \in V_m^{\text{out}} \cap N$. As described above, we can compute $a_i := \xi_i(p_i) - p_i$, to determine the additive fault function $\xi'_i(x) := x + a_i$ for each $i \in [d', |T| + d']$ such that $v'_i \in V_m^{\text{out}} \cap N$. Due to circuit fault-invariance with respect to additive faults,

we can push the injected faults to the inputs or outputs of G_m and perfectly simulate G_m as described in Step 2 of Lemma 9. If $m \leq |\tilde{I}| - 1$, continue with $m \rightarrow m + 1$. Otherwise, $|\mathcal{H}| = |N|$ and we stop.

G Proofs for Section 5

G.1 Proofs for Correctness and Error-Propagation of **P-Refresh**

*Proof (Correctness of **P-Refresh** (cf. Algorithm 1), Lemma 10).* The algorithm computes $S_j = \sum_{i=0}^{n-1} (Z_{i,j} + A_{j,i} \cdot F_i)$ for all $j \in [n]$. We have

$$\begin{aligned} S_j &= \sum_{i=0}^{n-1} (Z_{i,j} + A_{j,i} \cdot F_i) = \sum_{i=0}^{n-1} \left(Z_{i,j} + \left(\text{Id}_{j,i} + \sum_{k=d+1}^{n-1} R_j^{(k)} \cdot V_{k,i}^{-1} \right) \cdot F_i \right) \\ &= \sum_{i=0}^{n-1} \left(Z_{i,j} + \text{Id}_{j,i} \cdot F_i + \left(\sum_{k=d+1}^{n-1} R_j^{(k)} \cdot V_{k,i}^{-1} \right) \cdot F_i \right) \\ &= \sum_{i=0}^{n-1} (Z_{i,j} + \text{Id}_{j,i} \cdot F_i) + \left(\sum_{k=d+1}^{n-1} R_j^{(k)} \cdot \left(\sum_{i=0}^{n-1} V_{k,i}^{-1} \cdot F_i \right) \right) \\ &= \sum_{i=0}^{n-1} (Z_{i,j} + \text{Id}_{j,i} \cdot F_i) + \left(\sum_{k=d+1}^{n-1} R_j^{(k)} \cdot f_k \right). \end{aligned}$$

However, since $\mathbf{F}$ is a valid encoding, $f_k = 0 \ \forall k \in [d+1, n-1]$. Thus, it follows

$$\begin{aligned} S_j &= \sum_{i=0}^{n-1} (Z_{i,j} + \text{Id}_{j,i} \cdot F_i) + \left(\sum_{k=d+1}^{n-1} R_j^{(k)} \cdot f_k \right) = \\ &= \sum_{i=0}^{n-1} (Z_{i,j} + \text{Id}_{j,i} \cdot F_i) = \sum_{i=0}^{n-1} Z_{i,j} + F_j. \end{aligned}$$

Recall that $Z_{i,j}$ denotes the j -th share of the i -th zero-encoding of degree- d , and $(F_i)_{i \in [n]}$ is a valid encoding. Moreover, the sum of sharings with respect to degree- d polynomials results in a sharing of a polynomial of degree up to d . Thus, altogether, $s(X) = s_0 + \dots s_d \cdot X^d$ and $s_0 = f_0$.

*Proof (Error-Propagation of **P-Refresh** (cf. Algorithm 1), Lemma 11).* Due to its independence from the (input) shares $(F_i)_{i \in [n]}$, we know that $A = \text{Id} + \sum_{k=d+1}^{n-1} M^{(k)}$ where $M_{i,l}^{(k)} = \left(\sum_{j>d} r_j^{(k)} x_i^j \right) \cdot V_{k,l}^{-1}$. Following Algorithm 1, we

compute

$$\begin{aligned}
S_j &= \sum_{i=0}^{n-1} (Z_{i,j} + A_{j,i} \cdot F_i) \\
&= \sum_{i=0}^{n-1} \left(Z_{i,j} + \left(\text{Id}_{j,i} + \sum_{k=d+1}^{n-1} M_{j,i}^{(k)} \right) \cdot F_i \right) \\
&= \sum_{i=0}^{n-1} (Z_{i,j} + \text{Id}_{j,i} \cdot F_i) + \left(\sum_{k=d+1}^{n-1} \left(\sum_{l>d} r_l^{(k)} x_j^l \cdot f_k \right) \right) \\
&= \sum_{i=0}^{n-1} Z_{i,j} + F_j + \left(\sum_{k=d+1}^{n-1} \left(\sum_{l>d} r_l^{(k)} x_j^l \cdot f_k \right) \right).
\end{aligned}$$

Recall that $(Z_{i,j})_{j \in [n]}$ denotes a valid zero-encoding (Correctness of Algorithm ZENC). Thus, considering the coefficients of the polynomial corresponding to S_j , we see

$$s_i = \sum_{j=0}^{n-1} S_j \cdot \lambda_{i,j} \quad (9)$$

$$= \sum_{j=0}^{n-1} \left(\sum_{m=0}^{n-1} (Z_{m,j}) + F_j + \left(\sum_{k=d+1}^{n-1} \left(\sum_{l>d} r_l^{(k)} x_j^l \cdot f_k \right) \right) \right) \cdot \lambda_{i,j} \quad (10)$$

$$= \left(\sum_{m=0}^{n-1} \left(\sum_{j=0}^{n-1} Z_{m,j} \cdot \lambda_{i,j} \right) \right) + \left(\sum_{j=0}^{n-1} F_j \cdot \lambda_{i,j} \right) + \left(\sum_{k=d+1}^{n-1} \left(\sum_{j=0}^{n-1} \left(\sum_{l>d} r_l^{(k)} x_j^l \cdot f_k \right) \lambda_{i,j} \right) \right) \quad (11)$$

$$= \left(\sum_{m=0}^{n-1} h_{m,i} \right) + f_i + \left(\sum_{k=d+1}^{n-1} \left(\sum_{j=0}^{n-1} \left(\sum_{l>d} r_l^{(k)} x_j^l \cdot f_k \right) \lambda_{i,j} \right) \right), \quad (12)$$

where Equation 9 follows from polynomial interpolation. Moreover, $h_{m,i}$ denotes the i -th coefficient of the polynomial corresponding to shares $(H_{m,j})_{j \in [n]}$, $\forall m \in [n]$.

$$\begin{aligned}
s_i &= \left(\sum_{m=0}^{n-1} h_{m,i} \right) + f_i + \left(\sum_{k=d+1}^{n-1} \left(\sum_{l>d} \left(\sum_{j=0}^{n-1} r_l^{(k)} x_j^l \lambda_{i,j} \right) \right) \cdot f_k \right) \\
&= \left(\sum_{m=0}^{n-1} h_{m,i} \right) + f_i + \left(\sum_{k=d+1}^{n-1} \left(r_i^{(k)} \right) \cdot f_k \right),
\end{aligned}$$

where we set $r_i^{(k)} = 0 \forall i \in [d+1]$. The last equation follows by $(\lambda_{i,j})_{i,j \in [n]}$ being the inverse of $(x_i^j)_{i,j \in [n]}$, i.e., $\sum_{j=0}^{n-1} \lambda_{i,j} \cdot x_j^l = 0$ iff $i \neq l$ and $\sum_{j=0}^{n-1} \lambda_{i,j} \cdot x_j^l = 1$ iff $i = l$.

By assumption, $\exists i \in [d+1, n-1] : f_i \neq 0$. Since the elements $r_i^{(k)}, i \in [d+1, n-1], k \in [d+1, n-1]$ are independent of each other and chosen uniformly at random, the coefficients $s_i, i \in [d+1, n-1]$ are independent from each other and indistinguishable from uniform.

G.2 Proof P-Refresh is t -SNI, Lemma 13

Proof (P-Refresh (cf. Algorithm 1) is t -SNI, Lemma 13). In the following, we split the probes into several groups of variables. Afterwards, we define the set of input variables needed for the simulation. Then, we show how to perfectly simulate the different groups. We assume to consider ordered sets of elements (with respect to the indices), i.e., $\{a_{i-1}, a_i, a_{i+1}, \dots\}$. In case there are two variables $i, j \in [n]$ and we aim to consider $a_{i,j}$ ordered as $a_{i,j}, a_{i,j+1}, \dots, a_{i,n-1}, a_{i+1,0}, \dots$, we write $i \lesssim j$

- **Group 1** This group consists of probes in $r_j^{(k)}$ and $R_{i,j}^{(k)} := \sum_{\ell=d+1}^j r_\ell^{(k)} x_i^\ell$, where $k \lesssim j \lesssim i$ (i.e., probes in lines 5 and 7)
- **Group 2** In this group, we consider probes in $Z_{i,j}$ ($i \lesssim j, i, j \in [n]$) and probes during the computation of the i -th use of **ZEnc**.
- **Group 3** This group consists of probes in line 12, i.e., probes of the form $A_{j,i}^\ell = \text{Id}_{j,i} + \sum_{k=d+1}^\ell R_j^{(k)} \cdot V_{k,i}^{-1}$. Please note that probes of $A_{j,i}^{n-1}$ are written as probes in $A_{j,i}$
- **Group 4** In this group, we consider the probes during the calculation in line 13, and we denote with $Z_j^{(i)} := Z_{i,j} + A_{j,i} \cdot F_i$ the value computed in this iteration.
- **Group 5** This group consists of probes in line 14, namely $S_j \leftarrow S_j + Z_{i,j}$. We denote a probe of S_j in iteration i as $S_j^{(i)} = \sum_{\ell=0}^i Z_j^{(i)}$.

Let S denote the set of input indices we need. We initialize $S = \emptyset$. We introduce two sets, $G1$ and $G3$, denoting the set of elements from Group 1 and 3, respectively, that we use to simulate values from the other groups. Again, we initialize $G1 = \emptyset$ and $G3 = \emptyset$. We fill the simulation sets step by step.

- **Group 1:** For any probe in Group 1 that is of the form $R_{i,j}^{(k)}$, put $G1 \leftarrow G1 \cup \{R_{i,j}^{(k)}\}$
- **Group 2:** If there exists any probe during the i -th computation of **ZEnc**,

$$\begin{aligned} S &\leftarrow S \cup \{i\} \\ G1 &\leftarrow G1 \cup \{R_{j,n-1}^{(k)} | k \in [d+1, n], j \in [n]\} \\ G3 &\leftarrow G3 \cup \{A_{j,i} | j \in [n]\} \end{aligned}$$

- **Group 3:** For a probe in group 3, namely a probe during the calculation of $A_{j,i}^{(\ell)} \leftarrow A_{j,i} + \sum_{k=d+1}^\ell R_i^{(k)} V_{k,j}^{-1}$, insert

$$\begin{aligned} G1 &\leftarrow G1 \cup \{R_{i,n-1}^{(k)} | k \in [d+1, n]\} \\ G3 &\leftarrow G3 \cup \{A_{j,i}^{(\ell)}\} \end{aligned}$$

- **Group 4:** If $A_{i,j}$ is a probe in Group 4, we insert

$$\begin{aligned} S &\leftarrow S \cup \{i\} \\ G1 &\leftarrow G1 \cup \{R_{j,n-1}^{(k)} | k \in [d+1, n], j \in [n]\} \\ G3 &\leftarrow G3 \cup \{A_{j,i} | j \in [n]\} \end{aligned}$$

- **Group 5:** -

Note that for one internal probe, we put at most 1 element into the set of input indices, S .

Simulation: We now proceed with the simulation of the different groups. Please note that we assume that any elements that are simulated in the following are stored and can be reused.

- **Group 1:** For any probe of $r_j^{(k)}$, $k \in [d+1, n], j \in [d+1, n]$ choose $r_j^{(k)}$ uniformly at random from $\mathbb{F}$.
For any probe $R_{i,j}^{(k)} \in G1$, proceed as follows. Recall that $R_{i,j}^{(k)} = \sum_{\ell=0}^j r_\ell^{(k)} \cdot x_i^\ell$. If $r_\ell^{(k)}$ has already been sampled, use this element. Otherwise sample $r_\ell^{(k)}$ uniformly random from $\mathbb{F}$. Since x_i are assumed to be public and we perfectly simulate $r_\ell^{(k)}$, we can also perfectly simulate $R_{i,j}^{(k)}$.
- **Group 2:** For any probe in Group 2, run $(Z_{i,0}, \dots, Z_{i,n-1}) \leftarrow \mathbf{ZEnc}_{n,d}$ with independent randomness, and store the corresponding values. This is a perfect simulation, as each execution of **ZEnc** is input-independent.
- **Group 3:** For any probe in $G3$, we perfectly simulated all needed elements $R_i^{(k)}$ in Group 1, as for any probe during the calculation of $A_{j,i}$, we set $\{R_{j,n-1}^{(k)} | k \in [d+1, n]\} \subset G1$. Thus, we can perfectly simulate all probes in $G3$ by calculating $A_{j,i}^\ell = \text{Id}_{j,i} + \sum_{k=d+1}^\ell R_j^{(k)} \cdot V_{k,i}^{-1}$, for $\ell \in [d+1, n]$ as $V_{k,i}^{-1}$ is assumed to be publicly known for $k \in [n], j \in [n]$.
- **Group 4:** This group considers probes during the calculation of $Z_j^{(i)} = Z_{i,j} + A_{j,i} \cdot F_i$.
 - If $A_{j,i} \cdot F_i$ is probed, we can perfectly simulate this by computing $A_{j,i} = \text{Id}_{j,i} + \sum_{k=d+1}^{n-1} R_j^{(k)} \cdot V_{k,i}^{-1}$, where $R_j^{(k)} = R_{j,n-1}^{(k)}$ was simulated in Group 1, since $R_{j,n-1}^{(k)} \in G1, \forall k \in [d+1, n]$, and $V_{k,i}^{-1}$ are public parameters. This perfectly simulates $A_{j,i}$. Moreover, in this case $i \in S$, hence we can compute $A_{j,i} \cdot F_i$.
 - For probes of $Z_j^{(i)} = Z_{i,j} + A_{j,i} \cdot F_i$, we do the following. If $Z_{i,j}$ was already perfectly simulated due to probes in Group 2, we use this element. Otherwise, we choose $Z_{i,j}$ uniformly at random. This perfectly simulates $Z_{i,j}$, since **ZEnc** outputs t -wise uniformly independent sharings, and we consider at most t probes. Then, we calculate $Z_j^{(i)} = Z_{i,j} + A_{j,i} \cdot F_i$. This perfectly simulates the element $Z_j^{(i)}$, since $A_{j,i}$ and $Z_{i,j}$ are perfectly simulated and F_i is an input to the simulator.
- **Group 5:** We now simulate all values $S_j^{(\ell)} = \sum_{i=0}^\ell Z_j^{(i)}$ with $Z_j^{(i)} = (Z_{i,j} + A_{j,i} \cdot F_i)$, even if these elements were not probed. We show how to simulate these elements inductively with the help of the input set S and the previous probes.

$\ell = 0$: We can calculate $S_j^{(0)} = Z_{0,j} + A_{j,0} \cdot F_0$. If $Z_{0,j}$ is not calculated due to a probe in a previous group, choose $S_j^{(0)}$ uniformly at random. This perfectly simulates $S_j^{(0)}$ since **ZEnc** outputs a t -wise uniformly independent sharing, and we consider at most t probes.

Otherwise, $Z_{0,j}$ is already perfectly simulated due to previous probes. However, in this case $0 \in S$ (as $Z_{0,j}$ only occurs in Group 2 or Group 4 and in both cases, we put $0 \in S$). Moreover, $A_{j,0} \in G3$ and thus, $A_{j,0}$ is already perfectly simulated. Hence, we can perfectly simulate $S_j^{(0)} = Z_{0,j} + A_{j,0} \cdot F_0$.

$\ell \rightarrow \ell + 1$: $S_j^{(\ell+1)} = S_j^{(\ell)} + Z_{\ell+1,j} + A_{j,\ell+1} \cdot F_{\ell+1}$. By the induction hypothesis, we assume that we can perfectly simulate $S_j^{(\ell)}$. If $Z_{\ell+1,j}$ is not simulated yet, we choose $S_j^{(\ell+1)}$ uniformly at random. This perfectly simulates $S_j^{(\ell+1)}$, since $Z_{\ell+1,j}$ is indistinguishable from uniform due to the t -wise uniform independence of the outputs of **ZEnc**, and independence of $A_{j,\ell+1} \cdot F_{\ell+1}$.

Otherwise, $Z_{\ell+1,j}$ is simulated in a previous group. In this case, $\ell + 1 \in S$ and $A_{j,\ell+1} \in G3$ is already perfectly simulated. We use these elements to calculate $S_j^{(\ell+1)} = S_j^{(\ell)} + Z_{\ell+1,j} + A_{j,\ell+1} \cdot F_{\ell+1}$.

This finishes the proof.

G.3 Proof P-Refresh (cf. Algorithm 1) is fault-invariant with respect to $\mathcal{F}^+$, Lemma 14

Proof (P-Refresh (cf. Algorithm 1) is fault-invariant with respect to $\mathcal{F}^+$, Lemma 14).

Let $\mathbf{R} = r_{d+1}^{(d+1)}, \dots, r_{d+1}^{(n-1)}, r_{d+2}^{(d+1)}, \dots, r_{n-1}^{(n-2)}$ denote arbitrary but fixed randomness. Let $p = f^{\mathbf{R}}$ denote an arbitrary but fixed intermediate value in $C^{\mathbf{R}}$. Let p' denote the corresponding intermediate value (probe) on the same wire in $T[\mathbf{C}^{\mathbf{R}}]$, and let $f'^{\mathbf{R}}$ describe the way the circuit $T[\mathbf{C}^{\mathbf{R}}]$ computes p' .

In the following, we consider all possible fault injections within the calculation of p' and define the resulting fault functions in $\mathcal{F}^+$. Note that we consider faults that have an impact on the calculation of p' . Any faults that appear in $T[C^R]$ in comparison to C^R , but not in the calculation of p' are not considered in the following. In more detail, let $|T|$ denote the number of faults inserted to T . We define $\tilde{T} = \min_{|\bar{T}|} \{\bar{T} \in \mathcal{A}(\mathcal{F}) | \bar{p} = p'\}$, where the intermediate value $\bar{p}$ of $T'[\mathbf{C}^{\mathbf{R}}]$ denotes a value on the same wire as p' in $T[\mathbf{C}^{\mathbf{R}}]$. We consider $\tilde{T}$ in the following. Note that this happens without loss of generality, as any other faults do not have an impact on p' , by definition of $\tilde{T}$.

To show circuit fault invariance, it is sufficient to prove that any fault affecting p' can

- either be pushed to any previous wire on a path $P_{v \rightarrow p'}$, where v denotes an input or a random element

- or be pushed to a fault on p itself.

We show this by transforming the faults to faults on the inputs and by using the associativity of the addition.

We study these faults in $\tilde{T}$ (i.e., faults in T that affect p') one after another and shift them to the input or output as follows:

- **Group 1:** Faults inserted in line 5 and line 7: Faults inserted in line 5 are of the form $r_j^{(k)} + a$ and correspond to faults on the randomness, i.e., $\tilde{\xi}_{j,k}(x) := x + a$. Faults inserted in line 7 can be pushed to faults on the randomness as follows. Any fault inserted to $R_{i,\ell}^{(k)} := \sum_{j=0}^{\ell} r_j^{(k)} x_i^j$ can be shifted to the randomness, since x_i^j is a public constant. Namely, $R_{i,\ell}^{(k)} + a = \sum_{j=0}^{\ell} r_j^{(k)} x_i^j + a = \sum_{j=0}^{\ell-1} r_j^{(k)} x_i^j + (r_{\ell}^{(k)} + \frac{a}{x_i^j}) \cdot x_i^j$, and we define $\tilde{\xi}_{\ell,k}(x) := x + \frac{a}{x_i^j}$.
- **Group 2:** Any fault within the calculations of **ZEnc** in line 10. Since **ZEnc** is assumed to be fault-invariant, all faults within the calculation of **ZEnc** can be shifted to its inputs or outputs. Moreover, $Z_{i,j}$ linearly goes into the output S_j , hence, any fault a inserted to $Z_{i,j}$ can be written as $S_j + a$. For these faults, we define $\xi_{\text{out}}^{Z_{i,j}}(x) := x + a$.
- **Group 3:** Group 3 consists of any faults inserted into the calculation of $A_{j,i}$ (line 12). Such a fault a , i.e., faults to $A_{i,j}^{\ell} := \text{Id}_{i,j} + \sum_{k=d+1}^{\ell} R_j^{(k)} \cdot V_{k,i}^{-1}$ can be represented as

$$\begin{aligned}
A_{i,j}^{\ell} + a &= \text{Id}_{i,j} + \sum_{k=d+1}^{\ell} R_j^{(k)} \cdot V_{k,i}^{-1} + a \\
&= \text{Id}_{i,j} + \sum_{k=d+1}^{\ell-1} \left(R_j^{(k)} \cdot V_{k,i}^{-1} \right) + R_j^{(\ell)} \cdot V_{\ell,i}^{-1} + a \\
&= \text{Id}_{i,j} + \sum_{k=d+1}^{\ell-1} \left(R_j^{(k)} \cdot V_{k,i}^{-1} \right) + \sum_{m=d+1}^{n-1} r_m^{(\ell)} \cdot V_{\ell,i}^{-1} + a \\
&= \text{Id}_{i,j} + \sum_{k=d+1}^{\ell-1} \left(R_j^{(k)} \cdot V_{k,i}^{-1} \right) + \sum_{m=d+1}^{n-2} r_m^{(\ell)} \cdot x_i^j \cdot V_{\ell,i}^{-1} + \left(r_{n-1}^{(\ell)} + \frac{a}{V_{\ell,i}^{-1}} \right) \cdot x_i^j \cdot V_{\ell,i}^{-1}.
\end{aligned}$$

Hence, we can propagate such a fault a to the randomness (input) and set $\tilde{\xi}_{m,\ell}(r) := r + \frac{a}{V_{\ell,i}^{-1}}$.

- **Group 4:** Faults of group 4 correspond to faults in lines 13 and 14, i.e., faults inserted within the calculation of $Z_{i,j} + A_{i,j} \cdot F_i$ or $S_j \leftarrow S_j + Z_{i,j}$. These faults can be propagated to the outputs, since $S_j = \sum_{i=0}^{n-1} (Z_{i,j} + A_{i,j} \cdot F_i)$.
- **Group 5:** It is left to consider faults inserted into F_i . However, faults inserted into F_i can be directly propagated to the input, since F_i is the input and it exists no predecessor.

Since for all $\xi_1, \xi_2 \in \mathcal{F}^+$ it holds true that $\xi_1(\xi_2(x)) \in \mathcal{F}^+$ and by associativity of the addition, we can use the fault functions (that are relevant for calculating

p') as described above to define the overall fault functions on any predecessor of p' , the randomness, the inputs, and the output. This finishes the proof.

H Supplementary Material for Section 6

Algorithm 6 sZEnc_{n,d} for $n = d + e + 1$ and $k \leq d$ [12, Algorithm 4]

Output: Randomized sharing $(y_i)_{i \in [n]}$ of zeros $\mathbf{v}_{\text{zero}} = (0_0, \dots, 0_{k-1}, r_k, \dots, r_d)$ where $r_k, \dots, r_d \leftarrow_{\$} \mathbb{F}$

```

6.1 initialize  $(Y_i)_{i \in [n]}$ 
6.2 for  $j \in \{0, \dots, d - k\}$  do
6.3    $(Z_i)_{i \in [n]} \leftarrow \mathbf{ZEnc}_{n,d}$ 
6.4    $(Y_i)_{i \in [n]} \leftarrow (Y_i)_{i \in [n]} + (Z_i)_{i \in [n]}$ 
6.5 return  $(Y_i)_{i \in [n]}$ 

```

Algorithm 7 SWComp for $n = d + e + 1$ [12, Algorithm 8]

Input : Four degree- d sharings of $\mathbf{v}^j$ as $(H_i^j)_{i \in [n]}$, with $j \in [4]$.

Output: Degree- d sharing $(Q_i)_{i \in [n]}$ of $\mathbf{v}^0 + \mathbf{v}^1 + \mathbf{v}^2 + \mathbf{v}^3$.

```

7.1  $(Q_i)_{i \in [n]} \leftarrow \mathbf{sZEnc}$ 
7.2  $(Q_i)_{i \in [n]} \leftarrow \left( \left( (Q_i)_{i \in [n]} + (H_i^0)_{i \in [n]} \right) + (H_i^1)_{i \in [n]} \right) + (H_i^2)_{i \in [n]} + (H_i^3)_{i \in [n]}$ 
7.3 return  $(Q_i)_{i \in [n]}$ 

```

Algorithm 8 SWMult for $n = d + e + 1$ [12, Algorithm 2]

Input : Degree- d shares of f as $(F_i)_{i \in [n]}$ and g as $(G_i)_{i \in [n]}$.

Output: Degree- $2d$ shares of $f \cdot g$ as $(Q'_i)_{i \in [n]}$.

```

8.1 for  $i \in [n]$  do
8.2    $Q'_i \leftarrow F_i \cdot G_i$ 
8.3 return  $(Q'_i)_{i \in [n]}$ 

```

Proof (Correctness of $\hat{A}$, Lemma 15).

$$\begin{aligned}
S_i &= \sum_{j=0}^{n-1} \hat{A}_{i,j} \cdot F_j \\
&= \sum_{j=0}^{n-1} \left(\left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \right) + \sum_{k=d_{\text{in}}+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \right) \cdot F_j \\
&= \sum_{j=0}^{n-1} \left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \cdot F_j \right) + \sum_{k=d_{\text{in}}+1}^{n-1} \hat{R}_i^{(k)} \cdot f_k \\
&= \sum_{j=0}^{n-1} \left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \cdot F_j \right)
\end{aligned}$$

since $f_k = 0 \forall k \in [d_{\text{in}}+1, n]$ because $(F_i)_{i \in [n]}$ is assumed to be a valid encoding of degree up to d_{in} . By correctness of $\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})}$, $(S_i)_{i \in [n]}$ corresponds to a degree- d_{out} polynomial with $f(0) = s(0)$.

Proof (Re-randomization by $\hat{A}$, Lemma 16). By definition of $\hat{A}$, we know

$$\begin{aligned}
S_i &= \sum_{j=0}^{n-1} \hat{A}_{i,j} \cdot F_j \\
&= \sum_{j=0}^{n-1} \left(\left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \right) + \sum_{k=d_{\text{in}}+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \right) \cdot F_j \\
&= \sum_{j=0}^{n-1} \left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \cdot F_j \right) + \sum_{k=d_{\text{in}}+1}^{n-1} \hat{R}_i^{(k)} \cdot f_k
\end{aligned}$$

Hence, we obtain the following coefficients

$$\begin{aligned}
s_u &= \sum_{i=0}^{n-1} S_i \cdot V_{u,i}^{-1} \\
&= \sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} \left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \cdot F_j \right) + \sum_{k=d_{\text{in}}+1}^{n-1} \hat{R}_i^{(k)} f_k \right) \cdot V_{u,i}^{-1} \\
&= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \cdot F_j \cdot V_{u,i}^{-1} \right) + \sum_{k=d_{\text{in}}+1}^{n-1} \left(\sum_{i=0}^{n-1} \hat{R}_i^{(k)} \cdot V_{u,i}^{-1} \right) \cdot f_k \\
&= \sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-1} \left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \cdot F_j \cdot V_{u,i}^{-1} \right) \right) + \sum_{k=d_{\text{in}}+1}^{n-1} r_u^{(k)} \cdot f_k
\end{aligned}$$

where we define $r_u^{(k)} = 0 \forall u \in [1, d_{\text{out}}+1]$. Hence, since $r_u^{(k)}$ are assumed to be indistinguishable from uniform (and independent of each other) for $u \in [d_{\text{out}}+1, n] \cup \{0\}$ and $k \in [d_{\text{in}}+1, n]$, also $s_u, u \in [d_{\text{out}}+1, n] \cup \{0\}$ fulfill this property.

*Proof (Correctness of **AdaptedSplitRed** _{$\hat{A}$} (cf. Alg. 3), Lemma 17).*

$$\begin{aligned}
F_i'' &= \sum_{j \in [\frac{n}{2}]} \mathcal{F}_i'^{j+\frac{n}{2}} + Z_i^{j+\frac{n}{2}} \\
&= \sum_{j \in [\frac{n}{2}]} \hat{A}_{i,j+\frac{n}{2}} \cdot F_{j+\frac{n}{2}} + Z_i^{j+\frac{n}{2}} \\
&= \sum_{j \in [\frac{n}{2}]} \left(\lambda_{i,j+\frac{n}{2}}^{(d \rightarrow \frac{d}{2})} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j+\frac{n}{2}}^{-1} \right) \cdot F_{j+\frac{n}{2}} + Z_i^{j+\frac{n}{2}} \\
&= \sum_{j \in [\frac{n}{2}]} \lambda_{i,j+\frac{n}{2}}^{(d \rightarrow \frac{d}{2})} F_{j+\frac{n}{2}} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} V_{k,j+\frac{n}{2}}^{-1} \cdot F_{j+\frac{n}{2}} + \tilde{Z}_i^{j+\frac{n}{2}} - \hat{Z}_i^j.
\end{aligned}$$

Similarly, we can calculate

$$\begin{aligned}
F_i' &= \sum_{j \in [\frac{n}{2}]} \mathcal{F}_i'^j + Z_i^j \\
&= \sum_{j \in [\frac{n}{2}]} \hat{A}_{i,j} F_j + Z_i^j \\
&= \sum_{j \in [\frac{n}{2}]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \right) F_j + \tilde{Z}_i^j + \hat{Z}_i^j.
\end{aligned}$$

Let f', f'' denote the polynomials corresponding to the shares $(F_i')_i$ and $(F_i'')_i$ respectively.

$$\begin{aligned}
f_\ell' &= \sum_{i=0}^{n-1} F_i' \cdot V_{\ell,i}^{-1} \\
&= \sum_{i=0}^{n-1} \left(\sum_{j \in [\frac{n}{2}]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \right) F_j + \tilde{Z}_i^j + \hat{Z}_i^j \right) \cdot V_{\ell,i}^{-1} \\
&= \sum_{i=0}^{n-1} \sum_{j \in [\frac{n}{2}]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \right) F_j \cdot V_{\ell,i}^{-1} + \tilde{Z}_i^j \cdot V_{\ell,i}^{-1} + \hat{Z}_i^j \cdot V_{\ell,i}^{-1} \\
&= \sum_{i=0}^{n-1} \left(\sum_{j \in [\frac{n}{2}]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j \cdot V_{\ell,i}^{-1} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \cdot F_j \cdot V_{\ell,i}^{-1} \right) \right. \\
&\quad \left. + \tilde{Z}_i^j \cdot V_{\ell,i}^{-1} + \hat{Z}_i^j \cdot V_{\ell,i}^{-1} \right).
\end{aligned}$$

Similarly, we can calculate f''_ℓ as

$$\sum_{i=0}^{n-1} \sum_{j \in [\frac{n}{2}]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_{j+\frac{n}{2}} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j+\frac{n}{2}}^{-1} \cdot F_{j+\frac{n}{2}} \right) \cdot V_{\ell,i}^{-1} + \tilde{Z}_i^{j+\frac{n}{2}} \cdot V_{\ell,i}^{-1} - \tilde{Z}_i^j \cdot V_{\ell,i}^{-1}.$$

Hence, $f'_\ell + f''_\ell$ equals

$$\begin{aligned} & \sum_{i=0}^{n-1} \sum_{j \in [n]} \left(\left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \cdot F_j \right) \cdot V_{\ell,i}^{-1} + \tilde{Z}_i^j \cdot V_{\ell,i}^{-1} \right) \\ &= \sum_{i=0}^{n-1} \left(\sum_{j \in [n]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j \right) \cdot V_{\ell,i}^{-1} + \left(\sum_{j \in [n]} \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1} \cdot F_j \right) \cdot V_{\ell,i}^{-1} \right. \\ & \quad \left. + \sum_{j \in [n]} \left(\tilde{Z}_i^j \cdot V_{\ell,i}^{-1} \right) \right) \\ &= \sum_{i=0}^{n-1} \left(\sum_{j \in [n]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j \right) \cdot V_{\ell,i}^{-1} + \left(\sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot \sum_{j \in [n]} \left(V_{k,j}^{-1} \cdot F_j \right) \right) \cdot V_{\ell,i}^{-1} \right. \\ & \quad \left. + \sum_{j \in [n]} \left(\tilde{Z}_i^j \cdot V_{\ell,i}^{-1} \right) \right) \\ &= \sum_{i=0}^{n-1} \left(\sum_{j \in [n]} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j \right) \cdot V_{\ell,i}^{-1} + \left(\sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot f_k \right) \cdot V_{\ell,i}^{-1} + \sum_{j \in [n]} \left(\tilde{Z}_i^j \cdot V_{\ell,i}^{-1} \right) \right) \\ &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \left(\lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j + \tilde{Z}_i^j \right) \cdot V_{\ell,i}^{-1}, \end{aligned}$$

where the last equation holds due to $f_k = 0 \forall k \in [d+1, n]$ as $(F_i)_{i \in [n]}$ is assumed to correspond to a valid encoding of degree up to d . The term $\left(\sum_{j=0}^{n-1} \lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j \right)_{i \in [n]}$ corresponds to a degree up to $\frac{d}{2}$ encoding by assumption. Moreover, $\left(\tilde{Z}_i^j \right)_{i \in [n]}$ are shares of a zero-encoding of degree up to $\frac{d}{2}$ for all $j \in [n]$. It follows that $f'_\ell + f''_\ell$ is a polynomial of degree $\frac{d}{2}$.

Moreover, $\left(\sum_{j=0}^{n-1} \lambda_{i,j}^{(d \rightarrow \frac{d}{2})} F_j \right)_{i \in [n]}$ encodes the secret $f(0)$ by assumption. Since $\left(\tilde{Z}_i^j \right)_{i \in [n]}$ are zero-encodings for all $j \in [n]$, $f'_\ell + f''_\ell$ encodes $f(0)$ as well. This finishes the proof.

H.1 Mult – LaOla is t -SNI

In this section, we show that **Mult – LaOla** is t -SNI as stated in Lemma 18. The proof for this follows the proof of [12, Theorem 16], hence, we focus on the differences arising from our changes.

Recall that the multiplication gadget consists of three sub-gadgets. As done in [12], this allows us to split the proof into two parts: The probing security of **SWMult** and **SWComp** is considered in Lemma 25, while the NI properties of the composition of **Comp $\hat{A}$** and **AdaptedSplitRed** are addressed in Lemma 26. Combining the two lemmas result in the proof of Lemma 18.

In more detail, let $t' + t'' + t''' \leq t$ where t' denotes the number of output probes of **Mult – LaOla**, t'' the number of internal probes in **SWMult** and **SWComp** and t''' the number of internal probes in **Comp $\hat{A}$** and **AdaptedSplitRed**. It was shown by [12] that $t' + t''$ probes in **SWMult** and in **SWComp** can be simulated with at most t'' probes of each of their input sharings, meaning the simulation needs at most t'' output probes of each **AdaptedSplitRed** (i.e., t'' -SNI). We then prove that there exists a simulator that for **AdaptedSplitRed** composed with **Comp $\hat{A}$** can simulate t'' output probes and t''' internal probes with at most $t'' + t'''$ shares of each input sharing (i.e., $t'' + t'''$ -NI). Combining the two parts gives a simulator that simulates all $t' + t'' + t'''$ probes using only $t'' + t'''$ input shares of each input sharing.

We will now start by considering **SWMult** and **SWComp**.

Lemma 25. *Let t' be the number of output probes of **Mult – LaOla** and t'' the number of internal probes in **SWMult** and **SWComp**. Then, these probes can be simulated with at most t'' probes on the input sharings.*

Proof. This is proven in Corollary 3 of [12] since **SWMult** and **SWComp** remain unchanged.

We can now turn to **Comp $\hat{A}$** and **AdaptedSplitRed**.

Lemma 26. *The composition of **Comp $\hat{A}$** and **AdaptedSplitRed** is $(t'' + t''')$ -NI.*

Proof. In contrast to the original SplitRed algorithm from [12], which computes multiplications using public values $\hat{\lambda}_j^i$, the **AdaptedSplitRed** variant replaces these with the matrix entries $(\hat{A}i, j)_{i,j \in [n]^2}$, where $\hat{A}$ is generated by **Comp $\hat{A}$** . The latter computation is completely independent of any inputs $(F_i)_{i \in [n]}$. Consequently, our change does not affect the security: For any probe in **Comp $\hat{A}$** , we let the simulator run **Comp $\hat{A}$** in the head and fix the results to use in the rest of the simulation. Since [12] showed for public $\hat{\lambda}_j^i$ that the SplitRed algorithm can be perfectly simulated, using the simulator-chosen values also leads to a perfect simulation. It follows that the computation in line 3.8 and line 3.17 of **AdaptedSplitRed** is still secure.

The only other change that **AdaptedSplitRed** introduces is that the values $\mathcal{F}_i^j$ are not computed within their own loop but already directly after $\mathcal{F}_i^{tj}$. As this only changes the order of the computations and not anything else, this also does not impact the security. Together with Lemma 5 from [12], the lemma follows.

We can now show the above Lemma 18.

Proof (Mult – LaOla (cf. Algorithm 4) is t -SNI, Lemma 18). By combining Lemma 25 with Lemma 26, we get a simulator that simulates all $t' + t'' + t'''$ probes using only $t'' + t'''$ input shares of each input sharing.

H.2 Circuit Fault Invariance of Mult – LaOla

We show circuit fault invariance of **Mult – LaOla** by considering **SWMult** and **SWComp** first. Next, we show circuit fault invariance of the composition of **Comp $\hat{A}$** and **SplitRed**.

*Proof (Fault invariance of **SWMult** and **SWComp** [12], Lemma 19).* This result is already proved in Theorem 17 of [12], even for our stronger version of circuit fault-invariance, and we report it here only for completeness. By inspection, **SWMult** is circuit fault-invariant because it only consists of input and output wires, and therefore every additive fault is applied directly either to an input or to an output. We now argue about the circuit fault-invariance of **SWComp**. Since **sZEnc** is circuit fault-invariant with respect to $\mathcal{F}^+$, then the faults performed in line 7.1 either propagate to the intermediate values on a path to the random values, or the random values themselves, which are the same used by **SWComp**, or to the **sZEnc** outputs $(Q_i)_{i \in [n]}$. These Q_i s, along with all the intermediate values corresponding to line 7.2, are later only involved in linear, and thus associative, operations. Therefore, their faults propagate directly to the final output.

*Proof (Composition of **Comp $\hat{A}$** and **AdaptedSplitRed** is also circuit fault-invariant with respect to $\mathcal{F}^+$, Lemma 20).* We consider the following groups of faulted variables and explain how each fault can be propagated to the wires on a path from the inputs or randomness to the fault, or the outputs.

- **Group 1:** Faults injected into any $\hat{A}_{i,j}$ directly or within the execution of **Comp $\hat{A}$** . Since **Comp $\hat{A}$** consists of only linear operations in the randomness, we can propagate each fault to the random values and any intermediate step on the path from the fault to the random values (by associativity).
- **Group 2:** Faults injected on any share F_i . These would be faults in the input directly.
- **Group 3:** Faults on any of the $(\mathcal{F}_i^{j'})_{i,j \in [n]}$, $(F_i')_{i \in [n]}$, $(F_i'')_{i \in [n]}$, or within line 3.5, 3.8, 3.9, 3.11, 3.14, 3.17, 3.18, and 3.20 of Alg. 3. Since the variables $(\mathcal{F}_i^{j'})_{i,j \in [n]}$, $(F_i')_{i \in [n]}$, $(F_i'')_{i \in [n]}$ only appear in the lines listed in this group, which involve exclusively linear, and hence associative, operations, all such faults can be propagated step by step to the outputs.
- **Group 4:** Any fault within the calculation of **ZEnc** in line 3.2, 3.4 and 3.13. Since **ZEnc** is $\mathcal{F}^+$ circuit fault-invariant, we can propagate the faults either step by step to the randomness, or to the variables $\hat{Z}_i^j$, and $\tilde{Z}_i^j$. Faults in the variables $\hat{Z}_i^j$, and $\tilde{Z}_i^j$ can be propagated step by step to the outputs due to associativity of the addition.

This concludes the proof.

H.3 Fault-Robustness (proof of Theorem Theorem 1)

Before proving Theorem 1, we first prove the following two lemmas, where Lemma 27 calculates the probability that $r \in [n]$ out of n unfaulted shares $(F_i)_{i \in [n]}$ are zeros, and Lemma 28 computes the statistical distance between the high-order coefficients of a polynomial and a uniform distribution.

Lemma 27 (Number of Zero Shares in Shamir's Secret Sharings). *Let $p(x) = f_0 + f_1x + \dots + f_tx^t \in \mathbb{F}[x]$ be a degree- t polynomial over a finite field $\mathbb{F}$, where $f_1, \dots, f_t \stackrel{\$}{\leftarrow} \mathbb{F}$ are independent and uniformly random, and $f_0 \in \mathbb{F}$ is fixed. Let $G_i = p(x_i)$, $i \in [n]$, be the n Shamir shares evaluated at pairwise distinct non-zero support points $x_0, \dots, x_{n-1} \in \mathbb{F}$. Define $Z := \#\{i \in [n] : G_i = 0\}$ as the number of zero shares. Then the distribution of the event where Z shares is equal to zero is given by the following formulas,*

– Case 1: $f_0 \neq 0$,

$$\Pr(Z = r) = \begin{cases} \binom{n}{r} \sum_{j=0}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-(r+j)}, & 0 \leq r \leq t, \\ 0, & r > t. \end{cases} \quad (13)$$

– Case 2: $f_0 = 0$,

$$\Pr(Z = r) = \begin{cases} (1 - |\mathbb{F}|^{-t}) \binom{n}{r} \sum_{j=0}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-(r+j)}, & 0 \leq r \leq t, \\ 0, & t < r < n, \\ |\mathbb{F}|^{-t}, & r = n. \end{cases} \quad (14)$$

Moreover, if $|\mathbb{F}| \gg n$, the leading term, $\binom{n}{r} \sum_{j=0}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-(r+j)}$, of the above distribution of admits the approximation

$$\binom{n}{r} \sum_{j=0}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-(r+j)} \approx \binom{n}{r} |\mathbb{F}|^{-r} \left(1 - \frac{n-r}{|\mathbb{F}|} + \mathcal{O}\left(\frac{n^2}{|\mathbb{F}|^2}\right) \right) \quad (15)$$

$$\approx \binom{n}{r} |\mathbb{F}|^{-r}. \quad (16)$$

Proof. We only need to prove the statement for the Case 1 ($f_0 \neq 0$). The analysis of Case 2 follows the same reasoning as in Case 1 ($f_0 \neq 0$), except for an additional degenerate event corresponding to the zero polynomial $p(x) = 0$, i.e., when all coefficients are equal to zero. This event occurs with probability $|\mathbb{F}|^{-t}$ and leads to the special outcome that all n shares $(G_i)_{i \in [n]}$ are zero.

Step 1. For each index $i \in \{1, \dots, n\}$, define the event

$$A_i := \{G_i = 0\} = \{p(x_i) = 0\}.$$

For any subset $U \subseteq \{1, \dots, n\}$, define

$$A_U := \bigcap_{i \in U} A_i = \{G_i = 0 \text{ for all } i \in U\}.$$

Thus, A_U denotes the event that all shares indexed by U are equal to zero. Let $|U| = k$, then the equations

$$p(x_i) = 0, \quad i \in U,$$

form a system of k linear constraints on the coefficient vector $(f_1, \dots, f_t)$. When $k \leq t$, this system has exactly $|\mathbb{F}|^{t-k}$ solutions, and hence

$$\Pr(A_U) = \frac{|\mathbb{F}|^{t-k}}{|\mathbb{F}|^t} = |\mathbb{F}|^{-k}.$$

When $k > t$, the system is overdetermined and admits no solution as long as $f_0 \neq 0$. Therefore,

$$\Pr(A_U) = \begin{cases} |\mathbb{F}|^{-k}, & k \leq t, \\ 0, & k > t. \end{cases}$$

Next, we compute the probability of the event where *exactly* r shares of $(G_i)_{i \in [n]}$ are equal to zero. Without loss of generality, let's first fix a subset $S_0 \subseteq \{1, \dots, n\}$ with $|S_0| = r$. Define the event

$$B := A_{S_0} = \bigcap_{i \in S_0} A_i,$$

which corresponds to the event that $G_i = 0$ for all $i \in S_0$.

We further define the event that the set of indices of zero shares $G_i = 0$ is exactly S_0 by

$$E_{S_0} = B \cap \bigcap_{j \notin S_0} \overline{A_j} = B \setminus \bigcap_{j \notin S_0} A_j.$$

That is, the shares indexed by S_0 are zero, and all remaining shares are nonzero.

Since there are exactly $\binom{n}{r}$ subsets $S_0 \subseteq [n]$ of size r , and all occur with equal probability, we obtain

$$\Pr(Z = r) = \binom{n}{r} \Pr(E_{S_0}).$$

Then, we only need to focus on the calculation of $\Pr(E_{S_0})$. Let $J := \{1, \dots, n\} \setminus S_0$. Then we have,

$$\Pr(E_{S_0}) = \Pr\left(B \setminus \bigcup_{j \in J} A_j\right).$$

Step 2. Using the inclusion–exclusion principle, we obtain,

$$\Pr\left(B \setminus \bigcup_{j \in J} A_j\right) = \sum_{T \subseteq J} (-1)^{|T|} \Pr(B \cap A_T).$$

To prove $\Pr\left(B \setminus \bigcup_{j \in J} A_j\right) = \sum_{T \subseteq J} (-1)^{|T|} \Pr(B \cap A_T)$, we introduce indicator random variables. For each $j \in J$, we have

$$\mathbf{1}_{\bar{A}_j} = 1 - \mathbf{1}_{A_j}, \quad \mathbf{1}_{\bigcap_{j \in J} \bar{A}_j} = \prod_{j \in J} (1 - \mathbf{1}_{A_j}).$$

Consequently, we have,

$$\mathbf{1}_{B \cap \bigcap_{j \in J} \bar{A}_j} = \mathbf{1}_B \prod_{j \in J} (1 - \mathbf{1}_{A_j}).$$

Applying the inclusion–exclusion expansion to the product, we obtain

$$\prod_{j \in J} (1 - \mathbf{1}_{A_j}) = \sum_{T \subseteq J} (-1)^{|T|} \prod_{j \in T} \mathbf{1}_{A_j},$$

where the empty product corresponding to $T = \emptyset$ is equal to 1.

Multiplying both sides by $\mathbf{1}_B$ yields

$$\mathbf{1}_{B \cap \bigcap_{j \in J} \bar{A}_j} = \sum_{T \subseteq J} (-1)^{|T|} \mathbf{1}_{B \cap A_T},$$

which yields

$$\Pr\left(B \setminus \bigcup_{j \in J} A_j\right) = \sum_{T \subseteq J} (-1)^{|T|} \Pr(B \cap A_T),$$

when taking the expectations.

Since $B = A_{S_0}$ and $A_T = \bigcap_{j \in T} A_j$, it follows that

$$B \cap A_T = A_{S_0} \cap A_T = A_{S_0 \cup T}.$$

Therefore,

$$\Pr(E_{S_0}) = \sum_{T \subseteq J} (-1)^{|T|} \Pr(A_{S_0 \cup T}).$$

Step 3. *Evaluation of $\Pr(A_{S_0 \cup T})$.* Recall that $|S_0| = r$ and $J = [n] \setminus S_0$, so that $|J| = n - r$. Therefore, $|S_0 \cup T| = r + |T|$. From Step 1, in Case 1 ($f_0 \neq 0$), the probability that all shares indexed by a set U equal to zero is given by

$$\Pr(A_U) = \begin{cases} |\mathbb{F}|^{-|U|}, & |U| \leq t, \\ 0, & |U| > t. \end{cases}$$

Applying this with $U = S_0 \cup T$, we obtain

$$\Pr(A_{S_0 \cup T}) = \begin{cases} |\mathbb{F}|^{-(r+|T|)}, & r + |T| \leq t, \\ 0, & r + |T| > t. \end{cases}$$

Therefore, only these subsets $T \subseteq J$ satisfying $|T| \leq t - r$ contribute to the sum, and we have

$$\Pr(E_{S_0}) = \sum_{\substack{T \subseteq J \\ |T| \leq t-r}} (-1)^{|T|} \cdot |\mathbb{F}|^{-(r+|T|)}.$$

Step 4. Conversion to a Binomial Sum. We now convert the sum over subsets $T \subseteq J$ into an explicit binomial sum. For a fixed integer $j = |T|$, the number of subsets $T \subseteq J$ with cardinality j is equal to $\binom{n-r}{j}$. Grouping together all terms with the same value of $|T| = j$, the expression obtained in Step 3 can therefore be rewritten as

$$\Pr(E_{S_0}) = \sum_{j=0}^{t-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)}.$$

Step 5. Counting all choices of S_0 . We now account for all possible choices of the index set S_0 . Since there are exactly $\binom{n}{r}$ subsets $S_0 \subseteq [n]$ of size r , and each such subset occurs with the same probability, it follows that

$$\Pr(Z = r) = \binom{n}{r} \Pr(E_{S_0}).$$

Substituting the expression for $\Pr(E_{S_0})$ obtained in Step 4, we obtain, for all $0 \leq r \leq t$,

$$\Pr(Z = r) = \binom{n}{r} \sum_{j=0}^{t-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)} \quad (17)$$

Moreover, since a nonzero polynomial of degree t has at most t roots in $\mathbb{F}$, we necessarily have $\Pr(Z = r) = 0$ for all $r > t$. This completes the proof for the case $f_0 \neq 0$.

Approximation for $|\mathbb{F}| \gg n \geq t$. In this part, we derive a leading-order approximation for $\Pr(Z = r)$ when the field size $|\mathbb{F}|$ is much larger than the number of shares n and $0 \leq r \leq t$ (i.e., Case 1 ($f_0 \neq 0$)). Since we can derive the approximation for Case 2 ($f_0 = 0$) in a similar manner, we omit the detailed calculation.

By factoring out $|\mathbb{F}|^{-r}$ from (17), we obtain

$$\Pr(Z = r) = \binom{n}{r} |\mathbb{F}|^{-r} \sum_{j=0}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-j}. \quad (18)$$

We now separate the first two terms $j = 0, 1$ of (18):

$$\begin{aligned} \sum_{j=0}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-j} &= 1 - \frac{n-r}{|\mathbb{F}|} + \sum_{j=2}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-j} \\ &= 1 - \frac{n-r}{|\mathbb{F}|} + R_r(|\mathbb{F}|), \end{aligned} \quad (19)$$

where

$$R_r(|\mathbb{F}|) := \sum_{j=2}^{t-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-j}.$$

Substituting this identity back into (17), we obtain

$$\Pr(Z = r) = \binom{n}{r} |\mathbb{F}|^{-r} \left(1 - \frac{n-r}{|\mathbb{F}|} + R_r(|\mathbb{F}|) \right). \quad (20)$$

To bound the remaining term, we take absolute values of $|R_r(|\mathbb{F}|)|$:

$$|R_r(|\mathbb{F}|)| \leq \sum_{j=2}^{t-r} \binom{n-r}{j} |\mathbb{F}|^{-j}.$$

Using the standard bound $\binom{n-r}{j} \leq \frac{(n-r)^j}{j!}$, and the exponential series $e^x = \sum_{j=0}^{\infty} \frac{x^j}{j!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$, we obtain

$$|R_r(|\mathbb{F}|)| \leq \sum_{j=2}^{\infty} \frac{1}{j!} \left(\frac{n-r}{|\mathbb{F}|} \right)^j = e^{(n-r)/|\mathbb{F}|} - 1 - \frac{n-r}{|\mathbb{F}|}.$$

Since $|\mathbb{F}| \gg n \geq t$, we have $(n-r)/|\mathbb{F}| \ll 1$. Applying the Taylor expansion $e^x = 1 + x + x^2/2 + O(x^3)$, we obtain

$$|R_r(|\mathbb{F}|)| = \frac{(n-r)^2}{2|\mathbb{F}|^2} + O\left(\frac{n^3}{|\mathbb{F}|^3}\right) = O\left(\frac{n^2}{|\mathbb{F}|^2}\right).$$

Substituting this bound into (20), we obtain, for all $0 \leq r \leq t$,

$$\Pr(Z = r) \approx \binom{n}{r} |\mathbb{F}|^{-r} \left(1 - \frac{n-r}{|\mathbb{F}|} + O\left(\frac{n^2}{|\mathbb{F}|^2}\right) \right). \quad (21)$$

In particular, keeping only the leading-order term, we obtain the asymptotic approximation

$$\Pr(Z = r) \approx \binom{n}{r} |\mathbb{F}|^{-r}, \quad 0 \leq r \leq t, \quad (22)$$

with relative error of order $O(n/|\mathbb{F}|)$.

Case 2 ($f_0 = 0$). The analysis of Steps 1–5 remains valid without modification, except for a single degenerate case. When $f_0 = 0$, the polynomial $p(x)$ is identically zero if and only if

$$f_1 = \cdots = f_t = 0,$$

which occurs with probability $|\mathbb{F}|^{-t}$. In this event, all n Shamir shares satisfy $G_i = 0$, i.e., $Z = n$. Conditioning on the complement of this event, namely on $p(x)$ being nonzero, the distribution of Z for $0 \leq r \leq t$ is identical to the case $f_0 \neq 0$. Consequently, the unconditional distribution of Z is given by the same expression as in Case 1, scaled by a factor $1 - |\mathbb{F}|^{-t}$, with the remaining probability mass $|\mathbb{F}|^{-t}$ concentrated on the outcome $Z = n$.

Lemma 28. *Let $\mathbb{F}$ be a finite field and let $p(x) = f_0 + f_1x + \cdots + f_mx^m \in \mathbb{F}[x]$ be a polynomial. For all $i \in [n]$, let $G_i = p(x_i)$ be the n Shamir's shares evaluated at n pairwise distinct non-zero support points $x_0, \dots, x_{n-1} \in \mathbb{F}$, and assume $m \leq n - 1$. For a parameter e with $n - e \leq m$, define the high-order coefficient vector $\mathbf{F}_{\text{High-Order}} := (f_{n-e}, \dots, f_m)$, the complete coefficient vector $\mathbf{F}_{\text{Full}} := (f_0, \dots, f_m)$, and define the share vector $\mathbf{G} := (G_0, \dots, G_{n-1})$. Let $\mathcal{L}$ denote the distribution of these vectors. For any integer $k \geq 1$, let $\mathcal{U}^k$ denote the uniform distribution over the vector space $\mathbb{F}^k$. Suppose the shares of $\mathbf{G}$ are sampled from a distribution $\mathcal{L}(\mathbf{G})$, then,*

$$\Delta(\mathcal{L}(\mathbf{F}_{\text{High-Order}}), \mathcal{U}^{m-n+e+1}) \leq \Delta(\mathcal{L}(\mathbf{F}_{\text{Full}}), \mathcal{U}^{m+1}) \leq \Delta(\mathcal{L}(\mathbf{G}), \mathcal{U}^n).$$

Proof. The high-level idea is to use [7, Lemma 8] to bound the statistical distance. For the second bound, we need to build a channel (or Markov chain) $U \rightarrow V \rightarrow W \rightarrow X$, then we have $\Delta(V, W) \leq \Delta(U, X)$. Replace U with $\mathcal{U}^{m+1}$, V with $\mathcal{U}^n$, W with $\mathcal{L}(\mathbf{G})$, and X with $\mathcal{L}(\mathbf{F}_{\text{Full}})$, we have $\Delta(\mathcal{L}(\mathbf{F}_{\text{Full}}), \mathcal{U}^{m+1}) \leq \Delta(\mathcal{L}(\mathbf{G}), \mathcal{U}^n)$. We can prove the first bound in a similar manner to the first bound.

- Proof of $\Delta(\mathcal{L}(\mathbf{F}_{\text{Full}}), \mathcal{U}^{m+1}) \leq \Delta(\mathcal{L}(\mathbf{G}), \mathcal{U}^n)$. As the first step, we explain why we can assume the existence of a channel (or Markov chain), $\mathcal{U}^{m+1} \rightarrow \mathcal{U}^n \rightarrow \mathcal{L}(\mathbf{G}) \rightarrow \mathcal{L}(\mathbf{F}_{\text{Full}})$. Essentially, the existence of a channel (or Markov chain), $U \rightarrow V \rightarrow W \rightarrow X$, requires that there exists a joint coupling (U, V, W, X) with these four marginals whose joint distribution factorizes as

$$\Pr[U, V, W, X] = \Pr[U] \cdot \Pr[V | U] \cdot \Pr[W | V] \cdot \Pr[X | W].$$

When we replace U with $\mathcal{U}^{m+1}$, V with $\mathcal{U}^n$, W with $\mathcal{L}(\mathbf{G})$, and X with $\mathcal{L}(\mathbf{F}_{\text{Full}})$, we can easily prove that it satisfies the above requirement, because $\mathcal{U}^n$, $\mathcal{U}^{m+1}$, and $\mathbf{G}$ are independent of each other, and $\mathbf{F}_{\text{Full}} = \text{Van}^{-1} \cdot \mathbf{G}$, where Van^{-1} is the inverse Vandermonde matrix consist of the support points $x_0, \dots, x_{n-1}$. Using [7, Lemma 8], we have $\Delta(\mathcal{L}(\mathbf{F}_{\text{Full}}), \mathcal{U}^{m+1}) \leq \Delta(\mathcal{L}(\mathbf{G}), \mathcal{U}^n)$.

- Proof of $\Delta(\mathcal{L}(\mathbf{F}_{\text{High-Order}}), \mathcal{U}^{m-n+e+1}) \leq \Delta(\mathcal{L}(\mathbf{F}_{\text{Full}}), \mathcal{U}^{m+1})$. When we replace U with $\mathcal{U}^{m+1}$, V with $\mathcal{U}^n$, W with $\mathcal{L}(\mathbf{F}_{\text{Full}})$, and X with $\mathcal{L}(\mathbf{F}_{\text{High-Order}})$, we can build a channel (or Markov chain) $\mathcal{U}^{m+1} \rightarrow \mathcal{U}^n \rightarrow \mathcal{L}(\mathbf{F}_{\text{Full}}) \rightarrow \mathcal{L}(\mathbf{F}_{\text{High-Order}})$. Hence, $\Delta(\mathcal{L}(\mathbf{F}_{\text{High-Order}}), \mathcal{U}^{m+1}) \leq \Delta(\mathcal{L}(\mathbf{F}_{\text{Full}}), \mathcal{U}^n)$.

*Proof ((e, ϵ)-region fault robustness of **Mult – LaOla** Theorem 1).* First, we observe that an unfaulted run of **Mult – LaOla** always yields valid outputs. This is a direct consequence of the gadget’s correctness (cf. Lemma 10).

Now, let $\xi^{F^{\text{in}}}, \xi^{G^{\text{in}}}$ be joint (e, ϵ) -fault vectors, and let T be a fault attack on internal and output wires such that $|T| \leq e$. We want to prove that the output fault ω defined as $\mathbf{Y} + \omega \equiv T[G^R](\mathbf{F} + \xi^{F^{\text{in}}}, \mathbf{G} + \xi^{G^{\text{in}}})$ is a (s, ϵ) -fault vector independent of the input except with probability $1 - \delta'$, where $s \leq |T|$, $\delta' \leq \frac{4}{|\mathbb{F}|^{n-d-1}}$, and $\epsilon' \leq \frac{n}{|\mathbb{F}|}$. Moreover, we wish to prove that the distribution $\mathcal{D}_\omega$ for the fault vector ω only depends on the internal randomness of the gadget.

Proof Overview. The proof proceeds by a structured fault-classification and propagation analysis across the internal sub-gadgets of **Mult – LaOla**. At a high level, we show that under at most e faults per region, every fault occurring during the execution either (i) propagates in a linear, share-wise manner and thus contributes only to the deterministic part of the output fault vector, or (ii) induces sufficient re-randomization of the higher-order coefficients, resulting in a randomized fault vector whose distribution depends only on the internal randomness of the gadget and is ϵ' -indistinguishable from uniform, except with small probability δ' .

Concretely, the proof is organized as follows.

First, we classify all possible types of faults that can occur during the evaluation of the **Mult – LaOla**. We distinguish four types of faults according to their effect on the gadget: linear faults, quasi-linear faults, randomness faults and non-linear faults.

1. The class of *linear faults* (or *share-wise faults*) denotes an input/internal additive fault ξ , such that $v + \xi$ affects only one of the output shares for any input/intermediate value v .
2. The class of *quasi-linear faults* is defined to be the set of all input/internal faults that affect more than one value in a row of a (intermediate/output) matrix, e.g., the faults propagating to an arbitrary but fixed row $\hat{A}_j$. (cf. line 2.10 of **Comp $\hat{A}$**).
3. *Randomness faults* correspond to faults injected into the internal randomness itself (cf. line 2.3 of **Comp $\hat{A}$**).
4. All remaining faults are classified as *non-linear faults*, where a single non-linear fault may affect more than one output share.

Second, we analyze the above four fault types within each sub-gadget and how they propagate through the entire computation. Please note that in these sub-gadgets, a fault performed during the initialization of a variable is equivalent to the fault that is performed on this variable when it is first used in a calculation. Hence, we do not consider faults during the initialization of the variable separately.

Comp $\hat{\mathbf{A}}$ (*Internal and Output Faults*). We distinguish three types of faults (R1, R2, R3) that can be injected into the randomness or during the computation of matrix entries $\bar{R}_{i,j}$ or $\hat{A}_{i,j}$.

- As Algorithm **Comp $\hat{\mathbf{A}}$** does not take any sharings as inputs, we do not need to consider any input faults.
- (Group R1) Faults in random value $r_j^{(k)}$. Any fault directly injected into line 2.3 of **Comp $\hat{\mathbf{A}}$** corresponds to a fault in the randomness $r_j^{(k)}$ itself and is classified as a randomness fault. We highlight that the resulting element $r_j^{(k)} + \xi_j^{r_j^{(k)}}$ remains uniformly distributed, as we consider additive faults $\xi_j^{r_j^{(k)}}$. Recall that the **Comp $\hat{\mathbf{A}}$** helps to realize fault robustness by randomizing each high-order coefficient f_j of sharing $(F_i)_{i \in [n]}$ (where $j \in [\frac{d}{2} + 1, n] \cup \{0\}$) through computing $\tilde{f}_j = r_j^{(d+1)} \cdot f_{d+1} + \dots + r_j^{(n-1)} \cdot f_{n-1}$ in case faults are injected into the inputs $(F_i)_{i \in [n]}$ of **AdaptedSplitRed** (e.g., line 4.2 of **Mult – LaOla**).
- (Group R2) Faults in $\hat{R}_i^{(k)}$. If there is a fault $\xi_{i',j'}^{(k')}$ injected in line 2.5, i.e., in the calculation of the i' -th share $\hat{R}_{i'}^{(k')} = \hat{R}_{i'}^{(k')} + r_{j'}^{(k')} \cdot x_{i'}^{j'}$, of zero-encoding $(\hat{R}_i^{(k)})_{i \in [n]}$, this fault propagates to all the values in the i' -th row $\bar{R}_{[i',:]}$ of matrix $(\bar{R}_{i,j})_{i,j \in [n]}$, since $\bar{R}_{[i',:]}$ equals

$$\sum_{k=d+1}^{n-1} (\hat{R}_{i'}^{(k)} \cdot V_{k,j}^{-1} + \xi_{i',j'}^{(k')} \cdot V_{k,j}^{-1}) = \bar{R}_{i',j} + \sum_{k=d+1}^{n-1} \xi_{i',j'}^{(k')} \cdot V_{k,j}^{-1} \quad \forall j \in [n].$$

Hence, any fault $\xi_{i',j'}^{(k')}$ inserted into the calculation of $\hat{R}_{i'}^{(k)}$ propagates to the i' -th row of matrix $(\bar{R}_{i,j})_{i,j \in [n]}$. These faults are referred to as *quasi-linear faults* since they may affect multiple intermediate values. However, such a fault only affects one output share of the **Mult – LaOla**, which will be shown in Paragraph H.3.

- (Group R3) Faults in matrix $(\bar{R}_{i,j})_{i,j \in [n]}$. Any fault performed in line 2.8 results in a fault in one single element of $(\bar{R}_{i,j})_{i,j \in [n]}$, i.e., one fault inserted in the calculation of $(\bar{R}_{i,j})_{i,j \in [n]}$ only affects one element of this matrix. Similarly, faults in matrix $\hat{A}_{i,j}$, i.e., one fault injected during the share-wise addition in line 2.9, or the output $\hat{A}_{i,j}$ (in line 2.10) only affect one element $\hat{A}_{i,j}$. We classify these faults as linear faults.

AdaptedSplitRed (*Previous and Input Faults*). We now analyze the fault propagation of previously inserted faults, i.e., faults of Groups R1, R2, and R3 injected in **Comp $\hat{\mathbf{A}}$** , as well as faults injected to the input $(F_i)_{i \in [n]}$ of **AdaptedSplitRed** (cf. line 4.2 and 4.4 of **Mult – LaOla**).

- Class of *quasi-linear faults*: Any fault of Group R2 results in one fault in F_i' as well as one fault in F_i'' , since any fault within this class propagates to a fault in the i' -th row of matrix $(\bar{R}_{i,j})_{i,j \in [n]}$. Thus, due to the calculation

$\hat{A}_{i,j} = \lambda_{i,j}^{(d \rightarrow \frac{d}{2})} + \bar{R}_{i,j}$, this fault is present in all the element of the i' row of matrix $(\hat{A}_{i,j})_{i,j \in [n]}$. Since

$$F'_i = \sum_{j \in [\frac{n}{2}]} \hat{A}_{i,j} F_j + \sum_{j \in [\frac{n}{2}]} Z_i^j$$

and similarly

$$F''_i = \sum_{j \in [\frac{n}{2}+1, n]} \hat{A}_{i,j} F_j + \sum_{j \in [\frac{n}{2}+1, n]} Z_i^j$$

any additive fault in the i' -th row $A_{[i',:]}$ propagates only to $F'_{i'}$ as well as $F''_{i'}$, but does not occur in any other outputs F'_i or F''_i , $\forall i \neq i'$. We denote such a fault as *quasi-linear fault* (as we will see that within the remaining sub-gadgets of **Mult – LaOla**, these faults behave like the linear faults).

- Class of *randomness faults*: We call any fault of Group R1, i.e., a fault in $r_j^{(k)}$, $k \in [d+1, n]$, $j \in [\frac{d}{2}+1, n] \cup \{0\}$, a randomness fault. Notably, these faults affect all output shares whenever the faulted inputs result in a high-order polynomial, and thus yield a subclass of non-linear faults. When the randomness fault is performed in isolation, i.e. the inputs are unfaulted or correspond to a degree- d polynomial, these faults do not impact the computation.
- Class of *non-linear faults*: Let $(F_i)_{i \in [n]}$ denote the (possibly faulted) input. If the input sharing $(F_i + \xi_i^{F^{\text{in}}})_{i \in [n]}$ for a input fault vector $\text{weight}(\xi^{F^{\text{in}}}) \leq e$ does not interpolate to a polynomial of degree up to d , then any fault $\xi_i^{F^{\text{in}}}$ performed to F_i is considered to be a non-linear fault.
- *Bad input faults*: Let $(F_i)_{i \in [n]}$ denote the input and let $(\xi^{F^{\text{in}}})_{i \in [n]}$ correspond to the faults injected during the **AdaptedSplitRed**, with $\text{weight}(\xi^{F^{\text{in}}}) \leq e$. If the sharing $(F_i + \xi_i^{F^{\text{in}}})_{i \in [n]}$ does interpolate to a polynomial of degree up to d , we classify it as a bad input fault. However, this happens with very small probability

$$\Pr[\text{BAD}^{\xi^{F^{\text{in}}}, \text{valid}}] = (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$$

when $\leq e$ faults are injected. Here, we compute $\Pr[\text{BAD}^{\xi^{F^{\text{in}}}, \text{valid}}]$ using the same argument for proving Lemma 8. In addition, for any single fault of Group R3, only one output pair, F'_i, F''_i are affect. Hence, such faults are also classified as linear faults.

AdaptedSplitRed (*Internal Faults*). We next analyze the class of faults that can occur *during* the computation of **AdaptedSplitRed** (cf. line 4.2 and 4.4 of **Mult – LaOla**).

- Class of *linear faults*: Faults injected into any intermediate values of **AdaptedSplitRed** and faults within **ZEnc**. Any faults injected on the output of a **ZEnc** as well as

faults injected during its usage (i.e., lines 3.2, 3.4, 3.5, 3.9, 3.13, 3.14, 3.18 of **AdaptedSplitRed**) propagate *linearly* through the gadget **AdaptedSplitRed**, since the output of each **ZEnc** is only used in share-wise addition. Moreover, lines 3.11 and 3.20 perform linear addition exclusively. Finally, we consider lines 3.8 and 3.17. Note that the product $\hat{A}_{i,j} \cdot F_j$ is used to compute F'_i and F''_i , respectively. Since we consider additive faults, any fault injected in these lines propagates to only one output share. Hence, all these faults (except the non-linear internal faults on **ZEnc**) correspond to the class of *linear faults*.

- Class of *non-linear faults* from **ZEnc**: According to the fault robustness assumption of **ZEnc**, $e^{\mathbf{ZEnc},(i)}$ internal or output faults within the $\mathbf{ZEnc}^{(i)}$ result in the injection of a $(s^{(i)}, \epsilon)$ -fault vector $\mathbf{v}^{(i)}$ in the output of $\mathbf{ZEnc}^{(i)}$, except with probability $\delta^{\mathbf{ZEnc}}$, where $s^{(i)} \leq e^{\mathbf{ZEnc},(i)}$.
- *Bad ZEnc faults*. We call the event where $\mathbf{v}^{(i)}$ is not a $(s^{(i)}, \epsilon)$ -fault vector $\text{BAD}^{(\mathbf{ZEnc},i)}$, and therefore $\Pr[\text{BAD}^{(\mathbf{ZEnc},i)}] \leq \delta^{\mathbf{ZEnc}}$. Let $\text{BAD}^{(\mathbf{ZEnc})}$ be the event where at least one of the events $\text{BAD}^{(\mathbf{ZEnc},i)}$ happens. Using the independence of the fault vectors from each other, we get

$$\Pr[\text{BAD}^{(\mathbf{ZEnc})}] = 1 - \prod_i (1 - \Pr[\text{BAD}^{(\mathbf{ZEnc},i)}]).$$

Note that in **AdaptedSplitRed**, we use $\mathbf{ZEnc}_n^d$ to generate zero-encoding of degree- d for $n/2$ times, and $\mathbf{ZEnc}_{n/2}^{\frac{d}{2}}$ to generate zero-encoding of degree- $\frac{d}{2}$ for n times. Please note that we first add the zero-encoding generated by $\mathbf{ZEnc}_n^d$ at line 3.5 and then subtract the same zero-encoding at line 3.14. Hence, we only need to consider the effect of the faults injected in $\mathbf{ZEnc}_{n/2}^{\frac{d}{2}}$ in the following part.

Hence, we have,

$$\Pr[\text{BAD}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}})}] = 1 - \prod_{i \in [n]} (1 - \Pr[\text{BAD}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},i)}]) \leq 1 - (1 - \delta^{\mathbf{ZEnc}_{n/2}^{\frac{d}{2}}})^n.$$

In addition, a non-zero $(e^{\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},(i)}, \epsilon)$ -fault vector $\mathbf{v}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},i)}$ maintains validity (i.e., the degree of $\mathbf{v}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},i)}$ is not larger than d) only with probability $\leq (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}$. We call $\text{BAD}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},\text{valid})}$ the event where all the $\mathbf{ZEnc}_{n/2}^{\frac{d}{2}}$ output faults $\mathbf{v}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},i)}$ are valid $(s^{(i)}, \epsilon)$ -fault vectors, and there exists at least one $\mathbf{v}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},i)} \neq 0$. This event satisfies

$$\Pr[\text{BAD}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}},\text{valid})}] \leq (1 - \Pr[\text{BAD}^{(\mathbf{ZEnc}_{n/2}^{\frac{d}{2}})}]) (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1} \leq (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1}.$$

SWMult (*Input and Internal Faults*). In **SWMult** we perform share-wise operations only. Hence, a fault inserted within this gadget only affects one output share, and such a fault is classified as *linear fault*. Specifically, if a fault $\xi_{i'}^{\text{in}}$ is

injected into the input of **SWMult** (e.g., $F'_{i'}$ or $G'_{i'}$ on line 4.5 of **Mult — LaOla**), this fault affects only the i' -th share of the final output of **Mult — LaOla**. We will analyze the propagation of the previous non-linear faults from **AdaptedSplitRed** separately in paragraph "Fault Propagation" (cf. Paragraph H.3). Notably, this includes the fault propagation of faults that occur during a **ZEnc**, and the non-linear faults that are injected to the input of **AdaptedSplitRed** (cf. line 3.2 and 3.4 of **Mult — LaOla**).

SWComp (*Input and Internal Faults*). Any additive fault on one output of **sZEnc**, input share of **SWComp**, or value computed in line 7.2 with index i propagates *linearly* to an additive fault on the output share i .

As for the internal faults in **sZEnc**, we can easily verify that **sZEnc** satisfies $(e, \delta^{\mathbf{sZEnc}}, \epsilon^{\mathbf{sZEnc}})$ -threshold fault robustness based on the assumption that **ZEnc** is $(e, \delta^{\mathbf{ZEnc}}, \epsilon^{\mathbf{ZEnc}})$ -threshold fault robust, where $\delta^{\mathbf{sZEnc}} = 1 - (1 - \delta^{\mathbf{ZEnc}_n^d})^n$ and $\epsilon^{\mathbf{sZEnc}} = \epsilon^{\mathbf{ZEnc}_n^d}$ (cf. Lemma 24). According to the fault robustness of **sZEnc**, $\epsilon^{\mathbf{sZEnc}}$ internal or output faults within the **sZEnc** result in the injection of a (s, ϵ) -fault vector $\mathbf{v}$ in the output of **sZEnc**, except with probability $\delta^{\mathbf{sZEnc}}$, where $s \leq \epsilon^{\mathbf{sZEnc}}$. Similar to **ZEnc**, we can calculate the probability of event $\Pr[\text{BAD}^{(\mathbf{sZEnc})}] = \delta^{\mathbf{sZEnc}}$, and the probability of the event where the output fault vectors of **sZEnc** are non-zero valid vectors as $\text{BAD}^{(\mathbf{sZEnc}, \text{valid})} = (1 - \delta^{\mathbf{sZEnc}}) \cdot (|\mathbb{F}|^{-1} + \epsilon^{\mathbf{sZEnc}})^{n-d-1}$.

We will analyze the propagation of non-linear faults from **AdaptedSplitRed**, and the propagation of the internal faults in **sZEnc** in the paragraph "Fault Propagation" (cf. Paragraph H.3).

Ruling Out the Bad Events. We now put all the bad events together to provide an upper bound δ' to the failure probability. We call **BAD** the event where at least one of $\text{BAD}_i^{\xi_i^{\text{F}^{\text{in}} \text{valid}}}$, $\text{BAD}_i^{\xi_i^{\text{G}^{\text{in}} \text{valid}}}$, $\text{BAD}^{(\mathbf{ZEnc}_n^{\frac{d}{2}})}$, $\text{BAD}^{(\mathbf{ZEnc}_n^{\frac{d}{2}}, \text{valid})}$, $\text{BAD}^{(\mathbf{sZEnc})}$, and $\text{BAD}^{(\mathbf{sZEnc}, \text{valid})}$ happens.

By union bound,

$$\begin{aligned} \Pr[\text{BAD}] &\leq \text{BAD}_i^{\xi_i^{\text{F}^{\text{in}} \text{valid}}} + \text{BAD}_i^{\xi_i^{\text{G}^{\text{in}} \text{valid}}} + \text{BAD}^{(\mathbf{ZEnc}_n^{\frac{d}{2}})} + \text{BAD}^{(\mathbf{ZEnc}_n^{\frac{d}{2}}, \text{valid})} \\ &\quad + \text{BAD}^{(\mathbf{sZEnc})} + \text{BAD}^{(\mathbf{sZEnc}, \text{valid})} \\ &\leq 2(|\mathbb{F}|^{-1} + \epsilon)^{n-d-1} + 1 - (1 - \delta^{\mathbf{ZEnc}_n^d})^n + (|\mathbb{F}|^{-1} + \epsilon)^{n-d-1} \\ &\quad + 1 - (1 - \delta^{\mathbf{ZEnc}_n^d})^n + (1 - \delta^{\mathbf{sZEnc}}) \cdot (|\mathbb{F}|^{-1} + \epsilon^{\mathbf{sZEnc}})^{n-d-1}. \end{aligned}$$

If we use the **ZEnc** and **sZEnc** from [12,21], and assume the random sampling process (i.e., $r \leftarrow_s \mathbb{F}$ in **P-Refresh**, **Comp $\hat{A}$** , **ZEnc**, **sZEnc**) is $\epsilon (= 0)$ -indistinguishable from uniform random distribution. Then we have $\delta^{\mathbf{ZEnc}} = \delta^{\mathbf{sZEnc}} = 0$, $\epsilon^{\mathbf{ZEnc}} = \epsilon^{\mathbf{sZEnc}} = 0$, and $\Pr[\text{BAD}] = \frac{4}{|\mathbb{F}|^{n-d-1}}$. Therefore, we can set $\delta' = \Pr[\text{BAD}] = \frac{4}{|\mathbb{F}|^{n-d-1}}$. In the following paragraphs, we will always assume that **BAD** event has not happened.

Fault Propagation. In this part, we analyze the propagation of different types of faults injected inside the internal sub-gadgets of **Mult – LaOla**, and the faults injected into its inputs and outputs.

Quasi Linear Faults. We show that one *quasi-linear fault* inside **Comp $\hat{A}$** results in at most one fault in the output sharing of **Mult – LaOla**. Assume a quasi-linear fault was injected within the first (or the second) **Comp $\hat{A}$** (cf. line 4.1 and line 4.3 of **Mult – LaOla**) with respect to indices i, j . It follows that after **AdaptedSplitRed**, only the shares F'_i and F''_i (G'_i and G''_i , respectively) are affected.

Linear Faults and Deterministic Part of Fault Vector v from **ZEnc and **sZEnc**.** As already mentioned, a linear fault injected during the calculation is a type of fault that only affects one output share. We have a total of $\sum_{i \in [n]} e^{\mathbf{ZEnc}, (i)}$ faults within all the **ZEnc** evaluations, and $|T| - \sum_{i \in [n + \frac{n}{2}]} e^{\mathbf{ZEnc}, (i)}$ in the remaining internal wires. Now, the **ZEnc** faults contribute to the deterministic part of each **ZEnc** output fault vector $v^{(i)}$ on at most $e^{\mathbf{ZEnc}, (i)}$ shares, which propagate share-wise to the output. Similarly, the **sZEnc** faults contribute to the deterministic part of each **sZEnc** output fault vector v on at most $e^{\mathbf{sZEnc}}$ shares, which also propagate share-wise to the output. There are at most $|T| - \sum_{i \in [n + \frac{n}{2}]} e^{\mathbf{ZEnc}, (i)} - e^{\mathbf{sZEnc}}$ linear faults in the rest of the circuit. This means that all the linear faults contribute to the deterministic part of the output in at most $(\sum_{i \in [n + \frac{n}{2}]} e^{\mathbf{ZEnc}, (i)} + e^{\mathbf{sZEnc}}) + (|T| - \sum_{i \in [n + \frac{n}{2}]} e^{\mathbf{ZEnc}, (i)} - e^{\mathbf{sZEnc}}) = |T|$ shares.

Non-linear and Randomness Faults. Recall that this class of faults occurs if the inputs $(F_i)_{i \in [n]}$ (or $(G_i)_{i \in [n]}$) to **AdaptedSplitRed** are faulted, but the faulty inputs do not interpolate to a polynomial of degree up to d , or the randomness $r_j^{(k)}$ (generated on line 4 of **Comp $\hat{A}$**) is faulted, or when faults within **ZEnc** or **sZEnc** occur. We begin by considering the non-linear faults on the input shares $(F_i)_{i \in [n]}, (G_i)_{i \in [n]}$ of **AdaptedSplitRed**, as well as the randomness generated within **Comp $\hat{A}$** .

If the randomness $r_m^{(k)}$ is faulted but the inputs $(F_i)_{i \in [n]}$ (or $(G_i)_{i \in [n]}$) are not, then the underlying polynomial coefficient $f_k^{F_i} = 0$ (or $f_k^{G_i} = 0$) for every $k \geq d + 1$, and therefore the fault does not affect the final output $(Q_i)_{i \in [n]}$ (cf. line 4.10 of **Mult – LaOla**). Note that the faults injected into **ZEnc** or **sZEnc** have the same effect.

For simplicity, we only analyze the sharing $(F_i)_{i \in [n]}$ and highlight that an analogous analysis can be performed on $(G_i)_{i \in [n]}$. Since we assumed that none of the BAD event has occurred, at least one of the inputs to the **AdaptedSplitRed** summed with its injected faults during the multiplication gadget, i.e., $(F_i + \xi_i^{F_{\text{in}}})_{i \in [n]}$ and $(G_i + \xi_i^{G_{\text{in}}})_{i \in [n]}$, interpolates to a polynomial of degree greater than d .

We denote the coefficients of the polynomial corresponding to the faults $(\xi_i^{F_{\text{in}}})_{i \in [n]}$ by $\Delta^F = (\Delta_0^F, \dots, \Delta_{n-1}^F)$. Recall that we identified randomness faults as the faults injected to $r_j^{(k)}$, $j \in [\frac{d}{2} + 1, n] \cup \{0\}$ and $k \in [d + 1, n]$.

We abbreviate the set $[\frac{d}{2} + 1, n] \cup \{0\}$ by $\mathcal{S}$ and denote faults injected to $r_j^{(k)}$ by $\xi_j^{r(k)}$. Let $\tilde{f}', \tilde{f}''$ denote the polynomial coefficients corresponding to the faulty output of **AdaptedSplitRed**, i.e., $(\tilde{F}'_i)_{i \in [n]}$ and $(\tilde{F}''_i)_{i \in [n]}$, respectively, and let V^{-1} denote the inverse Vandermonde matrix of the support points (cf. line 2.9 of **CompA**). In what follows, we study the polynomial corresponding to their sum, i.e.,

$$\tilde{f} := \tilde{f}' + \tilde{f}'' = V^{-1} \cdot \left((\tilde{F}'_i)_{i \in [n]} \right)^T + V^{-1} \cdot \left((\tilde{F}''_i)_{i \in [n]} \right)^T$$

Hence,

$$\begin{aligned} \tilde{f}_\ell &= \tilde{f}'_\ell + \tilde{f}''_\ell \\ &= \sum_{i=0}^{n-1} \left(\tilde{F}'_i + \tilde{F}''_i \right) V_{\ell,i}^{-1} \\ &= \sum_{i=0}^{n-1} \left(\sum_{j \in [\frac{n}{2}]} \left(\hat{A}_{i,j} \cdot (F_j + \xi_j^{F^{\text{in}}}) + \tilde{Z}_i^j + \hat{Z}_i^j \right) \right. \\ &\quad \left. + \sum_{j \in [\frac{n}{2}]} \left(\hat{A}_{i,j+\frac{n}{2}} \cdot (F_{j+\frac{n}{2}} + \xi_{j+\frac{n}{2}}^{F^{\text{in}}}) + \tilde{Z}_i^{j+\frac{n}{2}} - \hat{Z}_i^j \right) \right) \cdot V_{\ell,i}^{-1} \\ &= \sum_{i=0}^{n-1} \left(\sum_{j \in [n]} (\lambda_{i,j}^{(d \rightarrow d/2)} + \sum_{k=d+1}^{n-1} \hat{R}_i^{(k)} \cdot V_{k,j}^{-1}) (F_j + \xi_j^{F^{\text{in}}}) + \tilde{Z}_i^j \right) \cdot V_{\ell,i}^{-1} \\ &= \sum_{i=0}^{n-1} \left(\sum_{j \in [n]} \left(\lambda_{i,j}^{(d \rightarrow d/2)} \right. \right. \\ &\quad \left. \left. + \sum_{k=d+1}^{n-1} \left(\sum_{m \in \mathcal{S}} (r_m^{(k)} + \xi_m^{r(k)}) x_i^m \right) \cdot V_{k,j}^{-1} \right) (F_j + \xi_j^{F^{\text{in}}}) + \tilde{Z}_i^j \right) \cdot V_{\ell,i}^{-1} \\ &= \sum_{i=0}^{n-1} \left(\sum_{j \in [n]} \lambda_{i,j}^{(d \rightarrow d/2)} \cdot (F_j + \xi_j^{F^{\text{in}}}) \right) V_{\ell,i}^{-1} \\ &\quad + \underbrace{\sum_{i=0}^{n-1} \left(\sum_{j \in [n]} \sum_{k=d+1}^{n-1} \left(\sum_{m \in \mathcal{S}} (r_m^{(k)} + \xi_m^{r(k)}) x_i^m \right) \cdot V_{k,j}^{-1} (F_j + \xi_j^{F^{\text{in}}}) \right)}_{\star)} \cdot V_{\ell,i}^{-1} \\ &\quad + \sum_{i=0}^{n-1} \sum_{j \in [n]} \left(\tilde{Z}_i^j \right) \cdot V_{\ell,i}^{-1} \end{aligned}$$

where

$$\begin{aligned}
\star) &= \sum_{i=0}^{n-1} \left(\sum_{k=d+1}^{n-1} \left(\sum_{m \in \mathcal{S}} (r_m^{(k)} + \xi_m^{r(k)}) x_i^m \cdot \sum_{j \in [n]} (V_{k,j}^{-1} (F_j + \xi_j^{F^{\text{in}}})) \right) \right) \cdot V_{\ell,i}^{-1} \\
&= \sum_{i=0}^{n-1} \left(\sum_{k=d+1}^{n-1} \left(\sum_{m \in \mathcal{S}} (r_m^{(k)} + \xi_m^{r(k)}) x_i^m \cdot (f_k + \Delta_k^F) \right) \right) \cdot V_{\ell,i}^{-1} \\
&= \sum_{k=d+1}^{n-1} \left(\sum_{m \in \mathcal{S}} (r_m^{(k)} + \xi_m^{r(k)}) \cdot (f_k + \Delta_k^F) \sum_{i=0}^{n-1} x_i^m \cdot V_{\ell,i}^{-1} \right) \\
&= \begin{cases} \sum_{k=d+1}^{n-1} (r_\ell^{(k)} + \xi_\ell^{r(k)}) \cdot (f_k + \Delta_k^F) & \text{if } \ell \in [\frac{d}{2} + 1, n] \cup \{0\} \\ 0 & \text{otherwise} \end{cases}
\end{aligned}$$

where the last equation holds true because

$$\sum_{i=0}^{n-1} x_i^m \cdot V_{\ell,i}^{-1} = \begin{cases} 1 & \text{if } \ell = m \\ 0 & \text{otherwise} \end{cases}$$

This holds because we are multiplying the Vandermonde matrix x_i^m , with the inverse Vandermonde matrix V^{-1} , and the result is the identity matrix, where the elements on the diagonal are ones, and the rest of the elements are zeros.

Recall that $(F_i + \xi_i^{F^{\text{in}}})_{i \in [n]}$ interpolates to a polynomial of degree greater than d . Thus, $\exists k^* \in [d+1, n] : (f_{k^*} + \Delta_{k^*}^F) \neq 0$. To simplify notation, we set $\Delta_{k^*} := (f_{k^*} + \Delta_{k^*}^F)$. As all elements $r_{\ell^*}^{(k^*)}$ for $\ell^* \in [d/2 + 1, n] \cup \{0\}$ are chosen uniformly at random and independent of each other and independent to $r_{\ell'}^{(k')}, \forall k^* \neq k', k' \in [d+1, n], \ell \in [d/2 + 1, n] \cup \{0\}$, the element $(r_{\ell}^{(k^*)} + \xi_{\ell}^{r(k^*)}) \cdot \Delta_{k^*}$ is sampled from uniform since $\xi_{\ell}^{r(k^*)}$ is an (wire independent) additive fault. Moreover, due to the assumptions on **ZEnc** and **sZEnc** and the choice of $r_{\ell}^{(k)}$, all the coefficients of the faulty polynomial $\tilde{f}_{\ell}$ are n -wise independent. More specifically, we create n zero-encodings of degree $\frac{d}{2}$ altogether in **SplitRed**. Since the adversary is allowed to insert at most e faults and by definition $e < n$, at least one zero-encoding is unfaulted and outputs a polynomial $z(X)$ of degree up to $\frac{d}{2}$ whose coefficients from $1, \dots, \frac{d}{2}$ are chosen uniformly at random and independent of each other. Thus, these coefficients randomize the faulty coefficients $\tilde{f}_i, i \in [1, \frac{d}{2} + 1]$. This holds because the multiplication with matrix $\hat{A}$ at line 8 of **AdaptedSplitRed** guarantee that the high-order coefficients $0, \frac{d}{2} + 1, \dots, n-1$ are re-randomized if the inputs are faulted. The $\frac{d}{2}$ degree **ZEnc** at line 4, 13 of **AdaptedSplitRed** guarantee that the lower order coefficients $1, \dots, \frac{d}{2}$ are re-randomized.

Altogether, all the coefficients, i.e., $\tilde{f}_0, \dots, \tilde{f}_{n-1}$ of $\tilde{f}$ are independent of each other and uniformly random. Since the (inverse) Vandermonde matrix over finite fields has full rank [34], it follows that the corresponding faulty shares $(\tilde{F}_i)_{i \in [n]}$

with respect to pairwise-distinct support points $x_i, i \in [n]$ are n -wise independent of each other and n -wise indistinguishable from uniformly distributed. Recall that the goal of matrix $\hat{A} \leftarrow \mathbf{Comp}\hat{A}$ is to re-randomize all the high order coefficients of $(F_i)_{i \in [n]}$ in case any high order coefficients are not zero. More importantly, both **ZEnc** and **Comp** $\hat{A}$ sample randomness from an independent uniform distribution. Hence, the randomized coefficients, $\tilde{f}_0, \dots, \tilde{f}_{n-1}$ are n -wise independent and uniform random. We can get a similar result for faulty share $(\tilde{G}_i)_{i \in [n]}$ when there is an input fault.

The output of **Mult** – **LaOla** can be represented as

$$(Q_i)_{i \in [n]} = (F'_i + F''_i + \Delta_i^F)_{i \in [n]} \cdot (G'_i + G''_i + \Delta_i^G)_{i \in [n]} + (\tilde{Q}'_i)_{i \in [n]} + \Delta_i^{\mathbf{SWMult} \vee \mathbf{SWComp}} \quad (23)$$

$$= (\tilde{F}_i)_{i \in [n]} \cdot (\tilde{G}_i)_{i \in [n]} + (\tilde{Q}'_i)_{i \in [n]} + \Delta_i^{\mathbf{SWMult} \vee \mathbf{SWComp}} \quad (24)$$

$$= (\tilde{C}_i)_{i \in [n]} + (\tilde{Q}'_i)_{i \in [n]} + \Delta_i^{\mathbf{SWMult} \vee \mathbf{SWComp}} \quad (25)$$

where Δ_i^F, Δ_i^G are either the linear faults injected to the inputs of **SWMult** or the non-linear faults caused by the faults injected to the initial valid inputs $(F_i)_{i \in [n]}$ and $(G_i)_{i \in [n]}$. $(\tilde{Q}'_i)_{i \in [n]} \leftarrow \mathbf{sZEnc}$ is the possible faulty output of **sZEnc** and $\Delta_i^{\mathbf{SWMult} \vee \mathbf{SWComp}}$ are the internal or output faults injected into **SWMult** and **SWComp**. Please note that each linear fault can only affect one output share of $(Q_i)_{i \in [n]}$. Hence, we focus on the case where Δ_i^F or Δ_i^G are non-linear faults. Besides, we know that the high-order coefficients of $\tilde{C}$ will be re-randomized when non-linear faults are injected, and it is easy to verify that the corresponding fault vector only depends on the internal randomness of the internal gadgets of **Mult** – **LaOla**. Recall that our goal is to compute the statistical distance between the high-order coefficients of $(\tilde{C}_i)_{i \in [n]}$ and a uniform distribution, i.e., ϵ' . Using Lemma 28, we know that

$$\epsilon' = \Delta((\tilde{C}_{\text{High-Order}}), \mathcal{U}) \leq \Delta((\mathbf{F}_{\text{Full}}), \mathcal{U}) \leq \Delta(\tilde{C}, \mathcal{U}), \quad (26)$$

where $\tilde{C}_{\text{High-Order}}$, $\mathbf{F}_{\text{Full}}$, and $\tilde{C}$, are the distribution of the high-order coefficients of shares $(\tilde{C}_i)_{i \in [n]}$, the distribution of all the coefficients of shares $(\tilde{C}_i)_{i \in [n]}$, and the distribution of shares $(\tilde{C}_i)_{i \in [n]}$.

- Both Δ_i^F and Δ_i^G are non-linear faults. Based on the above discussion, we know that the faulty sharing, $(\tilde{F}_i)_{i \in [n]}$ and $(\tilde{G}_i)_{i \in [n]}$, are n -wise independent and 0-indistinguishable from a uniform distribution. Next, we discuss the distribution of $(\tilde{C}_i)_{i \in [n]} = (\tilde{F}_i \cdot \tilde{G}_i)_{i \in [n]}$. It is easy to calculate that for all $i \in [n]$, $\Pr[\tilde{C}_i = 0] = \frac{2}{|\mathbb{F}|} - \frac{1}{|\mathbb{F}|^2}$, and $\Pr[\tilde{C}_i = x] = \frac{1}{|\mathbb{F}|-1}(1 - \frac{2}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|^2})$ for every nonzero x . Hence, we can compute the statistical distribution between

$(\tilde{C}_i)_{i \in [n]}$ and uniform random distribution $\mathcal{U}$ using Lemma 24 as:

$$\epsilon' \leq \Delta(\tilde{C}, \mathcal{U}) \quad (27)$$

$$= \frac{1}{2} \sum_{\mathbf{x} \in \mathbb{F}^n} |\Pr[\tilde{C} = \mathbf{x}] - \Pr[\mathcal{U} = \mathbf{x}]| \quad (28)$$

$$= \frac{1}{2} \left[\left| \frac{2}{|\mathbb{F}|} - \frac{1}{|\mathbb{F}|^2} - \frac{1}{|\mathbb{F}|} \right| + (|\mathbb{F}| - 1) \left| \frac{1}{|\mathbb{F}| - 1} \left(1 - \frac{2}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|^2} \right) - \frac{1}{|\mathbb{F}|} \right| \right] \quad (29)$$

$$= \frac{|\mathbb{F}| - 1}{|\mathbb{F}|^2} \quad (30)$$

- Without loss of generality, we assume that only Δ_i^F are non-linear faults, while $\Delta_i^G = 0$. Please note that the analysis for the case when Δ_i^G are linear faults is similar to the case when $\Delta_i^G = 0$. Without loss of generality, we consider linear fault on $G_i, \dots, \Delta_G$. Hence, we focus on the case when $\Delta_i^G = 0$. In this case, we have that the degree of $(\tilde{G}_i)_{i \in [n]} = (G'_i)_{i \in [n]} + (G''_i)_{i \in [n]}$ is $\frac{d}{2}$.

Before we compute the statistical distance $\Delta((\tilde{C}_i)_{i \in [n]}, \mathcal{U})$, where $(\tilde{C}_i)_{i \in [n]} = (\tilde{F}_i \cdot \tilde{G}_i)_{i \in [n]}$, we define a random variable $R := \#\{i \in \{1, \dots, n\} : \tilde{G}_i = 0\}$, which counts the number of shares in $(\tilde{G}_i)_{i \in [n]}$ that are equal to zero. For each integer $r \in \mathbb{F}$, we use $\Pr[R = r]$ to denote the probability of the event that exactly r shares of $(\tilde{G}_i)_{i \in [n]}$ are zero. Besides, we have $\sum_{r=0}^{\leq \frac{d}{2}} \Pr[R = r] = 1$ because the degree $\frac{d}{2}$ polynomial of $(\tilde{G}_i)_{i \in [n]}$ can generate at most $\frac{d}{2}$ shares that equal to zero unless it is a zero polynomial (with all coefficients equal to zero). Here, we assume that the constant coefficient of $(\tilde{G}_i)_{i \in [n]}$ is non-zero. The analysis when the constant coefficient of $(\tilde{G}_i)_{i \in [n]}$ is zero is analogous and yields a smaller bound on ϵ' . As we are interested in the worst-case bound on ϵ' , we omit this case. For more details, please see Lemma 27.

For each $\mathbf{x} \in \mathbb{F}^n$ and $r \in [n]$, let $\Pr[\tilde{C} = \mathbf{x} | R = r]$ denote the probability of $(\tilde{C}_i)_{i \in [n]} = (\tilde{F}_i \cdot \tilde{G}_i)_{i \in [n]} = \mathbf{x}$ (conditioned on the event that exactly r of $(\tilde{G}_i)_{i \in [n]}$ are zeros), where $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{F}^n$. We obtain ϵ' by

$$\epsilon' \leq \Delta(\tilde{C}, \mathbf{U}) \quad (31)$$

$$= \frac{1}{2} \sum_{\mathbf{x} \in \mathbb{F}^n} |\Pr[\tilde{C} = (\tilde{F}_i \cdot \tilde{G}_i)_{i \in [n]} = (x_i)_{i \in [n]}] - \Pr[\mathbf{U} = \mathbf{x}]| \quad (32)$$

$$= \frac{1}{2} \sum_{\mathbf{x} \in \mathbb{F}^n} |(\sum_{r \in [n]} \Pr[R = r] \cdot \Pr[\tilde{C} = \mathbf{x} | R = r]) - 1 \cdot \Pr[\mathbf{U} = \mathbf{x}]| \quad (33)$$

$$= \frac{1}{2} \sum_{\mathbf{x} \in \mathbb{F}^n} |(\sum_{r \in [n]} \Pr[R = r] \cdot \Pr[\tilde{C} = \mathbf{x} | R = r]) - (\sum_{r \in [n]} \Pr[R = r]) \cdot \Pr[\mathbf{U} = \mathbf{x}]| \quad (34)$$

$$= \frac{1}{2} \sum_{\mathbf{x} \in \mathbb{F}^n} |(\sum_{r \in [n]} \Pr[R = r] \cdot \Pr[\tilde{C} = \mathbf{x} | R = r]) - (\sum_{r \in [n]} \Pr[R = r] \cdot \Pr[\mathbf{U} = \mathbf{x}])| \quad (35)$$

$$= \frac{1}{2} \sum_{\mathbf{x} \in \mathbb{F}^n} |\sum_{r \in [n]} \Pr[R = r] \cdot (\Pr[\tilde{C} = \mathbf{x} | R = r] - \Pr[\mathbf{U} = \mathbf{x}])| \quad (36)$$

$$\leq \frac{1}{2} \sum_{\mathbf{x} \in \mathbb{F}^n} \sum_{r \in [n]} \Pr[R = r] \cdot |\Pr[\tilde{C} = \mathbf{x} | R = r] - \Pr[\mathbf{U} = \mathbf{x}]| \quad (37)$$

$$= \frac{1}{2} \sum_{r \in [n]} (\Pr[R = r] \cdot \sum_{\mathbf{x} \in \mathbb{F}^n} |\Pr[\tilde{C} = \mathbf{x} | R = r] - \Pr[\mathbf{U} = \mathbf{x}]|) \quad (38)$$

In the following part, we discuss how to compute $\Pr[R = r]$ and $\Pr[\tilde{C} = \mathbf{x} | R = r] - \Pr[\mathbf{U} = \mathbf{x}]$.

- $\Pr[R = r]$. As proved in Lemma 27, this probability not only depends on the number r of zero shares in $(\tilde{G})_{i \in [n]}$, but also depends on the secret value (i.e., the constant coefficient f_0^G of $(\tilde{G}_i)_{i \in [n]}$):

* Case 1: $f_0^G \neq 0$, we have:

$$\Pr(R = r) = \begin{cases} \binom{n}{r} \sum_{j=0}^{\frac{d}{2}-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-(r+j)}, & 0 \leq r \leq \frac{d}{2}, \\ 0, & r > \frac{d}{2}. \end{cases}$$

* Case 2: $f_0^G = 0$:

$$\Pr(R = r) = \begin{cases} \left(1 - |\mathbb{F}|^{-\frac{d}{2}}\right) \binom{n}{r} \sum_{j=0}^{\frac{d}{2}-r} (-1)^j \binom{n-r}{j} |\mathbb{F}|^{-(r+j)}, & 0 \leq r \leq \frac{d}{2}, \\ 0, & \frac{d}{2} < r < n, \\ |\mathbb{F}|^{-\frac{d}{2}}, & r = n. \end{cases}$$

- $\sum_{\mathbf{x} \in \mathbb{F}^n} |(\Pr[\tilde{C} = \mathbf{x} | R = r] - \Pr[\mathbf{U} = \mathbf{x}])|$. More specifically, we want to compute the statistical distance between sharing $(\tilde{C}_i)_{i \in [n]}$ (conditioned on event where exactly r shares of $(\tilde{G}_i)_{i \in [n]}$ are zeros) and uniform distribution $\mathbf{U}$. Note that inside $(\tilde{C}_i)_{i \in [n]}$ there are at least r sharings equal to zero, and the remaining $n - r$ sharings are independent of each other and uniformly random because of $(\tilde{F}_i)_{i \in [n]}$. Recall that $R = \#\{i : \tilde{G}_i = 0\}$ is the event where exactly r shares of $(\tilde{G}_i)_{i \in [n]}$ are zeros.

Step 1. We first fix an index set

$$I \subseteq [n] := \{0, 1, 2, \dots, n-1\}, \quad |I| = r,$$

which represents the positions for which $\tilde{G}_i = 0$.

Then, we define the support set

$$S_r := \{\mathbf{x} \in \mathbb{F}^n : x_i = 0 \text{ for all } i \in I\}.$$

That is, S_r consists of all vectors in $\mathbb{F}^n$ whose coordinates indexed by I are fixed to zero, while the remaining $n - r$ coordinates can take arbitrary values from $\mathbb{F}$. Hence, we have $|S_r| = |\mathbb{F}|^{n-r}$.

Step 2: Distribution of $\tilde{C}$ Conditioned on Event $R = r$. Recall that $\tilde{C}_i = \tilde{F}_i \cdot \tilde{G}_i$, where each $\tilde{F}_i$ is uniformly random over $\mathbb{F}$ and independent of the others.

For every index $i \in I$, we have $\tilde{G}_i = 0$, and hence $\tilde{C}_i = \tilde{F}_i \cdot 0 = 0$. Thus, in these r coordinates, $\tilde{C}_i$ is fixed and carries no randomness.

For every index $j \notin I$, we have $\tilde{G}_j \neq 0$. Multiplication by a field element $\tilde{F}_j$ is a bijection over $\mathbb{F}$, hence $\tilde{C}_j = \tilde{F}_j \cdot \tilde{G}_j$ is uniformly distributed over $\mathbb{F}$. Moreover, since the variables $\tilde{F}_j$ are independent, the random variables $\tilde{C}_j$ for all $j \notin I$ are also mutually independent.

Step 3: Computation of $\Pr[\tilde{C} = \mathbf{x} | R = r]$. We distinguish two cases:

- * *Case 1: $\mathbf{x} \notin S_r$.* Then there exists some $i \in I$ such that $x_i \neq 0$. However, under the event $R = r$, we have $\tilde{C}_i = 0$ deterministically for all $i \in I$. Therefore,

$$\Pr[\tilde{C} = \mathbf{x} | R = r] = 0.$$

- * *Case 2: $\mathbf{x} \in S_r$.* Then $x_i = 0$ for all $i \in I$. For such $\mathbf{x}$, we obtain

$$\Pr[\tilde{C} = \mathbf{x} | R = r] = \prod_{i \in I} \Pr[\tilde{C}_i = x_i] \cdot \prod_{j \notin I} \Pr[\tilde{C}_j = x_j].$$

For $i \in I$ we have $\Pr[\tilde{C}_i = x_i] = 1$, and for $j \notin I$ we have $\Pr[\tilde{C}_j = x_j] = 1/|\mathbb{F}|$. Hence,

$$\Pr[\tilde{C} = \mathbf{x} | R = r] = 1^r \cdot \left(\frac{1}{|\mathbb{F}|}\right)^{n-r} = |\mathbb{F}|^{-(n-r)}.$$

Combining these two cases, we obtain

$$\Pr[\tilde{C} = \mathbf{x} \mid R = r] = \begin{cases} |\mathbb{F}|^{-(n-r)}, & x \in S_r, \\ 0, & x \notin S_r. \end{cases}$$

This shows that, conditioned on event $R = r$, the distribution of $\tilde{C}$ is uniform over the support set S_r of size $|\mathbb{F}|^{n-r}$.

Step 4: We split the sum $\sum_{\mathbf{x} \in \mathbb{F}^n} |(\Pr[\tilde{C} = \mathbf{x} \mid R = r] - \Pr[\mathbf{U} = \mathbf{x}])|$ into the disjoint parts $x \in S_r$ and $x \notin S_r$:

$$\sum_{\mathbf{x} \in \mathbb{F}^n} |\Pr[\tilde{C} = \mathbf{x} \mid R = r] - |\mathbb{F}|^{-n}| = \sum_{x \in S_r} ||\mathbb{F}|^{-(n-r)} - |\mathbb{F}|^{-n}| + \sum_{x \notin S_r} |0 - |\mathbb{F}|^{-n}|.$$

* Contribution from $x \in S_r$. For every $x \in S_r$, we compute

$$||\mathbb{F}|^{-(n-r)} - |\mathbb{F}|^{-n}| = |\mathbb{F}|^{-n}(|\mathbb{F}|^r - 1).$$

Since $|S_r| = |\mathbb{F}|^{n-r}$, the total contribution is

$$|\mathbb{F}|^{n-r} \cdot |\mathbb{F}|^{-n}(|\mathbb{F}|^r - 1) = |\mathbb{F}|^{-r}(|\mathbb{F}|^r - 1) = 1 - |\mathbb{F}|^{-r}.$$

* Contribution from $x \notin S_r$. The number of such vectors is

$$|\mathbb{F}|^n - |\mathbb{F}|^{n-r} = |\mathbb{F}|^n(1 - |\mathbb{F}|^{-r}).$$

Each term contributes $|\mathbb{F}|^{-n}$, hence the total contribution equals

$$(|\mathbb{F}|^n - |\mathbb{F}|^{n-r})|\mathbb{F}|^{-n} = 1 - |\mathbb{F}|^{-r}.$$

Step 5: Statistical Distance Conditioned on Event $R = r$. Summing both contributions, we obtain

$$\sum_{\mathbf{x} \in \mathbb{F}^n} |\Pr[\tilde{C} = \mathbf{x} \mid R = r] - \Pr[\mathbf{U} = \mathbf{x}]| = 2(1 - |\mathbb{F}|^{-r}), \quad \text{for all } r \in [n+1].$$

- Now we can compute the statistical distance (here we consider the worst case, i.e., assuming that the constant coefficient of $(\tilde{G})_i$ is non-zero),

$$\Delta(\tilde{C}, \mathbf{U}) = \frac{1}{2} \sum_{r \in [n]} (\Pr[R = r] \cdot \sum_{\mathbf{x} \in \mathbb{F}^n} |(\Pr[\tilde{C} = \mathbf{x} \mid R = r] - \Pr[\mathbf{U} = \mathbf{x}])|) \quad (39)$$

$$= \sum_{r=1}^{\frac{d}{2}} \binom{n}{r} (1 - |\mathbb{F}|^{-r}) \sum_{j=0}^{\frac{d}{2}-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)}. \quad (40)$$

- Approximation of the statistical distance $\Delta(\tilde{C}, \mathbf{U})$. We start from

$$\Delta(\tilde{C}, \mathbf{U}) = \sum_{r=1}^{\frac{d}{2}} \binom{n}{r} (1 - |\mathbb{F}|^{-r}) \sum_{j=0}^{\frac{d}{2}-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)}, \quad (41)$$

and assume that $|\mathbb{F}| \gg n$.

Step 1: Isolating the Inner Sum Term. For each $1 \leq r \leq d/2$, define

$$A_r := \sum_{j=0}^{d/2-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)}.$$

Then

$$\Delta(\tilde{C}, \mathbf{u}) = \sum_{r=1}^{d/2} \binom{n}{r} (1 - |\mathbb{F}|^{-r}) A_r.$$

Next, we approximate A_r .

Step 2: Extending to a full Binomial Sum. Define the full sum

$$T_r := \sum_{j=0}^{n-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)}.$$

Factoring out $|\mathbb{F}|^{-r}$ gives

$$T_r = |\mathbb{F}|^{-r} \sum_{j=0}^{n-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-j} = |\mathbb{F}|^{-r} \left(1 - \frac{1}{|\mathbb{F}|}\right)^{n-r},$$

using the binomial theorem. Then, we can decompose

$$A_r = T_r - E_r,$$

where

$$E_r := \sum_{j=d/2-r+1}^{n-r} \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)}.$$

Step 3: Computing the Bound for E_r . For each term in E_r , we have

$$\left| \binom{n-r}{j} (-1)^j |\mathbb{F}|^{-(r+j)} \right| = \binom{n-r}{j} |\mathbb{F}|^{-(r+j)} \leq |\mathbb{F}|^{-r} \left(\frac{n-r}{|\mathbb{F}|} \right)^j.$$

Because of

$$\binom{n-r}{j} = \frac{(n-r)(n-r-1) \cdots (n-r-j+1)}{j!} \leq \frac{(n-r)^j}{j!} \leq (n-r)^j,$$

we have that

$$\binom{n-r}{j} |\mathbb{F}|^{-(r+j)} \leq |\mathbb{F}|^{-r} \left(\frac{n-r}{|\mathbb{F}|} \right)^j.$$

Let $j_0 = d/2 - r + 1$. Then using geometric series, we have

$$|E_r| \leq |\mathbb{F}|^{-r} \sum_{j=j_0}^{\infty} \left(\frac{n-r}{|\mathbb{F}|} \right)^j = |\mathbb{F}|^{-r} \frac{\left(\frac{n-r}{|\mathbb{F}|} \right)^{j_0}}{1 - \frac{n-r}{|\mathbb{F}|}} = \mathcal{O} \left(|\mathbb{F}|^{-r} \left(\frac{n}{|\mathbb{F}|} \right)^{d/2+1-r} \right),$$

since $|\mathbb{F}| \gg n$.

Step 4: Asymptotic expansion of A_r . Using the Taylor expansion

$$\left(1 - \frac{1}{|\mathbb{F}|}\right)^{n-r} = 1 - \frac{n-r}{|\mathbb{F}|} + \mathcal{O}\left(\frac{n^2}{|\mathbb{F}|^2}\right),$$

we obtain

$$\begin{aligned} A_r &= T_r - E_r \\ &= |\mathbb{F}|^{-r} \left(1 - \frac{n-r}{|\mathbb{F}|} + \mathcal{O}\left(\frac{n^2}{|\mathbb{F}|^2}\right)\right) + \mathcal{O}\left(|\mathbb{F}|^{-r} \left(\frac{n}{|\mathbb{F}|}\right)^{d/2+1-r}\right). \end{aligned}$$

Hence

$$A_r = |\mathbb{F}|^{-r} + \mathcal{O}\left(\frac{n}{|\mathbb{F}|^{r+1}}\right) + \mathcal{O}\left(|\mathbb{F}|^{-r} \left(\frac{n}{|\mathbb{F}|}\right)^{d/2+1-r}\right).$$

Step 5: Substituting into $\Delta(\tilde{C}, \mathbf{u})$. Using A_r , we have

$$\begin{aligned} \Delta(\tilde{C}, \mathbf{u}) &= \sum_{r=1}^{d/2} \binom{n}{r} (1 - |\mathbb{F}|^{-r}) A_r \\ &= \sum_{r=1}^{d/2} \binom{n}{r} (1 - |\mathbb{F}|^{-r}) (|\mathbb{F}|^{-r} + \beta_r), \end{aligned}$$

where

$$\beta_r = \mathcal{O}\left(\frac{n}{|\mathbb{F}|^{r+1}}\right) + \mathcal{O}\left(|\mathbb{F}|^{-r} \left(\frac{n}{|\mathbb{F}|}\right)^{d/2+1-r}\right).$$

Hence,

$$\begin{aligned} \Delta(\tilde{C}, \mathbf{u}) &= \sum_{r=1}^{d/2} \binom{n}{r} (|\mathbb{F}|^{-r} - |\mathbb{F}|^{-2r}) + \mathcal{O}\left(\frac{n^2}{|\mathbb{F}|^2}\right) + \mathcal{O}\left(\left(\frac{n}{q}\right)^{d/2+1}\right) \\ &= \sum_{r=1}^{d/2} \binom{n}{r} |\mathbb{F}|^{-r} + \mathcal{O}\left(\frac{n^2}{|\mathbb{F}|^2}\right) + \mathcal{O}\left(\left(\frac{n}{|\mathbb{F}|}\right)^{d/2+1}\right). \end{aligned}$$

Since all terms are non-negative and using binomial theory, we know that

$$\sum_{r=1}^{d/2} \binom{n}{r} |\mathbb{F}|^{-r} \leq \sum_{r=1}^n \binom{n}{r} |\mathbb{F}|^{-r} = (1 + 1/|\mathbb{F}|)^n - 1.$$

It can be expanded to

$$(1 + 1/|\mathbb{F}|)^n - 1 = \frac{n}{|\mathbb{F}|} + \frac{n(n-1)}{2|\mathbb{F}|^2} + \mathcal{O}\left(\frac{n^3}{|\mathbb{F}|^3}\right).$$

Therefore,

$$\sum_{r=1}^{d/2} \binom{n}{r} |\mathbb{F}|^{-r} = \frac{n}{|\mathbb{F}|} + \mathcal{O}\left(\frac{n^2}{|\mathbb{F}|^2}\right).$$

Combining all the above bounds, we have

$$\epsilon' \leq \Delta(\tilde{C}, \mathbf{u}) \quad (42)$$

$$= \frac{n}{|\mathbb{F}|} + \mathcal{O}\left(\frac{n^2}{|\mathbb{F}|^2}\right) + \mathcal{O}\left(\left(\frac{n}{|\mathbb{F}|}\right)^{d/2+1}\right) \quad (43)$$

$$\approx \frac{n}{|\mathbb{F}|}. \quad (44)$$

Please note that the statistical distance for the case where the constant coefficient of $(\tilde{G})_i$ is zero can be computed in a similar manner and the final result is also $\epsilon' \approx \frac{n}{|\mathbb{F}|}$.

To sum up, we proved that the internal and output (quasi) linear faults injected into **Mult – LaOla** satisfied the deterministic part of the output fault vector, while the non-linear faults injected into the inputs of **Mult – LaOla** satisfied the non-deterministic part of the output fault vector. In addition, since **AdaptedSplitRed** will re-randomized all the high order coefficients of $(F_i)_{i \in [n]}$ and $(G_i)_{i \in [n]}$ once there are input faults injected, we know that the statistical distance between all the e high order coefficients of the final outputs and the uniform distribution is $\epsilon' \leq \max(\frac{|\mathbb{F}|-1}{|\mathbb{F}|^2}, \frac{n}{|\mathbb{F}|})$, and their distribution only depends on the internal randomness sampled in **Mult – LaOla** and its sub-gadgets. The failure probability that injected faults cannot be detected is $\delta' \leq \frac{4}{|\mathbb{F}|^{n-d-1}}$. This finishes the proof.

I Limitations of the Fault Invariance from [12]

In this section, we discuss the fault invariance notion introduced in [12], as well as its role in the combined resilience of masked circuits. While this notion is well defined on its own, we are unsure of its applicability in the context of combined resilience. This section contains a discussion on the issues we found, while we refer the reader to Section 4 for the proposed solution, and to Subsection I.1 for a second approach that details the missing part in the fault invariance proposed in [12]. We begin with a revision of their fault invariance definition. Informally, fault invariance holds when the effect of faults at arbitrary circuit locations on a given wire can be *reduced* to the application of faults at the circuit inputs along with a single fault on the wire itself.

Definition 20 (Fault-Invariance [12, Def.8]). *A circuit C is fault-invariant with respect to a fault set $\mathcal{F}$ if for any $T \in \mathcal{A}(\mathcal{F})$, any intermediate value f in*

$\mathcal{C}^R$ and the according value $\tilde{f}$ in $T[\mathcal{C}^R]$ there are $\xi, \xi_0, \dots, \xi_{m-1} \in \mathcal{F}$ such that it holds

$$\tilde{f}^R(x_0, \dots, x_{n-1}) = \xi \left(f^R(\xi_0(x_0), \xi_1(x_1), \dots, \xi_{m-1}(x_{n-1})) \right)$$

for any input $(x_0, x_1, \dots, x_{n-1})$ and randomness R .

In [12], $\mathcal{F}$ -fault invariance and SNI are claimed to imply wfrSNI for faults in $\mathcal{F}$, provided that $\mathcal{F}$ contains only wire-independent functions. However, we have detected a flaw in their proof, which we describe in the following.

The simulator S' in the proof of Corollary 1 in [12] works as follows. It uses the d -SNI simulator as a building block, evaluates its sub-algorithms over the selection of inputs faulted with the fault-invariance input functions, and faults the resulting values again with the fault-invariance output functions. More formally, S' is defined as

$$S'(a_0, a_1, \dots) = \begin{pmatrix} \xi^{(0)}(S_0(\xi_0^{(0)}(a_0), \xi_0^{(1)}(a_1), \dots)) \\ \xi^{(1)}(S_1(\xi_0^{(1)}(a_0), \xi_1^{(1)}(a_1), \dots)) \\ \vdots \\ \xi^{(d'-1)}(S_{d'-1}(\xi_0^{(d'-1)}(a_0), \xi_1^{(d'-1)}(a_1), \dots)) \end{pmatrix},$$

where $d' \leq d$ and $\xi_j^{(j)}, \xi^{(i)}$ s are, respectively, the fault-invariance input and output functions. However, this construction does not consider configurations where the faults are pushed to input variables that the simulator ignores - with the probes being affected by the faults instead. As a simple example, consider a circuit that takes as input one field element x , sums it with a random value R yielding $x + R$, and then uses the 1-constant gate to copy the output of the sum. Call p_0 the circuit location where $x + R$ is stored right after the sum, and p_1 the circuit location where $x + R$ is stored after being copied. Assume an attacker that probes p_0 and p_1 , and additively faults p_1 as $p'_1 = x + R + 1$. This circuit is $\mathcal{F}^+$ -fault-invariant, as we can use the identity functions for the computation of p_0 , meaning $p'_0 = \text{id}(\text{id}(x) + R)$, and push the fault to the input for p_1 , namely $p'_1 = \text{id}((x+1) + R)$. It is also SNI, as we can consider the d -SNI simulator S that samples one random value $\tilde{R}$ and uses it to perfectly simulate p_0 and p_1 . Notably, S is completely independent of the input. A correct execution of the faulted circuit would compute p'_0, p'_1 as two consecutive random values $(x + R, x + R + 1)$. On the other hand, the input-independence of S would make the simulator S' "forget" the fault, and just output two identical values. In other words, S' would simulate p'_0, p'_1 as

$$\begin{aligned} & (\xi^{(0)}(S_0(\xi_0^{(0)}(a_0), \xi_1^{(0)}(a_1), \dots)), \xi^{(1)}(S_1(\xi_0^{(1)}(a_0), \xi_1^{(1)}(a_1), \dots))) \\ &= (\xi^{(0)}(S_0(.)), \xi^{(1)}(S_1(.))) \\ &= (\tilde{R}, \tilde{R}). \end{aligned}$$

I.1 Fault Invariance with Simulator Invariance

In this section, we generalize the notion of fault invariance introduced by Berndt et al. [12], and discuss its role in the combined resilience of masked circuits.

Informally, fault invariance holds when the effect of faults at arbitrary circuit locations on a given wire can be *reduced* to the application of faults at the circuit inputs along with a single fault on the wire itself. We slightly change the definition provided by Berndt et al. to handle a missing case-analysis in a proof and the limitations presented in the previous section. Put simply, our change allows to consider the randomness $R = R_0, \dots, R_m$ as dedicated inputs and allowing fault functions not only on the inputs $x_0, \dots, x_{n-1}$ of the gadget, but also on the randomness $R_0, \dots, R_m$. Our definition is a generalization of the one presented in [12, Definition 8], and we provide generalized proofs of similar properties.

Definition 21 (Generalized Fault Invariance). *Let $R = R_0, \dots, R_m$ denote uniformly i.i.d. eagerly sampled randomness. Let p denote an intermediate value (probe) in $\mathbb{C}^R$ and let f^R describe the way the circuit $\mathbf{C}$ computes p . Moreover, let p' denote the corresponding intermediate value (probe) on the same wire in $T[\mathbb{C}^R]$, and let f'^R describe the way the circuit $T[\mathbf{C}^R]$ computes p' .*

A circuit $\mathbf{C}(x_0, \dots, x_{n-1})$ is generalized-fault-invariant with respect to a fault set $\mathcal{F}$ if for any $T \in \mathcal{A}(\mathcal{F})$ and for any probe $p' = f'^{R_0, \dots, R_m}(x_0, \dots, x_{n-1})$ in the faulted circuit $T[\mathbf{C}^R]$, there exist fault functions $\xi, \xi_0, \dots, \xi_{n-1}, \tilde{\xi}_0, \dots, \tilde{\xi}_m \in \mathcal{F}$ such that it holds

$$f'^{R_0, \dots, R_m}(x_0, \dots, x_{n-1}) = \xi(f^{\tilde{\xi}_0(R_0), \dots, \tilde{\xi}_m(R_m)}(\xi_0(x_0), \dots, \xi_{n-1}(x_{n-1})))$$

for any inputs $(x_0, \dots, x_{n-1})$ and randomness $R_0, \dots, R_m$.

The first result we discuss is Corollary 1 of [12]. There, $\mathcal{F}$ -fault invariance and SNI are claimed to imply wfrSNI for faults in $\mathcal{F}$, provided that $\mathcal{F}$ contains only wire-independent functions. Our contribution here is twofold. First, we extend the result to also apply to generalized-fault-invariant circuits, by restricting the fault set to just $\mathcal{F}^+$. Second, we fix a missing case-analysis in the construction of the wfrSNI simulator under (two possible) sets of natural assumptions. Overall our approach restores all properties and proof techniques for gadgets: similarly to [12], we prove our constructions are fault-invariant in $\mathcal{F}^+$.

The simulator S' in the proof of Corollary 1 in [12] works as follows. It uses the d -SNI simulator as a building block, evaluates its sub-algorithms over the selection of inputs faulted with the fault-invariance input functions, and faults the resulting values again with the fault-invariance output functions. More formally, S' is defined as

$$S'(a_0, a_1, \dots) = \begin{pmatrix} \xi^{(0)}(S_0(\xi_0^{(0)}(a_0), \xi_0^{(1)}(a_1), \dots)) \\ \xi^{(1)}(S_1(\xi_0^{(1)}(a_0), \xi_1^{(1)}(a_1), \dots)) \\ \vdots \\ \xi^{(d'-1)}(S_{d'-1}(\xi_0^{(d'-1)}(a_0), \xi_1^{(d'-1)}(a_1), \dots)) \end{pmatrix},$$

where $d' \leq d$ and $\xi_j^{(j)}, \xi^{(i)}$ s are, respectively, the fault-invariance input and output functions. However, this construction does not consider configurations where the faults are pushed to input variables that the simulator ignores, as presented previously in this section. To tackle this problem, we work with eagerly sampled randomness R , and describe the randomness $\tilde{R}$ of the simulator S' as a function of R and the circuit input. This approach allows us to embed the faults pushed to the input⁸ within $\tilde{R}$, making sure that the simulator's behavior depends on the faults. Then, we consider the following fr(S)NI simulator S' :

$$S'^{\tilde{R}(x,R)}(a_0, a_1, \dots) = \begin{pmatrix} \xi^{(0)}(S_0^{\tilde{R}(x^{(0)},R)}(\xi_0^{(0)}(a_0), \xi_0^{(1)}(a_1), \dots)) \\ \xi^{(1)}(S_1^{\tilde{R}(x^{(1)},R)}(\xi_0^{(1)}(a_0), \xi_1^{(1)}(a_1), \dots)) \\ \vdots \\ \xi^{(d'-1)}(S_{d'-1}^{\tilde{R}(x^{(d'-1)},R)}(\xi_0^{(d'-1)}(a_0), \xi_1^{(d'-1)}(a_1), \dots)) \end{pmatrix},$$

where the $x^{(i)}$ s denote the faulted inputs $(\xi_0^{(i)}(x_0), \dots, \xi_{d'-1}^{(i)}(x_{d'-1}))$. A crucial step in this approach is ensuring that, $\tilde{R}$ is independent of the values which are not input to the simulator - otherwise, S' would not be valid. This motivates the assumptions 1, 2 and 3 in Lemma 29 and 30. For a simple application, consider again the example above. We know that S takes as input one random value $\tilde{R}$ only, corresponding to $\tilde{R}(x, R) = x + R$. Therefore S' perfectly simulates p'_0 and p'_1 as

$$\begin{aligned} & (\xi^{(0)}(S_0^{\tilde{R}(x^{(0)},R)}(\xi_0^{(0)}(a_0), \xi_0^{(1)}(a_1), \dots)), \xi_1^{\tilde{R}(x^{(1)},R)}(\xi_0^{(1)}(a_0), \xi_1^{(1)}(a_1), \dots)) \\ &= (\xi^{(0)}(S_0^{\tilde{R}(x^{(0)},R)}(.)), \xi^{(1)}(S_1^{\tilde{R}(x^{(1)},R)}(.))) \\ &= (\tilde{R}(x, R), \tilde{R}(x+1, R)) \\ &= (\tilde{R}(x, R), \tilde{R}(x, R) + 1) \\ &= (\tilde{R}, \tilde{R} + 1). \end{aligned}$$

An impossibility result for arbitrary $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$ in the generalized setting. We present a simple example showing that $\mathcal{F}$ -generalized fault invariance and d -SNI do not imply d -frSNI for any $\mathcal{F} \subseteq \mathcal{F}^{\text{ind}}$. Let $\mathcal{F}_0$ be the set containing only the constant zero function, and consider a simple gadget that takes one input x , adds a random value R to it, and outputs $x + R$. This gadget is both SNI and $\mathcal{F}_0$ -generalized-fault-invariant, but not d -frSNI. To understand why, consider the faulted circuit where R is set to 0, and only the resulting output is probed. The output of the faulted circuit is input-dependent, as it equals $x + 0 = x$, but a SNI-simulator is required to simulate it without any input probe. This proves the impossibility of constructing such a simulator in the generalized variant.

⁸ For simplicity, this informal explanation only covers the non-generalized version of fault invariance [12], and thus ignores the faults pushed to the randomness. The generalized case involves faults on R too, and is detailed in Lemma 29 and 30.

Notably, our sample gadget is not $\mathcal{F}_0$ -fault-invariant in the notion of [12], and therefore only provides a counterexample for our generalized setting.

We introduce a definition to simplify notation before presenting our formal results.

Definition 22 (Strong Simulator Invariance). *Let G be a gadget that is d -SNI. If there exists a function $\tilde{R}: \mathbb{F}^n \times \mathbb{F}^m \rightarrow \mathbb{F}^{\tilde{m}}$ of the circuit inputs $\mathbf{x} = (x_0, \dots, x_{n-1})$ and randomness $\mathbf{R} = (R_0, \dots, R_{m-1})$ such that*

1. $\tilde{R}$ is linear,
2. for every fixed input tuple $\mathbf{x}$, $\tilde{R}(\mathbf{x}, \mathbf{R})$ is uniformly distributed when $\mathbf{R}$ is uniformly distributed,
3. for every input tuple $\mathbf{x} \in \mathbb{F}^n$, randomness tuple $\mathbf{R} \in \mathbb{F}^m$, and probe p within G ,

$$f^{\mathbf{R}}(\mathbf{x}) = S^{\tilde{R}(\mathbf{x}, \mathbf{R})}(\mathbf{a}), \quad (45)$$

where f is the function describing how probe p is computed using input $\mathbf{x}$ and randomness $\mathbf{R}$, S is the simulator defined by d -SNI, and $\mathbf{a} = (a_0, a_1, \dots)$ is the selection of values (at most one per input sharing) that S takes as input for the simulation of probe p .

then we call the gadget strong simulator invariant.

We are now ready to introduce the formal results of this section.

Lemma 29 (Generalized version of [12, Cor.1], first variant). *Let G be a gadget that is d -SNI with strong Simulator Invariance and fault-invariant with respect to $\mathcal{F}^+$. Then, the gadget is $d\text{-fr}(S)\text{NI}$ with respect to $\mathcal{F}^+$.*

Lemma 29 under weaker assumptions. Interestingly, the linearity of $\tilde{\mathbf{R}}$ is not a necessary assumption for Lemma 29. In what follows, we provide a second version of Lemma 29 under a weaker (but more technical) assumption. Looking ahead, this variant will turn out useful in proving wfrSNI for the gadgets **P-Refresh** and **AdaptedSplitRed** (cf. Alg. 1 and Alg. 3).

Definition 23 (Weak Simulator Invariance). *Let G be a gadget that is d -SNI. If there exist d functions $\tilde{R}_i: \mathbb{F}^n \times \mathbb{F}^m \rightarrow \mathbb{F}$ of the circuit inputs $\mathbf{x} = (x_0, \dots, x_{n-1})$ and randomness $\mathbf{R} = (R_0, \dots, R_{m-1})$ such that*

1. for every randomness tuple $\mathbf{R} \in \mathbb{F}^m$, and every tuple of functions in $\mathcal{F}^+$, $\xi^{(0)}, \dots, \xi^{(d-1)}: \mathbb{F}^n \rightarrow \mathbb{F}^n$ and $\xi^{R, (0)}, \dots, \xi^{R, (d-1)}: \mathbb{F}^m \rightarrow \mathbb{F}^m$, then the joint distribution of

$$\tilde{R}_0(\xi^{(0)}(\mathbf{x}), \xi^{R, (0)}(\mathbf{R})), \dots, \tilde{R}_{d-1}(\xi^{(d-1)}(\mathbf{x}), \xi^{R, (d-1)}(\mathbf{R})) \quad (46)$$

is independent of at least $n - d$ input variables for $\mathbf{R} \leftarrow_s \mathbb{F}^m$.

2. for every $i \in [d]$, for every input tuple $\mathbf{x} \in \mathbb{F}^n$, randomness tuple $\mathbf{R} \in \mathbb{F}^m$, and probe p_i within G ,

$$f_i^{\mathbf{R}}(\mathbf{x}) = S_i^{\tilde{\mathbf{R}}_i(\mathbf{x}, \mathbf{R})}(\mathbf{a}), \quad (47)$$

where f_i is the function describing how p_i is computed using input $\mathbf{x}$ and randomness $\mathbf{R}$, S_i is the simulator defined by d -SNI, and $\mathbf{a} = (a_0, a_1, \dots)$ is the selection of values (at most one per input sharing) that S takes as input for the simulation of p_i .

3. The input variables from which the joint random variable in Equation 46 is independent include the ones that $S_0, \dots, S_{d-1}$ do not take as input.

Then, we call G d -SNI with weak Simulator invariant.

Now, we can state the second version of Lemma 29 as follows.

Lemma 30 (Generalized version of [12, Cor.1], second variant). *Let G be a gadget that is d -SNI with weak Simulator Invariance and fault-invariant with respect to $\mathcal{F}^+$. Then the gadget is d -fr(S)NI with respect to $\mathcal{F}^+$.*

Proof (Proof of Lemma 29, fixed). To prove d -fr(S)NI, we want to construct a simulator S' such that, for every $d' \leq d$ tuple of probes in the faulted circuit $\mathbf{p}' = (p_0, \dots, p_{d'-1})$,

$$\Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (f^{\mathbf{R}}(\mathbf{x}) = \mathbf{p}') = \Pr_{\tilde{\mathbf{R}} \leftarrow \mathbb{F}^{\tilde{m}}} (S'^{\tilde{\mathbf{R}}}(\mathbf{a}) = \mathbf{p}'), \quad (48)$$

where f' describes the way the faulted circuit computes the probes corresponding to $\mathbf{p}'$, $\mathbf{x} = (x_1, \dots, x_{n-1})$ are the circuit inputs, $\mathbf{R} = (R_0, \dots, R_{m-1})$ are the random values input to the circuit, $a_0, \dots$ are the inputs used by the simulator S' , and $\tilde{\mathbf{R}} = (\tilde{R}_0, \dots, \tilde{R}_{\tilde{m}-1})$ are the random values input to the simulator. To achieve (S)NI, we require the simulator S' to take at most as many input values as the number of (internal) probes. We can rewrite the left-hand side of Equation 48 by making the vector components explicit, yielding

$$\Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (f^{\mathbf{R}}(\mathbf{x}) = \mathbf{p}') = \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (f_0^{\mathbf{R}}(\mathbf{x}) = p'_0, \dots, f_{d'-1}^{\mathbf{R}}(\mathbf{x}) = p'_{d'-1}). \quad (49)$$

Since our circuit is $\mathcal{F}^+$ -fault-invariant, we know that for every $i \in [d']$,

$$f_i^{\mathbf{R}}(\mathbf{x}) = \xi^{(i)}(f_i^{\xi_0^{R, (i)}(R_0), \dots, \xi_{m-1}^{R, (i)}(R_{m-1})}(\xi_0^{(i)}(x_0), \dots, \xi_{n-1}^{(i)}(x_{n-1}))), \quad (50)$$

where $\xi_0^{(i)}, \dots, \xi_{m-1}^{(i)}, \xi^{(i)}, \xi_0^{R, (i)}, \dots, \xi_{m-1}^{R, (i)}$ are functions in $\mathcal{F}^+$. Therefore, we can rewrite the right-hand side of Equation 49 as

$$\begin{aligned} & \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (f_0^{\mathbf{R}}(\mathbf{x}) = p'_0, \dots, f_{d'-1}^{\mathbf{R}}(\mathbf{x}) = p'_{d'-1}) \\ &= \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(f_0^{\xi^{R, (0)}(\mathbf{R})}(\mathbf{x}^{(0)})) = p'_0, \dots, \xi^{(d'-1)}(f_{d'-1}^{\xi^{R, (d'-1)}(\mathbf{R})}(\mathbf{x}^{(d'-1)})) = p'_{d'-1}), \end{aligned} \quad (51)$$

where for every $i \in [n]$, we denote $\mathbf{x}'^{(i)}$ and $\xi^{R,(i)}(\mathbf{R})$ for $(\xi_0^{(i)}(x_0), \dots, \xi_{n-1}^{(i)}(x_{n-1}))$ and $(\xi_0^{R,(i)}(R_0), \dots, \xi_{m-1}^{R,(i)}(R_{m-1}))$ respectively. Due to the d -(S)NI property, for every chosen probe p_i there exists a simulator S_i that, for every input tuple, can simulate one probe given just one share per input sharing. According to the assumption in item 3, these simulators S_i satisfy Equation 47 for every input and randomness tuple. To avoid an overloading of symbols, we assume without loss of generality that all the simulators use the same number of random values.⁹ This implies that we can rewrite the right-hand side of Equation 51 as

$$\begin{aligned} & \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(f_0^{\xi^{R,(0)}(\mathbf{R})}(\mathbf{x}'^{(0)})) = p'_0, \dots, \xi^{(d'-1)}(f_{d'-1}^{\xi^{R,(d'-1)}(\mathbf{R})}(\mathbf{x}'^{(d'-1)})) = p'_{d'-1}) \\ &= \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(S_0^{\tilde{R}(\mathbf{x}'^{(0)}, \xi^{R,(0)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(0)})) = p'_0, \dots, \\ & \quad \xi^{(d'-1)}(S_{d'-1}^{\tilde{R}(\mathbf{x}'^{(d'-1)}, \xi^{R,(d'-1)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(d'-1)})) = p'_{d'-1}), \end{aligned} \quad (52)$$

where $\tilde{\mathbf{a}}^{(i)}$ is the tuple that the simulator takes as input to simulate the probe p'_i with input $\mathbf{x}'^{(i)}$, for every $i \in [d']$. In what follows, we use the linearity of $\tilde{R}$ (cf. item 1), together with the fact that $\xi_0^{(i)}, \dots, \xi_{m-1}^{(i)}, \xi^{(i)}$ are functions in $\mathcal{F}^+$. For every $i \in [d']$, we denote with $\mathbf{c}^{(i)} = (c_0^{(i)}, \dots, c_{n-1}^{(i)})$ and $\mathbf{c}^{R,(i)} = (c_0^{R,(i)}, \dots, c_{n-1}^{R,(i)})$ the constants added by the fault-invariance functions to, respectively, $\mathbf{x}$ and $\mathbf{R}$. More formally, $\xi_j^{(i)}(x_j) = x_j + c_j^{(i)}$ and $\xi_k^{R,(i)}(R_k) = R_k + c_k^{R,(i)}$ for every $j \in [n]$ and $k \in [m]$. Moreover, we denote $\tilde{\mathbf{c}}^{(i)} = \mathbf{c}^{(i)} - \mathbf{c}^{(0)}$ and $\mathbf{s}^{(i)} = \tilde{R}(\tilde{\mathbf{c}}^{(i)}, \mathbf{c}^{R,(i)})$. Notably, $\mathbf{s}^{(i)}$ is independent of both the gadget input $\mathbf{x}$ and randomness $\mathbf{R}$. Then we can rewrite for every $i \in [d']$

$$\begin{aligned} & \tilde{R}(\mathbf{x}'^{(i)}, \xi^{R,(i)}(\mathbf{R})) \\ &= \tilde{R}(\mathbf{x} + \mathbf{c}^{(i)}, \mathbf{R} + \mathbf{c}^{R,(i)}) && (\xi_0^{(i)}, \dots, \xi_{n-1}^{(i)} \in \mathcal{F}^+) \\ &= \tilde{R}((\mathbf{x} + \mathbf{c}^{(0)}) - (\mathbf{c}^{(0)} - \mathbf{c}^{(i)}), \mathbf{R} + \mathbf{c}^{R,(i)}) && (\text{add and subtract } \mathbf{c}^{(0)}) \\ &= \tilde{R}(\mathbf{x}'^{(0)} + \tilde{\mathbf{c}}^{(i)}, \mathbf{R} + \mathbf{c}^{R,(i)}) && (\mathbf{x}'^{(0)} = \mathbf{x} + \mathbf{c}^{(0)}, \tilde{\mathbf{c}}^{(i)} = \mathbf{c}^{(i)} - \mathbf{c}^{(0)}) \\ &= \tilde{R}(\mathbf{x}'^{(0)}, \mathbf{R}) + \tilde{R}(\tilde{\mathbf{c}}^{(i)}, \mathbf{c}^{R,(i)}) && (\text{linearity of } \tilde{R}) \\ &= \tilde{R}(\mathbf{x}'^{(0)}, \mathbf{R}) + \mathbf{s}^{(i)}. && (\mathbf{s}^{(i)} = \tilde{R}(\tilde{\mathbf{c}}^{(i)}, \mathbf{c}^{R,(i)})) \end{aligned} \quad (53)$$

⁹ The simulators have no limitation on the used randomness, therefore we can feed each S_i the randomness used by all other simulators $S_0, \dots, S_{d'-1}$.

According to Equation 53, we can replace $\tilde{R}(\mathbf{x}^{(i)}, \xi^{R,(i)}(\mathbf{R}))$ with $\tilde{R}(\mathbf{x}^{(0)}, \mathbf{R}) + \mathbf{s}^{(i)}$ in the right-hand side of Equation 52 for every $i \in [d']$, yielding

$$\begin{aligned} & \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(S_0^{\tilde{R}(\mathbf{x}^{(0)}, \xi^{R,(0)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(0)})) = p'_0, \dots, \\ & \quad \xi^{(d'-1)}(S_{d'-1}^{\tilde{R}(\mathbf{x}^{(d'-1)}, \xi^{R,(d'-1)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(d'-1)})) = p'_{d'-1}) \\ &= \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(S_0^{\tilde{R}(\mathbf{x}^{(0)}, \mathbf{R}) + \mathbf{s}^{(0)}}(\tilde{\mathbf{a}}^{(0)})) = p'_0, \dots, \\ & \quad \xi^{(d'-1)}(S_{d'-1}^{\tilde{R}(\mathbf{x}^{(d'-1)}, \mathbf{R}) + \mathbf{s}^{(d'-1)}}(\tilde{\mathbf{a}}^{(d'-1)})) = p'_{d'-1}). \end{aligned} \quad (54)$$

We now use item 2 to replace $\tilde{R}(\mathbf{x}^{(0)}, \mathbf{R})$ with an uniformly distributed variable $\tilde{\mathbf{R}}$. Therefore, the right-hand side of Equation 54 becomes

$$\begin{aligned} & \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(S_0^{\tilde{R}(\mathbf{x}^{(0)}, \mathbf{R}) + \mathbf{s}^{(0)}}(\tilde{\mathbf{a}}^{(0)})) = p'_0, \dots, \\ & \quad \xi^{(d'-1)}(S_{d'-1}^{\tilde{R}(\mathbf{x}^{(d'-1)}, \mathbf{R}) + \mathbf{s}^{(d'-1)}}(\tilde{\mathbf{a}}^{(d'-1)})) = p'_{d'-1}) \\ &= \Pr_{\tilde{\mathbf{R}} \leftarrow \mathbb{F}^m} (\xi^{(0)}(S_0^{\tilde{\mathbf{R}} + \mathbf{s}^{(0)}}(\tilde{\mathbf{a}}^{(0)})) = p'_0, \dots, \\ & \quad \xi^{(d'-1)}(S_{d'-1}^{\tilde{\mathbf{R}} + \mathbf{s}^{(d'-1)}}(\tilde{\mathbf{a}}^{(d'-1)})) = p'_{d'-1}). \end{aligned} \quad (55)$$

Finally, we define the simulator S' for the complete faulted circuit as sampling $\tilde{\mathbf{R}} \leftarrow \mathbb{F}^m$ and computing $(p'_0, \dots, p'_{d'-1})$ as

$$(\xi^{(0)}(S_0^{\tilde{\mathbf{R}} + \mathbf{s}^{(0)}}(\tilde{\mathbf{a}}^{(0)})), \dots, \xi^{(d'-1)}(S_{d'-1}^{\tilde{\mathbf{R}} + \mathbf{s}^{(d'-1)}}(\tilde{\mathbf{a}}^{(d'-1)}))).$$

We know that $\tilde{\mathbf{a}}^{(0)}, \dots, \tilde{\mathbf{a}}^{(d'-1)}$ contain at most one share per input sharings, therefore all together correspond to at most $d' - 1$ shares per input sharing. Moreover, every other value is independent of the input $\mathbf{x}$. The chain of equalities from Equation 49 to 53, together with Equation 54 and 55, proves Equation 48. This makes S' a valid simulator, and concludes the proof.

Proof (Proof of Lemma 30, sketch). The proof proceeds identically to the proof of Lemma 29 until Equation 51. Next, we use item 2 to rewrite

$$\begin{aligned} & \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(f_0^{\xi^{R,(0)}(\mathbf{R})}(\mathbf{x}^{(0)})) = p'_0, \dots, \\ & \quad \xi^{(d'-1)}(f_{d'-1}^{\xi^{R,(d'-1)}(\mathbf{R})}(\mathbf{x}^{(d'-1)})) = p'_{d'-1}) \\ &= \Pr_{\mathbf{R} \leftarrow \mathbb{F}^m} (\xi^{(0)}(S_0^{\tilde{R}_0(\mathbf{x}^{(0)}, \xi^{R,(0)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(0)})) = p'_0, \dots, \\ & \quad \xi^{(d'-1)}(S_{d'-1}^{\tilde{R}_{d'-1}(\mathbf{x}^{(d'-1)}, \xi^{R,(d'-1)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(d'-1)})) = p'_{d'-1}), \end{aligned} \quad (56)$$

where $\mathbf{x}^{(i)} = (\xi_0^{(i)}(x), \dots, \xi_{d'-1}^{(i)}(x))$ for every $i \in [d']$. We now define the simulator S' for the faulted circuit as the one sampling $\mathbf{R} \leftarrow \mathbb{F}^m$ and computing $(p'_0, \dots, p'_{d'-1})$ as

$$(\xi^{(0)}(S_0^{\tilde{R}_0(\mathbf{x}^{(0)}, \xi^{R,(0)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(0)})), \dots, \xi^{(d'-1)}(S_{d'-1}^{\tilde{R}_{d'-1}(\mathbf{x}^{(d'-1)}, \xi^{R,(d'-1)}(\mathbf{R}))}(\tilde{\mathbf{a}}^{(d'-1)}))).$$

We know that $\tilde{\mathbf{a}}^{(0)}, \dots, \tilde{\mathbf{a}}^{(d'-1)}$ contain at most one share per input each, therefore all together correspond to at most d' shares per input sharing. Moreover, item 1 and 3 guarantee that the tuple

$$(\tilde{R}_0(\mathbf{x}'^{(0)}, \xi^{R,(0)}(\mathbf{R})), \dots, \tilde{R}_{d'-1}(\mathbf{x}'^{(d'-1)}, \xi^{R,(d'-1)}(\mathbf{R})))$$

depends on at most d' input variables, which are taken as input from S' as $\tilde{\mathbf{a}}^{(0)}, \dots, \tilde{\mathbf{a}}^{(d'-1)}$. This makes S' a valid simulator, and concludes the proof.

Notably, the (S)NI simulators of the gadgets presented by Berndt et al. [12] fulfill weak/strong simulator Invariance.

J Counterexample for the Original **LaOla** Multiplication

In this section, we present an explicit counterexample showing that the original **LaOla** multiplication [12] violates both our definition of fault robustness and the one in [12]. More precisely, we show that there exist support points and a single injected fault at one input $(F_i)_{i \in [n]}$ of **LaOla** such that:

- (1) the single input fault propagates to at least two output shares of **LaOla**, and
- (2) the first high-order coefficient of the output is not uniformly distributed. In particular, it depends only on the embedded secret coefficient g_0 .

Parameters and Fault Injection. We consider the following parameter setting:

$$n = 8, \quad d = 4, \quad e = 3,$$

with all computations performed over the finite field $\mathbb{F}_{67}$. We fix the support points as

$$x = (7, 39, 29, 61, 31, 59, 58, 55),$$

and inject a single additive fault at the first input $(F_i)_{i \in [n]}$ of **LaOla** on share index $\ell = 0$, with fault value $\xi = 1$. The other input, which we denote by $(G_i)_{i \in [n]}$, remains unchanged.

J.1 Fault Propagation to Output Shares

We now show that this single input fault injected into the input sharing $(F_i)_{i \in [n]}$ of **LaOla** propagates to more than two, output shares. We compare the fault-free and faulty executions and denote by $(\Delta Y_i)_{i \in [n]}$ the resulting difference in the output shares of **LaOla**. Since the inputs of **LaOla** are first processed by **SplitRed**, we begin by analyzing the propagation to its outputs.

*Propagation to the outputs of **SplitRed**.* As a first step, $(F_i)_{i \in [n]}$ is input to **SplitRed** to yield two sharings $(F'_i)_{i \in [n]}, (F''_i)_{i \in [n]}$. By design, $(F'_i)_{i \in [n]}$ is only affected by the first $n/2$ shares of $(F_i)_{i \in [n]}$, while $(F''_i)_{i \in [n]}$ is affected by the second half. This implies that the fault injected on the first share only propagates to $(F'_i)_{i \in [n]}$, while $(F''_i)_{i \in [n]}$ remains unchanged. Faulting the first share of $(F_i)_{i \in [n]}$ by adding $\xi = 1$ introduces an additive factor in the computation of $(F'_i)_{i \in [n]}$, corresponding to the first column of the matrix $\hat{\lambda}$ defined in [12]. In other words, we can write

$$(F'_i)_{i \in [n]} = (F_i^{\text{unf}})_{i \in [n]} + (\Delta F'_i)_{i \in [n]},$$

where $(F_i^{\text{unf}})_{i \in [n]}$ corresponds to the value of $(F'_i)_{i \in [n]}$ in the fault-free setting, and the additive factor $(\Delta F'_i)_{i \in [n]}$ introduced by the fault equals the first column of $\hat{\lambda}$, namely $(\hat{\lambda}_{i,0})_{i \in [n]}$. In our parameter setting, this column equals

$$(\Delta F'_i)_{i \in [8]} = (\hat{\lambda}_{i,0})_{i \in [8]} = \begin{pmatrix} 33 \\ 42 \\ 36 \\ 38 \\ 34 \\ 42 \\ 63 \\ 47 \end{pmatrix}.$$

Therefore, the single input fault affects all shares of the vector $(F'_i)_{i \in [n]}$ output by **SplitRed**. Aside from that, our fault attack leaves every other sharing $(F''_i)_{i \in [n]}, (G'_i)_{i \in [n]}, (G''_i)_{i \in [n]}$ unchanged.

*From **SplitRed** to **LaOla** Multiplication.* It remains to relate the fault propagation at the level of **SplitRed** to the output of the entire **LaOla** multiplication. After the execution of **SplitRed**, all subsequent operations in **LaOla** are performed in a share-wise manner, i.e., each output share is computed independently from the corresponding shares of the intermediate sharings. In particular, no further mixing between different share indices is introduced, and the difference between a fault-free and a faulted **LaOla** output equals

$$(\Delta Y_i)_{i \in [n]} = (\Delta F'_i \cdot (G'_i + G''_i))_{i \in [n]}.$$

By construction, $(G'_i + G''_i)$ encodes a polynomial of degree at most $d/2$. By the fundamental Lemma (cf. Lemma 1), it is non-zero in at most $d/2$ locations, meaning that at least $n - d/2$ shares of $(G'_i + G''_i)$ are zero. We conclude that $(\Delta y)_{i \in [n]}$ is non-zero in at least $n - d/2 = 6 \geq 2$ shares.

J.2 A High-Order Coefficient Depending Only on g_0

So far, we have only proven that the single fault modifies at least two output shares. However, fault robustness could still hold if all the high order coefficients

(or the ones with indices from $d+1$ to $n-2$, according to the definition of [12]) of the **LaOla** output were uniformly random. In contrast, we show that a high-order coefficient depends only on the embedded secret g_0 of the input sharing $(G_i)_{i \in [n]}$.

By inspection, the high-order coefficients of the **LaOla** output equal those of the polynomial induced by the fault, namely the polynomial Δy underlying $(\Delta Y)_{i \in [n]} = (\Delta F'_i \cdot (G'_i + G''_i))_{i \in [n]}$. To compute them, we first multiply the polynomials Δf and g^{sum} underlying $(\Delta F'_i)_{i \in [n]}$ and $(G'_i + G''_i)_{i \in [n]}$, respectively. Since this results in a polynomial of degree $n + d/2$, we then reduce its degree using the matrix $\tilde{V}$ recalled in Supp. M. This eventually yields the description of the degree- n polynomial Δy and, in particular, of its high-order coefficients.

Computation of Δf . To get the polynomial Δf , it suffices to multiply its sharing $(\Delta F'_i)_{i \in [8]}$ with the inverse Vandermonde matrix $V^{-1} \in \mathbb{F}^{n \times n}$ associated with the support points $x = (x_0, \dots, x_{n-1})$. This yields the vector $\Delta \mathbf{f}$ containing its coefficients:

$$\Delta \mathbf{f} = V^{-1} \cdot (\Delta F'_i)_{i \in [n]} = \begin{pmatrix} 27 & 21 & 60 & 24 & 4 & 58 & 59 & 16 \\ 34 & 32 & 26 & 43 & 4 & 17 & 15 & 30 \\ 7 & 42 & 16 & 41 & 47 & 51 & 53 & 11 \\ 58 & 54 & 4 & 55 & 9 & 50 & 7 & 31 \\ 37 & 22 & 14 & 2 & 50 & 44 & 29 & 3 \\ 34 & 4 & 49 & 22 & 47 & 28 & 34 & 50 \\ 19 & 30 & 45 & 44 & 42 & 54 & 25 & 9 \\ 51 & 20 & 42 & 9 & 9 & 29 & 29 & 12 \end{pmatrix} \cdot \begin{pmatrix} 33 \\ 42 \\ 36 \\ 38 \\ 34 \\ 42 \\ 63 \\ 47 \end{pmatrix} = \begin{pmatrix} 27 \\ 0 \\ 0 \\ 0 \\ 0 \\ 34 \\ 19 \\ 51 \end{pmatrix}.$$

Computation of Δg^{sum} . Since no fault is injected during the computation of $(G'_i + G''_i)_{i \in [n]}$, the underlying polynomial has degree $d/2$ and encodes the same secret as the input $(G_i)_{i \in [n]}$. Therefore, we can represent the underlying polynomial using the following vector of coefficients

$$\mathbf{g}^{\text{sum}} = \begin{pmatrix} g_0 \\ z_1 \\ z_2 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix},$$

where g_0 denotes the embedded secret, and $z_1, z_2 \in \mathbb{F}_{67}$ are fresh random coefficients arising from the degree- $t = d/2$ zero encoding.

Computation of Δy . We compute the description of Δy by first forming the full product coefficients

$$h_m = \sum_{i+j=m} \Delta f_i \cdot g_j^{\text{sum}}, \quad 0 \leq m \leq 2n-2,$$

and then applying the conversion matrix $\tilde{V}$ (cf. Supp. M) to reduce its degree to n . Hence we can rewrite the k -th coefficient of the output fault polynomial Δy as

$$\Delta y_k = \sum_{m=0}^{2n-2} \tilde{V}_{k,m} \cdot \Delta h_m = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \tilde{V}_{k,i+j} \cdot \Delta f_i \cdot g_j^{\text{sum}} = \sum_{i=0}^{n-1} \sum_{j=0}^t \tilde{V}_{k,i+j} \Delta f_i g_j^{\text{sum}},$$

where the last equality holds because g_j^{sum} is zero for every $t > d/2$. Moreover, since $(g_0^{\text{sum}}, g_1^{\text{sum}}, g_2^{\text{sum}}) = (g_0, z_1, z_2)$, we can rewrite

$$\Delta y_k = c_{k,0} \cdot g_0 + c_{k,1} \cdot z_1 + c_{k,2} \cdot z_2,$$

where

$$c_{k,j} = \sum_{i=0}^{n-1} \tilde{V}_{k,i+j} \cdot \Delta f_i, \quad j \in \{0, 1, 2\}.$$

A direct computation for our parameter setting shows that, for the high-order index $k = 5$,

$$(c_{5,0}, c_{5,1}, c_{5,2}) = (34, 0, 0).$$

This implies that $\Delta y_5 = 34 g_0$. In particular, Δy_5 depends solely on the embedded secret g_0 and is independent of the degree- $d/2$ zero-encoding randomness z_1 and z_2 . We conclude that the **LaOla** multiplication gadget is not fault robust for this particular parameter setting.

K Degree Reduction Matrix

In this section, we describe how to get the $n \times n$ matrix $\left(\lambda_{i,j}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \right)_{i,j \in [n]^2}$ that can reduce the degree of the underlying polynomial $f^{(d_{\text{in}})}(X) = \sum_{i=0}^{d_{\text{in}}} f_i X^i$ of shares $\left(F_i^{(d_{\text{in}})} \right)_{i \in [n]}$ from d_{in} to d_{out} , and obtain shares $\left(F_i^{(d_{\text{out}})} \right)_{i \in [n]}$ generated by polynomial $f^{(d_{\text{out}})}(X) = \sum_{i=0}^{d_{\text{out}}} f_i X^i$ with degree d_{out} . Let us assume that the support points for generating shares are $x_0, \dots, x_i, \dots$, and the secrets are the polynomial coefficients $\mathbf{f} = (f_0, f_1, \dots, f_{d_{\text{in}}})$.

Let's denote the inverse of the above Vandermonde matrix V^{-1} by

$$\text{Van}_{n \times n}^{-1}(x_0, \dots, x_{n-1}) = \begin{pmatrix} \lambda_{0,0} & \dots & \lambda_{0,d_{\text{in}}} & \lambda_{0,d_{\text{in}}+1} & \dots & \lambda_{0,n-1} \\ \lambda_{1,0} & \dots & \lambda_{1,d_{\text{in}}} & \lambda_{1,d_{\text{in}}+1} & \dots & \lambda_{1,n-1} \\ \lambda_{2,0} & \dots & \lambda_{2,d_{\text{in}}} & \lambda_{2,d_{\text{in}}+1} & \dots & \lambda_{2,n-1} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{d_{\text{in}},0} & \dots & \lambda_{d_{\text{in}},d_{\text{in}}} & \lambda_{d_{\text{in}},d_{\text{in}}+1} & \dots & \lambda_{d_{\text{in}},n-1} \\ \lambda_{d_{\text{in}}+1,0} & \dots & \lambda_{d_{\text{in}}+1,d_{\text{in}}} & \lambda_{d_{\text{in}}+1,d_{\text{in}}+1} & \dots & \lambda_{d_{\text{in}}+1,n-1} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \lambda_{n-1,0} & \dots & \lambda_{n-1,d_{\text{in}}} & \lambda_{n-1,d_{\text{in}}+1} & \dots & \lambda_{n-1,n-1} \end{pmatrix}_{n \times n}.$$

We *truncate* the coefficients $f_0, \dots, f_{d_{\text{in}}}$ to $f_0, \dots, f_{d_{\text{out}}}$, then multiply these coefficients with the Vandermonde matrix up to degree d_{out} , as follows,

$$\begin{pmatrix} F_0^{(d_{\text{out}})} \\ F_1^{(d_{\text{out}})} \\ F_2^{(d_{\text{out}})} \\ \vdots \\ F_{n-1}^{(d_{\text{out}})} \end{pmatrix}_n = \begin{pmatrix} 1 & x_0 & \dots & x_0^{d_{\text{out}}} \\ 1 & x_1 & \dots & x_1^{d_{\text{out}}} \\ 1 & x_2 & \dots & x_2^{d_{\text{out}}} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & \dots & x_{n-1}^{d_{\text{out}}} \end{pmatrix}_{n \times (d_{\text{out}}+1)} \cdot \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{d_{\text{out}}} \end{pmatrix} \quad (57)$$

$$= \begin{pmatrix} 1 & x_0 & \dots & x_0^{d_{\text{out}}} \\ 1 & x_1 & \dots & x_1^{d_{\text{out}}} \\ 1 & x_2 & \dots & x_2^{d_{\text{out}}} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & \dots & x_{n-1}^{d_{\text{out}}} \end{pmatrix}_{n \times (d_{\text{out}}+1)} \cdot \begin{pmatrix} \lambda_{0,0} & \dots & \lambda_{0,n-1} \\ \lambda_{1,0} & \dots & \lambda_{1,n-1} \\ \lambda_{2,0} & \dots & \lambda_{2,n-1} \\ \vdots & \ddots & \vdots \\ \lambda_{d_{\text{out}},0} & \dots & \lambda_{d_{\text{out}},n-1} \end{pmatrix}_{(d_{\text{out}}+1) \times n} \cdot \begin{pmatrix} F_0^{(d_{\text{in}})} \\ \vdots \\ F_{n-1}^{(d_{\text{in}})} \end{pmatrix} \quad (58)$$

$$= \begin{pmatrix} \lambda_{0,0}^{(d_{\text{in}} \rightarrow d_{\text{out}})} & \dots & \lambda_{0,n-1}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \\ \vdots & \vdots & \vdots \\ \lambda_{n-1,0}^{(d_{\text{in}} \rightarrow d_{\text{out}})} & \dots & \lambda_{n-1,n-1}^{(d_{\text{in}} \rightarrow d_{\text{out}})} \end{pmatrix}_{n \times n} \cdot \begin{pmatrix} F_0^{(d_{\text{in}})} \\ \vdots \\ F_{n-1}^{(d_{\text{in}})} \end{pmatrix} \quad (59)$$

L Degree of the Output of SplitRed is $\leq n - 1$

In the original paper [12], the variable $\hat{\lambda}_j^{(i)} = \left(\lambda_j^{(0)} + \sum_{k>d} \lambda_j^{(k)} \alpha_i^k \right)$ used in lines 8 and 18 of SplitRed is not only necessary to reconstruct and re-share the secret, but also to increase the order of the polynomial $(F'_i + F''_i)_{i \in [n]}$ in case the attacker has injected errors into the input shares $(F)_{i \in [n]}$.

More specifically, the secret can be reconstructed by using the first part of $\hat{\lambda}_j^{(i)}$, namely $\lambda_j^{(0)}$: The secret is then computed as $f_0 = \sum_{j=0}^{\frac{n}{2}} \lambda_j^{(0)} \cdot F_j + \sum_{j=\frac{n}{2}+1}^{n-1} \lambda_j^{(0)} \cdot F_j$ with the first addend, $\sum_{j=0}^{\frac{n}{2}} \lambda_j^{(0)} \cdot F_j$, being part of $(F'_i)_{i \in [n]}$, and the second addend, $\sum_{j=\frac{n}{2}+1}^{n-1} \lambda_j^{(0)} \cdot F_j$, being part of $(F''_i)_{i \in [n]}$. If we were to set $\hat{\lambda}_j^{(i)} := \lambda_j^{(0)}$, the degree of output polynomials of $(F'_i)_{i \in [n]}$, $(F''_i)_{i \in [n]}$ would be d because of the degree- d zero-encoding ZEnc in line 2 and 15.

However, when looking at the full computation of SplitRed, the degree of the output $(F'_i)_{i \in [n]}$, $(F''_i)_{i \in [n]}$ is $\leq n - 1$ instead of d due to the error detection term $\sum_{k>d} \lambda_j^{(k)} \alpha_i^k$. When no faults are injected, the error detection term is zero, i.e., $(F'_i + F''_i)_{i \in [n]}$ has degree $\frac{d}{2}$. However, note that we do not output the sum $(F'_i + F''_i)_{i \in [n]}$ but the two addends $(F'_i)_{i \in [n]}$, $(F''_i)_{i \in [n]}$ separately. Hence, half of the error detection term $(\sum_{k>d} \lambda_j^{(k)} \cdot \alpha_i^k \cdot F_j)_{j \in [\frac{n}{2}]}$ is added to $(F')_{i \in [n]}$, and the other half $(\sum_{k>d} \lambda_j^{(k)} \cdot \alpha_i^k \cdot F_j)_{j \in [\frac{n}{2}, n]}$ is added to $(F'')_{i \in [n]}$. It follows that even if we know that no faults were injected and the error detection term

$\sum_{i \in [n]} \sum_{j \in [n]} (\sum_{k > d} \lambda_j^{(k)} \cdot \alpha_i^k \cdot F_j) = \sum_{k > d} (f_k \cdot \sum_{i \in [n]} \alpha_i^k) = 0$ thus is zero, we cannot guarantee that the two separate parts of the error detection term are both still zero.

If the first part are not zero and added to $(F')_{i \in [n]}$, then the degree of $(F')_{i \in [n]}$ is $\leq n - 1$ instead of d , because when we add some non-zero terms (that are also not equal to each other) to a degree d polynomial $(F')_{i \in [n]}$ which would increase the degree of the polynomial to $\leq n - 1$. Similar things happen when the second part is added to $(F'')_{i \in [n]}$.

Therefore, the degree of the output polynomials $(F'_i)_{i \in [n]}, (F''_i)_{i \in [n]}$ is $\leq n - 1$ instead of d .

M Coefficients Calculation

The idea of the SplitRed algorithm is to create functions f', f'' such that $f'(0) + f''(0) = f(0)$, where f is of degree $\frac{d}{2}$ and encodes the same secret as the input element. Similarly, g', g'' are created with $g'(0) + g''(0) = g(0)$ and where g is of degree $\frac{d}{2}$. Let $\Delta^{i,j}$ denote the error polynomials related to the functions i and j . Hence, after the injection of errors, we obtain

$$\begin{aligned} h = & (f' + \Delta^{f',g'}) \cdot (g' + \Delta^{g',f'}) + (f'' + \Delta^{f'',g'}) \cdot (g' + \Delta^{g',f''}) \\ & + (f' + \Delta^{f',g''}) \cdot (g'' + \Delta^{g'',f'}) + (f'' + \Delta^{f'',g''}) \cdot (g'' + \Delta^{g'',f''). \end{aligned} \quad (60)$$

Since the error polynomials both have a degree of up to $n - 1$, the polynomial h is calculated by adding multiple products of degree- $(n - 1)$ polynomials. This results in a polynomial of degree $2n - 2$. However, note that we only consider n support points. Given n support points, one can only interpolate a degree- $(n - 1)$ polynomial, we thus *lose information* about the originally embedded polynomial. Luckily, this is not a problem, as these n shares are enough information to still embed the correct secret. However, when analyzing the corresponding polynomials, we need to take care that we look at the correct polynomial. Let h denote the degree- $(2n - 2)$ polynomial we obtain after multiplication, as written in Equation 60. By $\check{h}$ we denote the degree- $(n - 1)$ polynomial which satisfies $h(x_i) = \check{h}(x_i)$ for all $i \in [n]$. We can calculate the coefficients of $\check{h}$ as

$$\check{h}^T = \underbrace{\text{Van}_{[n] \times [n]}^{-1}(x_0, \dots, x_{n-1})}_{\text{interpolating over the shares to obtain a degree-}(n-1)\text{ polynomial}} \cdot \overbrace{(\text{Van}_{[n] \times [2n-1]}(x_0, \dots, x_{n-1}) \cdot h^T)}^{\text{evaluating the degree-}(2n-2)\text{ polynomial at } n \text{ points}} \quad (61)$$

where we use $\mathbf{Van}_{[n] \times [2n-1]}(x_0, \dots, x_{n-1})$ to consider all coefficients $0, \dots, 2n-1$. Since matrix multiplication is associative, we can define

$$\begin{aligned}
\tilde{V} &= \mathbf{Van}_{[n] \times [n]}^{-1}(x_0, \dots, x_{n-1}) \cdot \mathbf{Van}_{[n] \times [(2n-1)]}(x_0, \dots, x_{n-1}) \\
&= \begin{pmatrix} \sum_{j=0}^{n-1} \lambda_{0,j} x_j^0 & \sum_{j=0}^{n-1} \lambda_{0,j} x_j^1 & \dots & \sum_{j=0}^{n-1} \lambda_{0,j} x_j^{2n-2} \\ \sum_{j=0}^{n-1} \lambda_{1,j} x_j^0 & \sum_{j=0}^{n-1} \lambda_{1,j} x_j^1 & \dots & \sum_{j=0}^{n-1} \lambda_{1,j} x_j^{2n-2} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{j=0}^{n-1} \lambda_{n-1,j} x_j^0 & \sum_{j=0}^{n-1} \lambda_{n-1,j} x_j^1 & \dots & \sum_{j=0}^{n-1} \lambda_{n-1,j} x_j^{2n-2} \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 & \dots & 0 & \sum_{j=0}^{n-1} \lambda_{0,j} x_j^n & \dots & \sum_{j=0}^{n-1} \lambda_{0,j} x_j^{2n-2} \\ 0 & 1 & 0 & \dots & 0 & \sum_{j=0}^{n-1} \lambda_{1,j} x_j^n & \dots & \sum_{j=0}^{n-1} \lambda_{1,j} x_j^{2n-2} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & \sum_{j=0}^{n-1} \lambda_{n-1,j} x_j^n & \dots & \sum_{j=0}^{n-1} \lambda_{n-1,j} x_j^{2n-2} \end{pmatrix} \\
&= (\text{Id}_{n \times n} \ \overline{V})
\end{aligned}$$

where $\lambda_{i,j}$ is the element in the i -th row and j -th column of $\mathbf{Van}_{[n] \times [n]}^{-1}(x_0, \dots, x_{n-1})$ and $\overline{V}$ denotes an $n \times n$ matrix. Altogether, this allows us to write $\check{h}$ as

$$\check{h}^T = \tilde{V} \cdot h^T = \left(\sum_{i=0}^{2n-2} \tilde{V}_{0,i} h_i, \dots, \sum_{i=0}^{2n-2} \tilde{V}_{n-1,i} h_i \right)^T.$$